Optimisation of Computational Intelligence Techniques for Hyperspectral Image Segmentation

Mzoxolo Mbini

A Thesis
Submitted in fulfilment of the requirements for the degree
Doctor of Philosophy
(Computer Science)



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Faculty of Engineering, Built Environment and IT
School of Information Technology
Department of Computer Science
University of Pretoria
Pretoria, South Africa
17th August 2026

Contents

Page

1 Introduction **1**
- 1.1 Purpose of the study 1
- 1.2 Research Objectives 2
- 1.3 Scope of the Study 2
- 1.4 Contributions 3
- 1.5 Outline of the Thesis 4

2 Computational Intelligence Approaches **6**
- 2.1 Introduction 6
- 2.2 Single-Point Search Approaches 7
- 2.3 Multipoint Search Approaches 8
- 2.4 Comparative Summary 12
- 2.5 Conclusion 12

3 Hyperspectral Imaging **13**
- 3.1 Introduction 13
- 3.2 Characteristics of Hyperspectral Data 13
- 3.3 Applications of Hyperspectral Imaging 15
- 3.4 Challenges for Hyperspectral Segmentation 15
- 3.5 Conclusion 17

4 Image Segmentation **18**
- 4.1 Introduction 18
- 4.2 Detailed Analysis of Traditional Image Segmentation Methodologies 18
- 4.3 Deep Learning Approaches 23
- 4.4 Hybrid Segmentation Techniques 26
- 4.5 Performance Metrics 26
- 4.6 Conclusion 27

5 Research Methodology **29**
- 5.1 Research Methodology 29
- 5.2 Datasets 31
- 5.3 Performance Evaluation 32
- 5.4 Technical Specifications 34

6 Computational Intelligence for Image Segmentation **35**
- 6.1 Overview 35
- 6.2 Multilevel Thresholding 35
- 6.3 Watershed Transform 37
- 6.4 Holistically-Nested Edge Detection 40
- 6.5 Chapter Summary 41

7 Automated Design of Multilevel Thresholding **42**
- 7.1 Introduction 42
- 7.2 Motivation for Automated Design 42
- 7.3 Automated Design Framework 43
- 7.4 Experimental Design 45
- 7.5 Conclusion 46

8 Watershed Transform Hybridisation **47**
8.1 Introduction 47
8.2 Motivation for Watershed Enhancement 47
8.3 Enhanced Watershed Framework 48
8.4 Experimental Design 49
8.5 Conclusion 50

9 Holistic-Nested Edge Detection Enhancement **51**
9.1 Introduction 51
9.2 Motivation for Deep Learning Enhancement 51
9.3 Enhanced Deep Learning Framework 52
9.4 System Architecture 54
9.5 Experimental Design 55
9.6 Conclusion 55

10 Results **56**
10.1 Introduction 56
10.2 Experimental Setup 56
10.3 Multilevel Thresholding Results 56
10.4 Watershed Transform Results 58
10.5 Holistic-Nested Edge Detection Results 61
10.6 Cross-Approach Comparison 65
10.7 Per-Class Performance and Rare Class Analysis 67
10.8 Computational Efficiency 69
10.9 Convergence Analysis 69
10.10Conclusion 70

11 Discussions and Recommendations **71**
11.1 Introduction 71
11.2 Summary of Findings 71
11.3 Discussion 72
11.4 Recommendations 75
11.5 Limitations 75
11.6 Future Work 76
11.7 Conclusion 77

Chapter 1

Introduction

The human eye perceives the world in three chromatic categories: red, green, and blue (RGB) [97]. All other colours can be generated by adjusting the amounts of these three hues. This principle underlies the functioning of devices such as television screens, computer monitors, mobile devices, and colour printers. A typical camera captures images in the RGB spectrum. However, the limitations of RGB imaging become evident when attempting to capture detailed information regarding material properties, as RGB sensors can only record a narrow portion of the electromagnetic spectrum [97]. To overcome this limitation, hyperspectral imaging has been developed to capture a much broader range of spectral information [5]. An RGB pixel is defined by its spatial location (x, y) and a triplet of intensities (R, G, B). However, a hyperspectral pixel replaces the simple RGB triplet with a continuous spectral vector. Hyperspectral imaging captures a much broader range of light and divides it into narrow spectral bands. Each element of this spectral vector corresponds to the measured intensity of the reflected or emitted light at a specific wavelength, that is, at a specific position within the electromagnetic spectrum defined by the distance between successive wave peaks. Hyperspectral imaging records detailed spectral signatures for materials across the electromagnetic spectrum, rather than compressing this information into three broad colour channels. These spectral signatures capture subtle differences in the material composition that are not visible in RGB images [5]. Consequently, hyperspectral images enable more accurate identification and classification of materials based on their unique spectral responses [7].

Before applying complex analysis, computer vision systems use image processing to standardise data and remove redundancies, ensuring that subsequent algorithms operate on the most relevant visual information. One such image processing task is image segmentation [44, 46]. Image segmentation refers to the process of partitioning a digital image into multiple meaningful regions, each consisting of pixels that share similar characteristics. While object detection identifies the location of an object using a bounding box, segmentation determines the exact pixels that belong to that object.
Because hyperspectral images encode both spectral and spatial information, they cannot be handled in the same way as standard RGB images. Therefore, effective HSI segmentation must use methods that consider the relationships between spectral bands and the spatial consistency between neighbouring pixels. Segmentation methods designed for RGB images, which mainly rely on spatial features or single intensity values, are not suitable for this type of data, creating the need for segmentation approaches specifically developed for hyperspectral imagery.

1.1 Purpose of the study

Hyperspectral image (HSI) segmentation is challenging because of the complex data and differences between regions in real-world images [49]. Many existing segmentation methods rely on a fixed global strategy that applies the same processing assumptions to an entire image [30]. Although such approaches are often computationally efficient, they struggle to adapt to local variations in spectral and spatial characteristics, resulting in suboptimal segmentation performance in complex scenes [90].

Deep learning-based approaches have achieved good results in HSI segmentation by directly learning complex patterns from data [79]. However, these methods usually require large amounts of carefully labelled training data and high computing power for training and deployment [23, 34]. In practical applications such as remote sensing, environmental monitoring, and agriculture, these requirements

limit their use because labelled data are expensive, ground-truth data are limited, and real-world deployment environments have resource constraints [48].
The purpose was to replace manual parameter tuning with automated computational intelligence search strategies that optimise global pipeline parameters per dataset, eliminating the need for expert trial-and-error while maintaining competitive accuracy and efficiency on the majority land-cover classes. This thesis is motivated by, but does not claim to solve, the applied problems of precision agriculture and mineral exploration, since the economically important classes in both domains are typically rare classes occupying a small fraction of labelled pixels. As shown in Section 10.7, none of the frameworks developed in this thesis achieve reliable rare-class detection, with per-class F1 scores below 0.31 for classes occupying less than 0.5% of labelled pixels across every method tested. The contribution of this thesis is therefore majority-class segmentation performance and efficiency under automated design, established as infrastructure that a rare-class-aware extension, outlined in Section 11.6, would need to build upon before deployment in either motivating application. Rare-class performance is analysed explicitly in Section 10.7 to expose this limitation and guide future work.

The proposed framework does not assert that it can deliver superior performance while using fewer labelled samples than current supervised approaches. All experiments used a 70%/30% stratified split that provided a generous quantity of labelled training data. Rather, the claim is that the enhanced methods reduce the need for *manual engineering effort*: parameter values that would otherwise require expert trial and error are automatically discovered by the search strategy. The question of whether these frameworks retain their relative advantage over the baselines at lower labelling densities is identified as a direction for future work in Section 11.6.

1.2 Research Objectives

The objectives of this study were as follows.

1. To design and develop a baseline multilevel thresholding for segmenting hyperspectral images.
2. To design and develop automated design strategies for multilevel thresholding for segmenting hyperspectral images.
3. To design and develop a baseline watershed-based approach for segmenting hyperspectral images.
4. To design and develop a hybrid watershed approach for segmenting hyperspectral images.
5. To design and develop a baseline deep learning approach for segmenting hyperspectral images.
6. To design and develop an enhanced baseline deep learning approach for segmenting hyperspectral images.

1.3 Scope of the Study

This study focuses on hyperspectral image segmentation using a combination of classical and learning-based techniques. Its scope is limited to deep learning, watershed, and thresholding approaches, along with enhanced versions of each method. These enhanced approaches were designed to improve segmentation performance by incorporating local adaptive processing strategies.

Benchmark hyperspectral datasets commonly used in remote sensing research were used. The analysis was restricted to offline processing; real-time or embedded system implementations were not considered in this study. In addition, this study focuses on segmentation performance and does not address higher-level tasks, such as object recognition or change detection.

The proposed methods were evaluated using standard segmentation performance metrics, including per-class F1 scores and macro-averaged metrics, to assess minority class performance. Issues related to sensor design, data acquisition, and manual ground truth generation were not considered. Instead, the focus was on comparing the effectiveness and computational efficiency of the enhanced methods with their corresponding baseline approaches under controlled experimental conditions.

A critical scope limitation: Although this study is motivated by applications in precision agriculture and mineral exploration, the proposed frameworks are not designed for rare-class detection in their current form. As shown in Section 10.7, classes occupying fewer than 0.5% of labelled pixels achieve F1 scores below 0.31 across all methods, which is inadequate for applications where these classes are the analytical targets. Therefore, this study focuses on majority-class segmentation performance and explicitly identifies rare-class handling as a required extension (Section 11.6) before deployment in rare-class-critical applications. Practitioners requiring rare-class detection should treat the current frameworks as baseline infrastructure rather than a complete solution.
A key practical constraint addressed in this study is the enhancement of deployed models without retraining. Contemporary deep architectures (SegFormer, HyCoT) are discussed in Chapter 4 as a contextual background but are not employed as direct baselines because they require full retraining and millions of parameters, violating the no-retraining constraint central to the enhanced deep learning framework developed in this study.

1.4 Contributions

The main contribution of this study is a set of hyperspectral image segmentation frameworks that replace fixed, manually chosen global processing strategies with automated, search-guided configurations selected for each dataset. Unlike traditional methods that rely on hand-tuned defaults, these frameworks discover their parameter or feature settings through automated per-dataset search rather than relying on hand-tuned defaults. As Chapter 7 shows, automated configuration is not applied uncritically in this thesis, and its costs and benefits are evaluated rather than being assumed. Section 11.3.4 discusses the extent to which this automation operates at the dataset level rather than within the individual scenes.
This study makes the following contributions.

1. **Algorithm configuration for thresholding:** A budget-equalised comparison of single-point and multipoint algorithm configurators applied to a baseline GA/ABC multilevel thresholding optimiser: a search-enhanced trajectory method (ILS) that augments the threshold search directly, and a multipoint meta-genetic configurator that tunes the base optimiser's hyperparameters via an outer search loop. Unlike the other two contributions, this comparison shows that automated configuration does not uniformly improve on a well-chosen baseline: the search-enhanced variant reproduces baseline accuracy at substantially lower computational cost, while the multipoint configurator underperforms the baseline due to an evaluation budget insufficient to recover its own configuration overhead. This is itself a contribution: a controlled empirical demonstration of when automated design pays for itself and when it does not.

2. **Evolutionary metaheuristic watershed optimisation:** A Genetic Algorithm framework that co-optimises five components of the watershed pipeline simultaneously (PCA dimensionality, smoothing, gradient thresholding, minimum region size, and classifier selection). While sharing the adaptive spirit of hyper-heuristics, this is an evolutionary meta-heuristic that automates the configuration of the operator pipelines.

3. **Hyper-heuristic deep learning enhancement:** A strict hyper-heuristic selection framework that uses Iterative Local Search to select optimal classical feature detector configurations from a space of 5,184 low-level heuristics, augmenting a fixed pre-trained HED model without retraining. This satisfies the Burke et al. [9] definition of hyper-heuristic selection, which operates over heuristic configurations rather than domain parameters.

4. **Comparative evaluation under shared protocol:** A systematic comparison of the enhanced deep learning and watershed approaches, and the automated thresholding configurators, against their respective baselines under a consistent 70%/30% stratified split protocol, including per-class F1 analysis and computational efficiency assessment.

These findings offer practical insights into segmentation strategies that balance performance and efficiency in real-world hyperspectral image applications.

Why These Three Frameworks Together

A natural question is why these approaches were studied in a single thesis rather than independently. The answer is not merely organisational. The enhanced deep learning framework, enhanced

watershed framework, and automated thresholding configurators occupy distinct positions on the automation spectrum. The deep learning framework is a strict hyper-heuristic selector that operates in a space of predefined low-level heuristic configurations. The watershed framework is an evolutionary metaheuristic that directly optimises pipeline parameters. The thresholding strategies are algorithm configurators in the sense of Hutter et al. [31], tuning the hyperparameters of a base GA/-ABC optimiser rather than the segmentation output itself. Studying them together under a shared evaluation protocol allows for a cross-paradigm comparison that neither study could support independently. Specifically, evaluating all of them on the same five benchmark datasets under the same 70%/30% split makes it possible to answer a question that transcends any individual method: *given a fixed application context defined by accuracy requirement, inference speed, and deployment constraint, which class of adaptive automated search strategy is the most appropriate, and under what conditions does automated search actually improve on a well-chosen baseline rather than merely adding cost?* The recommendations in Chapter 11 are grounded in this cross-paradigm comparison rather than per-method tuning. This is the unified contribution of the frameworks studied in this thesis.

1.5 Outline of the Thesis

Having set out what the study contributes and why these frameworks are studied together, the remainder of this thesis is structured as follows.

1.5.1 Chapter 2: Computational Intelligence Approaches

This chapter introduces the computational intelligence techniques used throughout this thesis: metaheuristic optimisers (genetic algorithms, artificial bee colony, particle swarm optimisation, differential evolution, and simulated annealing), iterated local search, and the terminological distinction between strict hyper-heuristic selection, evolutionary metaheuristics, and algorithm configuration. It establishes the theoretical foundation on which the later baseline and automated design chapters are built.

1.5.2 Chapter 3: Hyperspectral Imaging

This chapter introduces hyperspectral imaging, explains the unique properties of hyperspectral data, and reviews key application areas where hyperspectral analysis is important. It also discusses the major challenges of hyperspectral image segmentation and evaluates the existing segmentation approaches.

1.5.3 Chapter 4: Image Segmentation

This chapter presents a comprehensive overview of image segmentation, covering classical methods, deep learning approaches, and hybrid techniques, along with their principles, strengths, and limitations. It identifies the key challenges in applying these methods to complex, high-dimensional data. This chapter also clarifies the distinction between strict hyper-heuristic selection, evolutionary metaheuristic optimisation, and algorithm configuration as they apply specifically to segmentation, establishing the terminological framework used throughout this thesis.

1.5.4 Chapter 5: Research Methodology

This chapter outlines the research methodology used to design, implement, and evaluate the proposed hyperspectral image segmentation method. It introduces the benchmark hyperspectral datasets, describes the evaluation metrics (including per-class F1 and macro-averaged metrics), and the experimental protocol used to compare the baseline and enhanced methods, and concludes with the implementation details and computational settings used in the experiments.

1.5.5 Chapter 6: Computational Intelligence for Image Segmentation

This chapter presents the baseline implementation of each of the three segmentation paradigms studied in this thesis: multilevel thresholding, the watershed transform, and Holistically-Nested Edge Detection (HED). For each, the problem formulation and algorithmic implementation are described, followed by the experimental protocol used for its evaluation. No automated design or

hybridisation strategies are introduced in this chapter; these are presented separately in Chapters 7, 8, and 9.

1.5.6 Chapter 7: Automated Design of Multilevel Thresholding

This chapter presents the automated design strategies applied to the baseline multilevel thresholding optimisers introduced in Chapter 6: a search-enhanced trajectory method (ILS), a single-point configurator (simulated annealing), and two multipoint configurators (a meta-genetic algorithm and differential evolution), evaluated under a strictly equalised evaluation budget. The taxonomy of design strategies, their mechanisms, and the experimental protocol used to evaluate them are described.

1.5.7 Chapter 8: Watershed Transform Hybridisation

This chapter introduces the automated design strategies applied to the baseline watershed pipeline introduced in Chapter 6: a particle-swarm-optimised benchmark that automates a single gradient-threshold parameter, and a genetic-algorithm-based framework that jointly optimises preprocessing, segmentation, and classification parameters. The chapter details how these adaptive strategies are integrated into the watershed pipeline and describes the experimental protocol used to evaluate them.

1.5.8 Chapter 9: Holistic-Nested Edge Detection Enhancement

This chapter presents the hyper-heuristic hybridisation strategy applied to the baseline HED model introduced in Chapter 6: a fixed deep backbone selectively augmented with parameterised classical edge detectors, combined through a learned fusion mechanism, and selected via an iterated local search hyper-heuristic. The chapter describes this framework and the experimental protocol used to evaluate it.

1.5.9 Chapter 10: Results

This chapter presents and analyzes the experimental results obtained for all segmentation approaches: order thresholding, watershed, and deep learning. The performances of the baseline and automated design or hybridisation strategies are compared in terms of segmentation accuracy, per-class F1 scores, robustness, and computational efficiency, and the results are discussed in relation to the objectives of this study.

1.5.10 Chapter 11: Discussions and Recommendations

This chapter summarises the main findings and contributions of this study, highlighting the conditions under which adaptive design and hybridisation strategies improve hyperspectral image segmentation over a well-chosen baseline, and the conditions under which they do not. It also discusses the limitations of the current study and outlines directions for future research.

Chapter 2

Computational Intelligence Approaches

2.1 Introduction

Computational intelligence provides a family of search strategies that can find good solutions to problems where the search space is too large to explore exhaustively and where no closed-form analytical solution exists. Rather than checking every possible configuration, these strategies use a guided, iterative process to move from an initial solution toward better ones, using feedback from a fitness or objective function to steer the search.
Search strategies of this kind are broadly divided into two families, distinguished by the number of candidate solutions they maintain and evaluate at any one time [33, 45]. *Single-point search* strategies maintain and refine one candidate solution at a time, moving through the search space along a single trajectory. *Multipoint search* strategies maintain a population of candidate solutions simultaneously and exploit the interactions between them, either through recombination and selection, as in evolutionary algorithms, or through shared positional information, as in swarm intelligence. Table 2.1 summarises this taxonomy and lists the specific algorithms covered in this chapter.

Table 2.1: Taxonomy of computational intelligence search strategies considered in this chapter.

Family	Category	Algorithm
Single-point	Trajectory-based	Simulated Annealing
Single-point	Trajectory-based	Iterative Local Search (ILS)
Multipoint	Evolutionary algorithm	Genetic Algorithm (GA)
Multipoint	Evolutionary algorithm	Differential Evolution (DE)
Multipoint	Swarm intelligence	Particle Swarm Optimisation (PSO)
Multipoint	Swarm intelligence	Artificial Bee Colony (ABC)

A further distinction relevant to the application of these strategies to a problem is the distinction between *hyper-heuristics* and *metaheuristics* that operate directly on the domain parameters. A hyper-heuristic searches over a space of low-level heuristics or heuristic configurations using a high-level strategy to decide which heuristics to apply, rather than searching the problem's decision variables directly [9, 10]. In contrast, a metaheuristic typically directly searches over the domain variables of the problem itself. Both framings are compatible with either a single-point or multipoint search strategy; the distinction is not in the number of candidate solutions maintained but in what those candidate solutions represent: a configuration of heuristics in the hyper-heuristic case and a direct assignment of domain parameters in the metaheuristic case. The hyper-heuristic search has been applied across a range of problem domains, including feature selection [59], medical image processing [19], image segmentation [80], and problems requiring adaptive parameter control [18] or multi-objective formulations [39].

2.2 Single-Point Search Approaches

Single-point search strategies operate on one candidate solution at a time. At each iteration, the current solution is modified in some way, and the new solution is evaluated against a fitness or cost function. Depending on the outcome of this comparison and the specific acceptance rule used by the algorithm, the new solution either replaces the current one or is discarded. Because only one solution is held in memory at any time, these methods are computationally lightweight compared with population-based approaches. However, they are also more prone to becoming trapped in local optima unless an explicit mechanism for escaping them is built into the algorithm.

2.2.1 Simulated Annealing

Simulated Annealing (SA) is a single-point search strategy inspired by the physical process of annealing in metallurgy, where a material is heated and then slowly cooled to reduce defects in its crystal structure [101]. At high temperatures, the physical system can move freely between energy states, including states of higher energy. As the temperature decreases, the system becomes increasingly restricted to low-energy states, eventually settling near a minimum.
In the optimisation analogue, a candidate solution x plays the role of the physical state, and the value of the objective function $f(x)$ plays the role of the energy. At each iteration, a neighbouring solution x' is generated by perturbing x. If x' improves on x, it is accepted unconditionally. If it does not, it is still accepted with a probability that depends on the size of the degradation and the current temperature T:

$$P(\text{accept } x') = \exp\left(-\frac{f(x') - f(x)}{T}\right). \tag{2.1}$$

The temperature T is reduced according to a cooling schedule, most commonly geometric, $T_{k+1} = \alpha T_k$ with $\alpha \in (0.8, 0.99)$ over the course of the search. Early in the run, a high temperature allows the algorithm to accept worsening moves and explore broadly. As the temperature falls, the algorithm increasingly behaves like a greedy local search, converging onto a single basin of attraction.
The principal strength of SA is its explicit and theoretically grounded mechanism for escaping local optima: unlike a pure greedy local search, it can, with some probability, move uphill in the objective landscape. This makes it well-suited to problems with many shallow local optima. Its principal weaknesses are practical rather than theoretical. The performance of the algorithm is highly sensitive to the choice of cooling schedule. A schedule that cools too quickly behaves like a greedy search with none of SA's exploratory benefit, whereas a schedule that cools too slowly is computationally wasteful. In addition, SA explores the search space along a single trajectory; therefore, in the same computational budget, it samples fewer distinct regions of the space than a population-based method.

2.2.2 Iterative Local Search

Iterative Local Search (ILS) is a single-point search strategy that escapes local optima structurally, rather than probabilistically, as Simulated Annealing does, by explicitly perturbing an already-converged local optimum and restarting a local search from that perturbed point [57]. The general ILS procedure consists of four components: an initial solution, a local search procedure, a perturbation operator, and an acceptance criterion.

1. **Initial solution.** A starting solution s_0 is generated either randomly or by a constructive heuristic.

2. **Local search.** A local search procedure is applied to s_0 until no further improving move can be found within its neighbourhood, producing a local optimum s^*.

3. **Perturbation.** A perturbation operator is applied to s^* to produce a new starting point s'. The perturbation must be sufficiently strong to move the search out of the current basin of attraction, but sufficiently weak that the local search from s' still benefits from the structure already discovered in s^*.

4. **Acceptance criterion.** The local search is applied to s', producing a new local optimum $s^{*\prime}$. An acceptance criterion then decides whether the search continues from $s^{*\prime}$ or reverts to s^*. The simplest and most common criterion is "better", in which $s^{*\prime}$ is accepted only if $f(s^{*\prime}) > f(s^*)$.

This cycle of perturb, locally search, and accept-or-reject repeats until a termination condition is reached, typically a maximum number of iterations or a patience threshold on the number of iterations without improvement. Algorithm 1 provides the generic ILS procedure in this form.

Algorithm 1: Generic Iterative Local Search

Input: Initial solution generator, local search procedure $LS(\cdot)$, perturbation operator $Perturb(\cdot)$, maximum iterations I, patience P
Output: Best solution found s_{best}

```
s_0 ← GenerateInitialSolution();
s* ← LS(s_0);
s_best ← s*;
no_improve ← 0;
for i = 1 to I do
    s' ← Perturb(s*);
    s*' ← LS(s');
    if f(s*') > f(s*) then
        s* ← s*';
        if f(s*) > f(s_best) then
            s_best ← s*;
            no_improve ← 0;
        end
    else
        no_improve ← no_improve + 1;
    end
    if no_improve ≥ P then
        break;
    end
end
return s_best
```

Advantages. ILS has three properties that make it attractive relative to other single-point and population-based strategies. First, it is conceptually simple and has very few control parameters compared with population-based methods, essentially only the perturbation strength and the acceptance criterion, which reduces the tuning burden. Second, because ILS perturbs an already-good solution rather than generating a fresh random one at each restart, it retains structural information across iterations; this is well suited to discrete, combinatorial configuration spaces, where a good configuration typically differs from another good configuration by only one or two component choices. Third, ILS tends to converge quickly in practice on such spaces, because each perturbation explores only a bounded neighbourhood around a solution already known to be good, rather than restarting the search from an arbitrary point. ILS has also been used directly within the image segmentation domain, for example, to automate the design and configuration of a genetic algorithm applied to segmentation tasks [66], illustrating that single-point and population-based strategies can be combined rather than treated as mutually exclusive choices.

Disadvantages. The same single-trajectory property that gives the ILS its efficiency also limits it. Because only one candidate solution is maintained, ILS cannot exploit the type of population-level recombination that allows an evolutionary algorithm to combine good partial solutions from two unrelated regions of the search space in a single step; when a discrete configuration space has several widely separated good regions, ILS may find only one of them, depending on where it started. ILS is also more sensitive than population-based methods to the design of its perturbation operator: too weak a perturbation and the search re-converges to the same local optimum it just left; too strong a perturbation and the search effectively restarts from elsewhere, losing the benefit of the local structure already found. Finally, ILS, like Simulated Annealing, is a sequential procedure and does not parallelise across candidate evaluations as naturally as a population-based method does, since each perturb-search-accept cycle depends on the outcome of the previous one.

2.3 Multipoint Search Approaches

Multipoint search strategies maintain a population of candidate solutions and evaluate them at each iteration. Because the population as a whole covers a much wider area of the search space than

a single point can, these methods are generally more robust against local optima than single-point methods, at the cost of requiring more fitness evaluations per iteration and a larger set of control parameters. Multipoint strategies are further divided into two families: evolutionary algorithms, which are inspired by biological reproduction and natural selection, and swarm intelligence, which is inspired by the collective, decentralised behaviour of social organisms.

2.3.1 Evolutionary Algorithms

Evolutionary algorithms maintain a population of candidate solutions called individuals or chromosomes and evolve this population over successive generations using operators modelled on biological reproduction: selection, recombination (crossover), and mutation. Fitter individuals are more likely to be selected as parents, and the resulting offspring inherit and recombine traits from their parents, with mutation introducing new variation. Over many generations, the population as a whole tends to drift toward fitter regions of the search space, while crossover and mutation continue to supply the diversity needed to avoid premature convergence.

Genetic Algorithms

The Genetic Algorithm (GA) is the most established evolutionary algorithm. In a GA, each individual in the population is encoded as a chromosome, most commonly a fixed-length vector of genes, where each gene represents one decision variable of the problem being optimised. A GA proceeds through the following stages at each generation:

1. **Selection.** Individuals are chosen to become parents with a probability related to their fitness, such that fitter individuals are more likely, although not certain, to reproduce. Common schemes include tournament selection, in which a small random subset of the population competes, and the fittest member wins, and steady-state selection, in which the best individuals from the current generation are directly carried forward.

2. **Crossover.** Pairs of parent chromosomes exchange genetic material to produce offspring. In single-point crossover, a cut point is chosen randomly along the chromosome, and the genes on either side of that point are swapped between the two parents. Crossover is applied with a probability p_c, which is typically in the range $[0.6, 0.9]$.

3. **Mutation.** Individual genes in the offspring are perturbed at random with a small probability p_m, typically in the range $[0.01, 0.2]$, to introduce variation that crossover alone cannot produce and to help the search escape local optima [53].

4. **Replacement and elitism.** The new generation is formed from the offspring, usually with a small proportion of the fittest individuals from the previous generation carried over unchanged (elitism), which guarantees that the best solution found to date is never lost.

Advantages. Because a GA searches with an entire population rather than a single point, it naturally maintains diversity across many regions of the search space simultaneously, which makes it considerably more robust than single-point methods against multimodal fitness landscapes, that is, landscapes with several widely separated good regions. GA is also comparatively easy to adapt to mixed variable types, real-valued, integer, and categorical genes coexisting on the same chromosome, and to problems in which several qualitatively different decision variables must be optimised jointly. Genetic algorithms have been applied directly to hyperspectral image segmentation, including multi-objective formulations that optimise several segmentation quality criteria simultaneously rather than a single combined score [61].

Disadvantages. A GA requires many more fitness evaluations per generation than a single-point method, since every individual in the population must be evaluated at every generation. This cost is compounded whenever a single fitness evaluation is expensive to compute. A GA also introduces more control parameters than a single-point method, population size, crossover rate, mutation rate, and elitism rate, each of which can influence the final result and typically requires some sensitivity analysis to select responsibly. Finally, because a GA's crossover operator recombines entire genes rather than reasoning about their individual contribution, it can be less sample-efficient than a targeted local-search method on small, well-structured discrete spaces where good solutions are known to lie close together.

Differential Evolution

Differential Evolution (DE) is a second evolutionary algorithm, distinguished from the standard GA primarily by the way it generates new candidate solutions. Where a GA relies on crossover between two parents to generate offspring, DE generates a mutant vector directly from the vector difference between other members of the population, which gives it a strong and well-documented ability to explore continuous search spaces efficiently. DE has been applied specifically to multilevel image segmentation, for example by optimising threshold vectors against a minimum cross-entropy criterion [76].
For each individual x_i in the population, DE selects three other distinct individuals, x_{r1}, x_{r2}, and x_{r3}, and constructs a mutant vector

$$v_i = x_{r1} + F \cdot (x_{r2} - x_{r3}), \tag{2.2}$$

where F is the mutation factor, typically 0.5. The difference vector $(x_{r2} - x_{r3})$ supplies both a search direction and a step size that automatically adapts to the current spread of the population: early in the search, when the population is diverse, the differences are large and the search explores broadly; as the population converges, the differences shrink and the search naturally refines its estimate. The trial vector u_i is then formed by crossing the mutant vector with the original target vector on a per-dimension basis, with each gene taken from the mutant vector with probability CR (the crossover rate) and from the target vector otherwise, subject to the constraint that at least one gene is always taken from the mutant. The trial vector replaces the target vector in the next generation only if it achieves an equal or better fitness, a form of greedy selection that guarantees that the population's best fitness never degrades.
Advantages. DE's difference-vector mutation is self-adaptive in step size, which typically gives it faster and more reliable convergence than a standard GA on continuous, real-valued search spaces. DE also has very few control parameters, essentially only F and CR, both of which are comparatively easy to tune and for which well-established default ranges exist in the literature.

Disadvantages. DE's principal weakness is a tendency towards premature convergence: as the population's differences shrink, DE's own exploratory mechanism weakens along with them, and the population can settle on a suboptimal region of the search space before it has been adequately explored, an effect that becomes more pronounced under certain combinations of population size and mutation strategy [98]. DE also has no dedicated mechanism analogous to a GA's crossover for recombining structurally distinct partial solutions, since its variation operator always operates through vector arithmetic rather than gene exchange. Hybridising DE with a GA, alternating differential mutation for exploration with GA-style selection, crossover, and mutation for exploitation and diversity maintenance, is one established way of offsetting this weakness.

2.3.2 Swarm Intelligence

Swarm intelligence methods are population-based. However, rather than modelling reproduction and selection, they model the decentralised, emergent coordination seen in social organisms, such as flocks of birds or colonies of insects, where no individual has global knowledge of the search space, but simple local rules of interaction between individuals produce effective collective search behaviour.

Particle Swarm Optimisation

Particle Swarm Optimisation (PSO) models a population or swarm of particles that move through the search space, with each particle's movement influenced by its own best-found position and the best position found by the swarm as a whole [78]. Each particle i has a position x_i and a velocity v_i, updated at each iteration according to

$$v_i^{t+1} = \omega v_i^t + c_1 r_1 \left(pbest_i - x_i^t\right) + c_2 r_2 \left(gbest - x_i^t\right), \qquad x_i^{t+1} = x_i^t + v_i^{t+1}, \tag{2.3}$$

where ω is the inertia weight, c_1 and c_2 are acceleration constants that determine how strongly the particle is pulled toward its own historical best position, $pbest_i$, and the swarm's historical best position, $gbest$, respectively, and r_1, r_2 are random numbers drawn uniformly from $[0, 1]$ at each iteration to introduce stochastic variation. PSO has also been applied directly to hyperspectral image analysis tasks, including band selection, where a particle represents a candidate subset or weighting

of spectral bands [107, 109].

Advantages. PSO has a small number of control parameters, converges quickly in practice on unimodal or mildly multimodal continuous problems, and its velocity term provides it with a form of search momentum that a GA's crossover operator does not have, allowing particles to continue moving productively in a promising direction across several iterations.
Disadvantages. Because every particle is continuously pulled towards the single global best position *gbest*, PSO is prone to premature convergence on multimodal landscapes: once a sufficiently good position is found, the whole swarm tends to collapse towards it, reducing diversity before the wider search space has been adequately explored, a convergence behaviour that has been analysed formally in terms of particle trajectory stability [16, 64]. PSO is also, in its standard form, naturally suited to optimising a single continuous variable or a small set of jointly continuous variables, rather than a large, mixed-type, multi-component configuration problem involving categorical or structurally distinct decision variables.

Artificial Bee Colony

Artificial Bee Colony (ABC) is a second swarm intelligence algorithm that is prominent in the wider nature-inspired computational intelligence literature [29, 101]. ABC models three categories of bees: employed bees, each associated with a specific candidate solution (a food source) and responsible for exploiting the neighbourhood around it; onlooker bees, which probabilistically choose a food source to exploit further based on its fitness, in a manner conceptually similar to fitness-proportional selection in a GA; and scout bees, which abandon food sources that have not improved after a set number of attempts and replace them with newly generated random solutions, providing a built-in mechanism against premature convergence.
Each food source $x_i = [x_{i1}, x_{i2}, \ldots, x_{iD}]$ represents a candidate solution in a D-dimensional search space. In the employed bee phase, each employed bee generates a candidate solution v_i in the neighbourhood of its current food source x_i by perturbing a single randomly chosen dimension j against a randomly chosen neighbouring food source x_k, $k \neq i$:

$$v_{ij} = x_{ij} + \phi_{ij} \left(x_{ij} - x_{kj} \right), \tag{2.4}$$

where ϕ_{ij} is a random number drawn uniformly from $[-1, 1]$, and all other dimensions of v_i are copied unchanged from x_i. If v_i has better fitness than x_i, it replaces x_i (greedy selection); otherwise x_i is retained and its trial counter is incremented.
In the onlooker bee phase, each onlooker bee selects a food source to exploit further, in the same manner as Equation (2.4), with a probability proportional to that source's fitness relative to the fitness of the population as a whole:

$$p_i = \frac{fit_i}{\sum_{n=1}^{N} fit_n}, \tag{2.5}$$

where fit_i is the fitness of food source i and N is the number of food sources. This probabilistic, fitness-proportional selection concentrates further search effort on the more promising regions already found by the employed bees in a manner conceptually similar to fitness-proportional selection in a GA.
In the scout bee phase, any food source x_i whose trial counter has reached a predefined abandonment limit L without improvement is discarded and replaced with a new food source generated uniformly randomly within the bounds of the search space:

$$x_{ij} = x_j^{\min} + rand(0, 1) \cdot \left(x_j^{\max} - x_j^{\min} \right), \tag{2.6}$$

where $x_j^{\min}$ and $x_j^{\max}$ are the lower and upper bounds of dimension j. This scout mechanism provides the ABC with built-in protection against premature convergence: a stagnated food source is not merely mutated but abandoned outright and replaced with a fresh, unrelated candidate.
ABC's principal strength is its explicit scout mechanism, which gives it a self-contained way of reintroducing diversity into the population without requiring a separate mutation operator, and it has been shown to perform competitively with GA and PSO on a range of continuous optimisation benchmarks. Its principal weaknesses are that its exploitation phase, governed by the onlooker bees, tends to converge more slowly than a difference-vector mutation on ordered, continuous search spaces, and that it introduces at least one additional control parameter, the abandonment limit, beyond those already required by GA or DE.

2.4 Comparative Summary

Table 2.2 summarises the six search strategies covered in this section, summarising the type of search space each is best suited to and the principal advantage and disadvantage of each [29, 45].

Table 2.2: Comparative summary of the computational intelligence search strategies covered in this chapter.

Algorithm	Best suited to	Principal advantage	Principal disadvantage
Simulated Annealing	Discrete or continuous spaces with many shallow local optima	Theoretically grounded escape from local optima via probabilistic uphill moves	Highly sensitive to the cooling schedule; single trajectory limits coverage
ILS	Discrete, combinatorial configuration spaces with connected neighbourhood structure	Retains structural information across restarts; few control parameters	Cannot recombine solutions from separated regions; sensitive to perturbation strength
GA	Mixed-type, multimodal parameter spaces with several qualitatively different variables	Population-level diversity is robust to multimodal landscapes; handles mixed variable types	Many fitness evaluations per generation; more control parameters to tune
DE	Continuous, ordered, real-valued search spaces	Self-adapting step size via difference-vector mutation; few control parameters	Prone to premature convergence as population differences shrink
PSO	Unimodal or mildly multimodal continuous spaces	Search momentum from the velocity term; fast convergence, few parameters	Whole swarm can collapse prematurely towards a single global best position
ABC	Continuous spaces requiring built-in diversity maintenance	Explicit scout mechanism reintroduces diversity without a separate mutation operator	Slower exploitation than difference-vector mutation; additional control parameter

2.5 Conclusion

This chapter introduces the computational intelligence background, organised around the standard single-point versus multipoint taxonomy. Simulated Annealing and Iterative Local Search were presented as single-point strategies, with ILS receiving particular attention, its mechanics, advantages, and disadvantages, given its suitability for discrete, combinatorial configuration problems. The Genetic Algorithm and Differential Evolution were presented as the evolutionary branch of the multipoint family, and Particle Swarm Optimisation and Artificial Bee Colony as the swarm-intelligence branch. Across all six algorithms, the recurring theme is a trade-off between the efficiency and simplicity of single-point, trajectory-based search and the robustness, at a higher computational cost, of population-based search against multimodal landscapes. The comparative summary in Table 2.2 distils this trade-off into the specific type of search space each algorithm is best suited to, which is the primary consideration that should guide the choice of search strategy for any given optimisation problem.

Chapter 3

Hyperspectral Imaging

3.1 Introduction

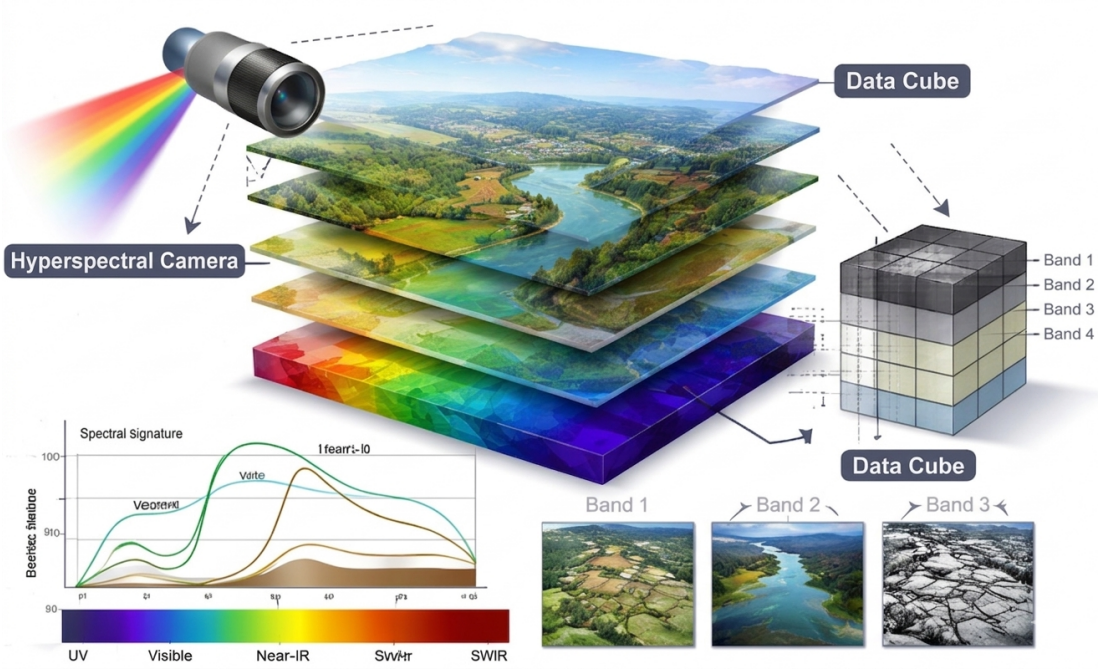
A conventional digital camera captures images using three broad channels of the electromagnetic spectrum: red (R), green (G), and blue (B) channels. Although this is sufficient for human vision, this three-band representation ignores a large amount of spectral information, which refers to how light of different wavelengths is reflected or emitted by a material. In contrast, hyperspectral imaging captures light across hundreds of narrow and continuous wavelength bands, many of which lie beyond the visible spectrum. This allows a full spectral signature, describing the material response across wavelengths, to be recorded for each pixel in the image [7, 69, 72].
The detailed spectral information recorded in hyperspectral imaging forms a three-dimensional data structure known as a *hyperspectral cube* (Figure 3.1). This cube consists of two spatial dimensions ($H \times W$) and one spectral dimension (B). Each pixel $p_{ij} \in \mathbb{R}^B$ is represented by a high-dimensional vector that describes how a material reflects or emits light at various wavelengths. This detailed information makes it possible to distinguish between materials that appear similar in standard images, such as separating real vegetation from artificial camouflage, identifying specific mineral types from satellite images, and detecting early signs of plant stress in agricultural fields [26, 106].
However, the same high-dimensional information that enables these applications also introduces computational and statistical challenges. These challenges make hyperspectral image segmentation different from conventional analysis. This chapter introduces the key properties of hyperspectral data, reviews its main application areas, and discusses the challenges that motivate the development of the adaptive segmentation framework presented in this thesis.

3.2 Characteristics of Hyperspectral Data

3.2.1 High Dimensionality and Spectral Resolution

A characteristic of hyperspectral imagery is its high spectral resolution (SR). In contrast, a standard RGB image contains only three spectral channels, whereas hyperspectral sensors typically capture 100–400 narrow, continuous spectral bands. For example, the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) records 224 spectral bands covering wavelengths from 400–2500 nm, with a spectral resolution of approximately 10 nm [7]. This fine spectral sampling makes it possible to observe subtle absorption features linked to specific materials, allowing material identification through spectroscopic analysis.
However, this large number of spectral bands leads to the *curse of dimensionality*. As the number of spectral bands B increases, the volume of the feature space grows exponentially, following $\mathcal{O}(2^B)$, whereas the number of available training samples N remains limited. This results in sparse data in a high-dimensional space, which reduces the reliability of the distance measures used in clustering and classification algorithms. Consequently, the class boundaries become less distinct, and the classification accuracy decreases. Figure 3.2 illustrates this effect using the Indian Pines dataset, where an increase in dimensionality leads to greater variation in spectral similarity within the same class and reduced separation between classes.

Figure 3.1: Conceptual illustration of hyperspectral imaging. (Left) A hyperspectral sensor acquires narrow contiguous spectral bands across the electromagnetic spectrum at each spatial location in the image. (Right) The stacked bands form a three-dimensional data cube with two spatial dimensions and one spectral dimension. Example spectral signatures for common land cover types are shown across visible (VIS), near-infrared (NIR), and short-wave infrared (SWIR) wavelengths.



3.2.2 Spectral Variability and BRDF Effects

The spectral signature of the same material can vary significantly owing to changes in illumination conditions, viewing angles, atmospheric effects, and surface properties. This phenomenon, known as spectral variability, makes it difficult to design segmentation models that assume consistent spectral behaviour across an image [72]. One important source of variability is the Bidirectional Reflectance Distribution Function (BRDF), which describes how the amount of reflected light from a surface changes with different illumination and viewing angles. For instance, a vegetation pixel can show more than 15% variation in reflectance when observed under different solar zenith angles, such as 30° and 60°, owing to changes in the canopy structure and shadowing [7]. In addition, atmospheric water vapour can absorb energy in certain wavelength regions, particularly around 1400–1900 nm, causing spatially varying spectral distortion across an image.

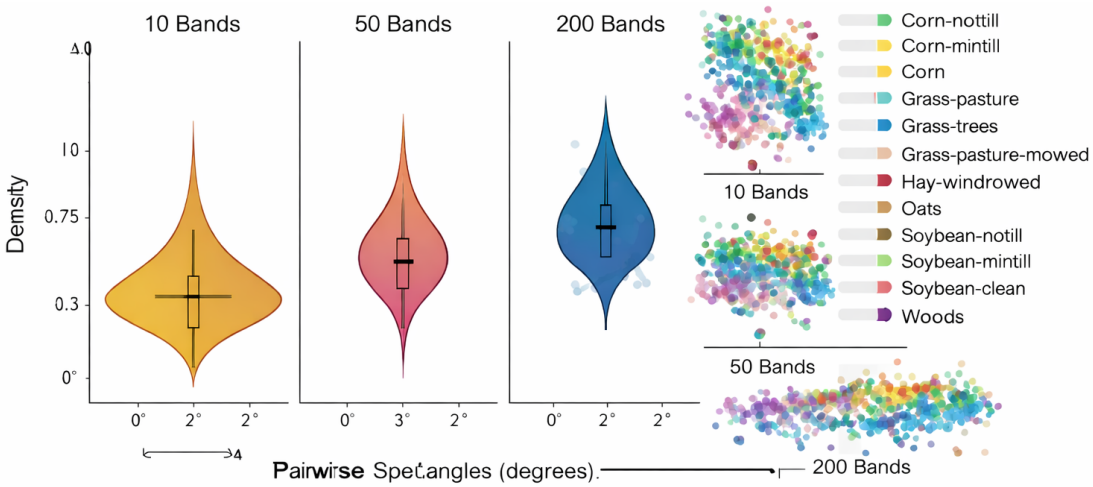
3.2.3 Mixed Pixels and Sub-Pixel Heterogeneity

Because hyperspectral sensors often have moderate spatial resolution, typically ranging from 1 to 30 m for airborne and spaceborne platforms, a single pixel may contain multiple materials. This results in the *mixed pixel* problem. For example, a pixel located at the boundary between soil and vegetation may capture a combination of both materials, producing a spectrum that is a linear or nonlinear mixture of their individual spectral signatures. This mixing effect introduces subpixel heterogeneity, which is common in hyperspectral imagery and makes accurate segmentation more challenging. Addressing this issue often requires specialised spectral unmixing techniques or methods that incorporate spatial and spectral regularisation [72].

3.2.4 Data Volume and Computational Complexity

Hyperspectral images also involve large data volumes, which lead to high storage and processing demands. For instance, a standard AVIRIS scene with dimensions of 145×145 pixels and 224 spectral bands, stored at a 16-bit radiometric resolution, requires approximately 9.5 MB of storage. In contrast, a larger scene captured by the Hyperion sensor, with dimensions of 256×3240 pixels and 242 spectral bands, can exceed 400 MB. Processing such large datasets requires efficient memory management, dimensionality reduction techniques, and parallel computing strategies [88].

Figure 3.2: Demonstration of the curse of dimensionality using the Indian Pines dataset. (Left) Distribution of pairwise spectral angles within the same class for different numbers of spectral bands (10, 50, and 200). (Right) As dimensionality increases, class separation deteriorates, with inter-class variance decreasing relative to intra-class variance beyond 100 bands.



3.3 Applications of Hyperspectral Imaging

3.3.1 Precision Agriculture and Environmental Monitoring

Hyperspectral imaging enables non-invasive monitoring of crop health [83, 103], nutrient status [17], and water stress [50, 104] through spectral indices such as the normalised difference vegetation index (NDVI) and derivative analysis of the red-edge position [26].
However, these applications often require the detection of rare but economically important events, such as early disease outbreaks affecting $< 1\%$ of plants or stress symptoms in specific cultivars that occupy only a few pixels per scene. The segmentation frameworks developed in this study are not currently capable of reliably detecting such rare classes (see Section 10.7). The agricultural and mineral exploration applications cited here motivate the broader problem domain, not a claim that the proposed methods are ready for deployment in these contexts without modification or further research.

3.3.2 Geological and Mineral Exploration

The unique spectral signatures of minerals in the short-wave infrared (SWIR) region enable the identification of alteration zones associated with hydrothermal deposits [7, 43]. SWIR refers to the short-wave infrared portion of the electromagnetic spectrum, which typically spans wavelengths from approximately 1000 to 2500 nm [47]. For example, minerals such as kaolinite, alunite, and calcite exhibit diagnostic absorption features near 2200 nm [47], allowing prospective mineralisation to be mapped remotely from airborne platforms [42, 43].

3.3.3 Industrial Quality Control

In manufacturing processes, hyperspectral imaging is applied for quality control by analysing the spectral response of materials [24, 70, 100]. This enables the reliable detection of defects [99], separation of different materials [82], and identification of contaminants that may not be visible in conventional imaging [8].

3.4 Challenges for Hyperspectral Segmentation

3.4.1 Curse of Dimensionality

Hyperspectral images contain a large number of spectral bands, which means the data has very high dimensionality. This leads to the *curse of dimensionality*, a problem where the number of features becomes very large compared to the number of available training samples. As a result, the data become sparse, and similarity measures and statistical estimates become unreliable. This makes it

difficult for many clustering and classification algorithms to function effectively [11]. Because of this imbalance, distance measures become less meaningful, and class boundaries become more difficult to estimate accurately.

3.4.2 Spectral Variability and Atmospheric Effects

In hyperspectral images, the same material can produce different spectral signatures across an image [7]. This is because real-world surfaces are often non-Lambertian, which means that they do not reflect light equally in all directions [7]. Instead, the amount of reflected light depends on the direction of illumination and the viewing angle of the sensor, an effect described by the Bidirectional Reflectance Distribution Function (BRDF) [7]. Consequently, changes in the illumination geometry, surface structure, and viewing conditions can cause reflectance variations for the same material. In addition, atmospheric scattering and absorption can distort specific wavelength regions, further altering the measured spectra [11]. These sources of spectral variability make segmentation difficult for methods that assume stable and consistent spectral behaviour across an image [7, 11].

3.4.3 Limited Training Data and Annotation Costs

Creating labelled hyperspectral datasets is time-consuming and costly because it requires expert knowledge to interpret spectral information. For example, manually labelling a hyperspectral scene of size 512×217 pixels can take between 20 and 30 h. As a result, the available training datasets are often small, typically containing fewer than 2000 labelled samples.
Small training sets do not capture the full spectral variability present in real scenes, which can lead to overfitting and poor generalisation in supervised learning approaches [91]. This problem is especially severe for rare classes, such as diseased crops or unusual minerals, which may represent less than 5% of the pixels in an image.

3.4.4 Computational Complexity

Processing hyperspectral images is computationally demanding because the data are three-dimensional, consisting of two spatial and one spectral dimensions. Algorithms that process the full data cube typically have a computational complexity of $\mathcal{O}(H \cdot W \cdot B)$ and require large memory sizes.
For example, a 3D convolutional neural network processing a $145 \times 145 \times 200$ hyperspectral cube requires more than 2.1 million floating-point operations for a single forward pass. This makes real-time processing difficult, particularly on resource-limited devices such as mobile devices. To enable practical deployment, hyperspectral segmentation methods often rely on dimensionality reduction, feature selection, and patch-based processing strategies [88].

3.4.5 Critical Synthesis of Existing HSI Segmentation Approaches

Although traditional, computational intelligence, and deep learning approaches differ in their underlying mechanisms, a common limitation is observed across all three categories. Most existing segmentation methods implicitly assume that the spectral distribution of a given material is consistent throughout an image. In practice, hyperspectral data violate this assumption because of local spectral variability caused by changes in illumination, BRDF effects, atmospheric conditions, and subpixel mixing.
Traditional methods, such as thresholding and clustering, rely on fixed global criteria or distance measures that are sensitive to the local spectral shifts. Computational intelligence methods, including particle swarm optimisation (PSO) and genetic algorithms (GA), improve optimisation and feature selection but typically operate on static fitness functions that do not adapt to the local image context. Similarly, deep learning approaches, despite their strong representational power, are usually trained to learn global decision functions and require large amounts of labelled data to generalise across heterogeneous regions.
Hybrid methods partially address these issues by combining deep learning outputs with classical post-processing techniques. However, these pipelines are manually designed and remain fixed during inference, limiting their ability to respond to spatially varying spectral characteristics within a scene.
Collectively, these observations suggest that the primary challenge in hyperspectral image segmentation is not the absence of effective models, but the lack of mechanisms that explicitly account for local spectral nonstationarity. This gap motivates the development of segmentation strategies that replace fixed, manually chosen global parameters with automated, search-guided configuration,

though existing work — including this thesis — has yet to close the gap down to the level of local spectral-spatial adjustment within a single scene (see Section 11.3.4).
The non-stationary spectral characteristics discussed herein motivate the development of automated configuration methods. This study addresses this motivation at the dataset level rather than at the pixel or region level, and no single strategy examined here has been universally successful. The three frameworks proposed in this thesis, the enhanced deep learning framework, the enhanced watershed framework, and the automated thresholding configurators, each encounter fundamental limits where optimisation cannot fully compensate for local spectral non-stationarity, producing dataset-specific failure modes rather than a single, universal weakness.

3.5 Conclusion

This chapter introduces hyperspectral imaging and describes its key characteristics, application areas, and challenges in image segmentation. The high dimensionality of hyperspectral data presents computational and statistical challenges, and spectral variability, limited training data, and subpixel heterogeneity further complicate the task of accurately segmenting hyperspectral images. Existing segmentation approaches range from traditional thresholding and clustering methods to deep learning and computational intelligence techniques, each with its own strengths and limitations. Although deep learning methods often achieve strong performance, their dependence on large labelled datasets remains a significant challenge in hyperspectral image analysis.
The issues discussed in this chapter highlight a fundamental trade-off in hyperspectral image segmentation between the need for adaptive local spectral-spatial processing and the requirement for improved computational efficiency. Many existing methods rely on fixed global strategies that struggle to cope with spatially varying spectral behaviours, whereas more flexible models often incur high data and computational costs. Collectively, these challenges define the broader problem context this study engages with; the specific scope addressed dataset-level automated configuration rather than within-scene local adaptivity is described in Chapter 1 and revisited in Section 11.3.4).

Chapter 4

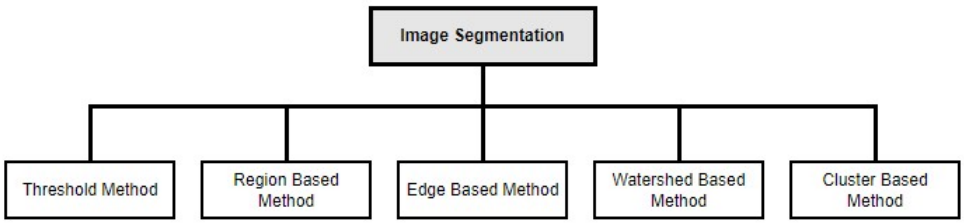
Image Segmentation

4.1 Introduction

Image segmentation is a computer vision technique that partitions a digital image into meaningful, discrete pixel groups [63]. Its primary objective is to assign a specific label or colour to every pixel so that different objects or areas within the image can be clearly identified and separated. This process is important for image processing applications, ranging from medical diagnosis to autonomous navigation.

Segmentation approaches fundamentally operate on two distinct principles: detecting abrupt changes in pixel intensity (discontinuity) or grouping neighbouring pixels with similar attributes (similarity). Discontinuity-based methods identify edges and boundaries where pixel properties change sharply, whereas similarity-based methods cluster regions with homogeneous characteristics, such as colour, texture, or brightness.

Figure 4.1: Taxonomy of image segmentation methodologies, categorising techniques based on their underlying operational principles.



As illustrated in Figure 4.1, classical segmentation methodologies can be broadly classified into five categories: threshold-based, region-based, edge-based, watershed-based, and clustering-based. Each category offers distinct trade-offs in terms of computational efficiency, robustness to noise, and boundary localisation accuracy. Recent advances have integrated these traditional techniques with deep learning architectures to form hybrid systems that leverage the strengths of both paradigms.

This section provides a comprehensive analysis of image segmentation techniques, establishing the foundational knowledge required to appreciate the novelty of the adaptive computational intelligence framework proposed in this study. We examined both classical methods and state-of-the-art deep learning approaches, with particular attention to their limitations in the hyperspectral domain, which motivates the need for adaptive automated search strategies.

4.2 Detailed Analysis of Traditional Image Segmentation Methodologies

Building on the foundational taxonomy introduced in Section 4.1, this section provides an in-depth analysis of image segmentation methods. We discuss the fundamental principles, computational

mechanics, and specific strengths and weaknesses of each methodology, with an emphasis on their applicability to high-dimensional data.

4.2.1 Threshold Method

Thresholding is a computationally efficient segmentation method. It operates on the principle of discontinuity, classifying pixels into two categories, foreground and background, based on their intensity values relative to a predefined threshold value, T. This relationship is mathematically expressed as follows:

$$g(x,y) = \begin{cases} 1 & \text{if } f(x,y) > T \\ 0 & \text{if } f(x,y) \leq T \end{cases}$$

where $f(x,y)$ is the original pixel intensity and $g(x,y)$ is the segmented output.

While global thresholding applies a single value of T across an entire image, adaptive thresholding enhances performance by calculating local thresholds for different regions, which is effective for images with uneven lighting [77]. A key challenge in using thresholding methods is the automatic selection of an optimal threshold, which is commonly addressed by an objective function. Otsu's method provides a widely used fitness criterion that determines a global T by maximising the between-class variance of the foreground and background classes [68] and has been adopted as an objective function in various optimization-based segmentation approaches.

Despite being originally formulated as a classic global technique, Otsu's between-class variance function remains an important benchmark objective; however, its limitation in handling non-uniform illumination when used as a single global criterion is well-recognised [74]. This limitation has prompted researchers to develop advanced and adaptive techniques that utilise Otsu's principle as a local or multilevel fitness function for thresholding. For instance, Shah et al. [77] developed an adaptive thresholding method designed to overcome this challenge, demonstrating how classical objective functions, such as Otsu's between-class variance, provide building blocks for new optimisation-driven solutions. However, thresholding objective functions, even in adaptive forms, are generally ineffective for images with complex textures or low contrast [74].

To overcome these limitations, researchers have combined thresholding with other techniques to create hybrid approaches that extend their applicability. In remote sensing, Zhang et al. [106] effectively combined adaptive thresholding with deep statistical models to achieve accurate and reliable segmentation of satellite imagery, even through haze and clouds. Similarly, in medical imaging, Patel and Singh [71] enhanced thresholding optimisation by incorporating entropy-based fitness functions alongside Otsu's between-class variance criterion, thereby improving the segmentation accuracy of magnetic resonance imaging (MRI) and computed tomography (CT) scans. These developments demonstrate that by integrating statistical learning and optimisation, thresholding objective functions continue to serve as important components in segmentation pipelines for applications ranging from document processing to medical imaging.

Multilevel Thresholding

The single-threshold formulation described above assumes that an image can be adequately partitioned into only two classes: foreground and background. This assumption is invalid for images in which more than two meaningful classes must be distinguished by a single global criterion, such as separating several land-cover types in a remote sensing scene or several tissue types in a medical image. Multilevel thresholding generalises the binary case by selecting a set of k ordered threshold values, $t_1 < t_2 < \cdots < t_k$, which partitions the image into $k+1$ classes:

$$g(x,y) = \begin{cases} 0 & \text{if } f(x,y) \leq t_1 \\ 1 & \text{if } t_1 < f(x,y) \leq t_2 \\ \vdots & \\ k & \text{if } f(x,y) > t_k \end{cases}$$

As with the binary case, the central challenge is the automatic selection of the threshold vector $\mathbf{t} = [t_1, t_2, \ldots, t_k]$ that best separates the underlying classes. Otsu's between-class variance criterion

extends naturally to this setting, with the optimal threshold vector chosen to maximise the variance between all $k+1$ classes simultaneously [68]:

$$\hat{\mathbf{t}} = \arg\max_{\mathbf{t}} \sum_{j=0}^{k} \omega_j \left(\mu_j - \mu_T\right)^2,$$

where ω_j is the probability of class j, μ_j is the mean intensity of class j, and μ_T is the global mean intensity of the image. An alternative objective function is Kapur's entropy criterion, which selects the threshold vector that maximises the total entropy across all classes, favouring partitions that preserve the greatest amount of information about the underlying intensity distribution.
Unlike the single-threshold case, which can be solved by an exhaustive search over all possible values of T, multilevel thresholding suffers from a combinatorial explosion as k increases. An exhaustive search over a grey-level range of L intensity values (typically $L = 256$) has a computational complexity of $O(L^k)$, which becomes computationally prohibitive beyond a handful of thresholds; a five-threshold search over 256 grey levels, for example, already implies several billion candidate combinations. This has motivated a substantial body of work applying metaheuristic search strategies, rather than exhaustive or greedy search, to efficiently locate near-optimal threshold vectors against an Otsu or Kapur-style objective function [29, 45, 76]. This thesis presents a search-driven reinterpretation of multilevel thresholding, along with a dedicated optimisation approach designed specifically for this purpose.

4.2.2 Region-Based Method

The region-based method for segmentation focuses on similarity rather than discontinuity. This technique groups adjacent pixels into meaningful regions based on shared characteristics such as intensity, colour, or texture.

The two main strategies within this method are region growing, region splitting, and merging. Region growing is an iterative process that begins with a set of seed pixels chosen from the image. These regions are then expanded by checking the neighbouring pixels. If a neighbour satisfies a predefined similarity criterion, such as its intensity being close to the region's average, it is added to the region. This process continues until no more pixels can be added that meet the criterion, effectively growing a complete object [44, 46]. The final result is a segmentation in which all pixels within a region are mathematically similar.

Region splitting and merging first divide the entire image into a set of arbitrary, often smaller square regions (splitting). The algorithm then evaluates whether the pixels in each region share similar visual characteristics based on a mathematical rule that defines what 'similar' is. If they do not, the region is divided into smaller subregions. Conversely, neighbouring regions that are sufficiently similar are grouped or merged to create larger, more meaningful regions [44, 46].
The key strength of the region-based method is its ability to produce connected and closed regions, which is particularly useful for segmenting images with fuzzy boundaries or smooth transitions that may be missed by edge detection. However, its performance relies on the correct selection of initial seed points (for region growing) and the design of the similarity criterion, which makes it sensitive to noise and lighting variations.

Recent studies have focused on automating and optimising the seed points used to start region growing and the homogeneity criteria that decide whether a pixel belongs to a region. A 2022 review by Adeyanju et al. [1] highlighted how region-based principles are integrated with optimisation algorithms to automatically select seed points and improve homogeneity criteria, thereby enhancing their robustness in medical image segmentation.

Region-based methods are being enhanced through integration with deep learning and advanced spatial modelling. He et al. [26] applied region growing with dynamic similarity constraints for crop field segmentation in precision agriculture, enabling real-time adaptation to varying field conditions. Kaur et al. [40] introduced CNN-driven region merging for organ segmentation, where convolutional neural networks guide the merging process based on the learned anatomical priors. These approaches demonstrate how traditional region-based methods are enhanced using region-based CNNs and spatial priors, thereby maintaining their relevance in modern segmentation pipelines.

4.2.3 Edge-Based Method

The edge-based method offers a contrasting approach to similarity-focused techniques, relying on the principle of discontinuity in images. This technique focuses on identifying the boundaries that separate distinct regions and objects in an image. These boundaries or edges are points in an image where the pixel intensity changes significantly.

Edge detection involves three main steps: filtering, enhancement, and localisation. In filtering, the image is smoothed using a filter, such as a Gaussian filter, to reduce noise. This is important because sharp noise spikes can be mistaken for real edges. Then, to perform the enhancement, edge detection operators such as the Sobel, Prewitt, and Canny operators are applied. These are derivative masks that calculate the gradient magnitude and direction of the intensity change at every pixel [44, 46]. A large gradient magnitude indicates a potential image edge. The Canny operator is considered effective because of its ability to accurately detect meaningful edges while ignoring noise. Finally, in the localisation step, the detected edges are further refined. The algorithm refines its detected edges by first making them thin and sharp (non-maximum suppression) and then determining which ones are real and continuous while removing weak or noisy edges (hysteresis thresholding).

The strength of the edge-based method is its ability to accurately localise object boundaries and its computational speed compared with that of some region-based methods. However, this method is sensitive to noise despite initial filtering and can produce discontinuous edges (gaps in the boundaries) in images with complex textures and low contrast. Closing these gaps requires additional post-processing.

The principles of edge detection remain fundamentally important and often serve as a critical preprocessing step in complex models. Recent studies, such as that by Urrea and Vélez [85], demonstrate this enduring utility by showing how classic operators, such as Canny, are effectively used. They generate precise edge maps (clean outline maps) that are used to guide or regularise modern deep learning models. By feeding these structural outlines directly into the network, the process forces the model to accurately define the boundaries of the objects. This approach provides an important enhancement for segmenting low-contrast images (such as medical scans or foggy scenes), where objects are otherwise difficult to distinguish from their environment.

Edge detection remains relevant in modern segmentation systems, particularly for enhancing the boundaries of hybrid architectures. Nguyen et al. [65] provided an example of this utility by hybridising the Canny edge detection operator with a deep contour prediction network for medical boundary delineation. This hybridisation addresses the challenge of fuzzy boundaries in low-contrast medical images. The Canny operator first extracts precise, thin structural outlines, which are then integrated into the architecture of the deep network to regularise its learning. By incorporating this geometric information, the model is forced to respect sharp edges, ultimately achieving enhanced precision in organ segmentation tasks, where accurate delineation is important for clinical diagnosis. Bhardwaj et al. [6] developed multiscale Sobel-Laplacian hybrid filters for industrial defect detection, enabling robust edge preservation in noisy manufacturing environments.

These studies show that edge-based methods remain an important component of modern segmentation techniques, providing reliable boundary information that complements deep learning approaches.

Where the operators discussed above rely on hand-crafted derivative masks, a deep learning counterpart to this same discontinuity-based principle learns the edge-detection filters directly from data instead of specifying them by hand. This deep learning extension of the edge-based method, Holistically-Nested Edge Detection [94], is discussed in Section 4.3.2.

4.2.4 Watershed-Based Method

The watershed-based method is a segmentation technique that is rooted in mathematical morphology. It operates by treating the intensity landscape of an image as a topographical map [44, 46]. In this analogy, the pixel intensity values are interpreted as elevation, where bright pixels represent high peaks or mountains, and dark pixels represent low valleys or basins.

The process simulates the gradual flooding of this landscape from a series of markers placed at the local minima (the deepest valleys, usually corresponding to the object's interior). As the "water level" rises, dams or watershed lines are constructed at the points where water from different basins meets. These constructed dams effectively partition the image into distinct, non-overlapping regions, with watershed lines representing the desired segmentation.

The strength of the watershed method is that it guarantees closed and continuous contours around segmented regions, which is often superior to the fragmented boundaries produced by simple-edge detectors. Furthermore, it is computationally efficient and can be effectively used to segment crowded and overlapping objects. However, this technique is sensitive to noise and minor intensity variations, which often result in oversegmentation and the creation of too many small, meaningless regions owing to the presence of several local minima [30]. This challenge is commonly mitigated through preprocessing steps, such as filtering, or by using marker-controlled watershed algorithms, which predefine the object interiors (markers) and boundaries to guide the flooding process.

Marker-controlled watershed variants require an explicit definition of internal (objects) and external (background) markers. The specific marker generation strategy employed in this study used Otsu's thresholding on the gradient map to generate internal markers with morphological dilation for external markers.

The ability of the watershed algorithm to produce closed, continuous contours makes it valuable in bioimage informatics, particularly when dealing with images containing touching or overlapping objects. Deep convolutional neural networks (CNNs) often struggle with such instances, creating single merged regions instead of individual, distinct objects. Li et al. [52] addressed this by successfully coupling a CNN with a marker-controlled watershed transform. In this study, a CNN was used to generate internal "markers" (seeds within each object) and a preliminary boundary map, while the watershed used these markers as starting points to precisely flood the image structure and force the separation of overlapping objects. This integration demonstrates how a classical approach can be strategically hybridised with deep learning to achieve state-of-the-art results, such as the accurate separation of complex overlapping structures, such as chromosomes.

The success of these hybrid strategies highlights the importance of intelligent marker generation to enhance classical methods. For instance, extending the marker-controlled approach, Ali et al. [4] proposed a technique that integrates the watershed with graph cuts for cell nuclei segmentation, providing an additional layer of optimisation for effectively handling overlapping cells in microscopic-image. Furthermore, Zhou et al. [112] utilised pretrained CNNs to generate reliable markers for watershed segmentation, thereby addressing the core challenge of reducing oversegmentation while maintaining boundary accuracy.

These advancements demonstrate the evolution of watershed methods, which continue to adapt by integrating with other techniques.

4.2.5 Cluster-Based Method

The cluster-based method offers a distinct approach to image segmentation using an unsupervised learning algorithm. It treats an image as a collection of data points rather than simple pixel intensity changes. Each pixel is represented by measurable features such as colour (RGB values), brightness, texture, and spatial position. The clustering algorithm groups pixels with similar features into clusters that represent meaningful regions or segments of the image. This method eliminates the need for fixed thresholds and instead depends on the natural similarity between pixels within the feature space.

The most common algorithm used in this technique is K-Means clustering. The process begins with initialisation, where the algorithm randomly selects K points in the feature space to serve as the initial cluster centres or centroids, where K represents the desired number of segments. This is followed by the assignment phase, where every pixel (data point) in the image is assigned to the nearest cluster centre based on a measure of distance. Once all pixels are assigned, the algorithm updates the cluster centres by calculating the mean position of all pixels currently belonging to that cluster. These assignment and update steps are then repeated iteratively until the cluster centres stabilise (for example, they no longer move significantly) or a maximum number of iterations is

reached. This iterative process results in partitioning the image into K segments, ensuring that all pixels within a given segment are mathematically similar in terms of their chosen features.

The strength of the cluster-based method is its independence from training data owing to its unsupervised nature, and it can segment images based on multiple features simultaneously, such as separating objects that have similar colours but different textures. This approach is also computationally efficient for processing large image datasets. However, cluster-based methods have several limitations. They are sensitive to the initial random choice of cluster centres and require the user to prespecify the number of clusters (K) [36]. Furthermore, this method does not consider the spatial adjacency of pixels [36]. This means that pixels that are far apart in the image but highly similar in colour could be grouped into the same cluster, which can often lead to the creation of fragmented or non-contiguous segments in the final output [1].

Cluster-based methods do not require pre-labelled training data, making them efficient for initial analyses or when ground truth annotations are scarce. A 2022 survey by Minaee et al. [63] discussed their application in segmenting textured images and noted that they often serve as an important baseline for evaluating more complex deep learning models. However, a limitation is that traditional K-means only considers pixel intensity or colour, which often leads to non-contiguous segments (speckled, scattered pixels assigned to the same cluster).

To directly address this problem, recent approaches have incorporated spatial constraints directly into the clustering process [75] as follows: This improved method ensures that clustered pixels not only share similar colour/intensity values but are also geographically close to each other in the image, effectively mitigating fragmentation and generating more coherent and meaningful regions in the image. The integration of spatial information has led to the broader evolution of cluster-based segmentation into refined techniques, such as spatial-spectral clustering and methods that integrate deep feature embeddings.

This progress is evidenced by recent studies that have extended the capabilities of cluster-based methods. For example, Tian et al. [84] improved K-means with fuzzy clustering and feature embedding for remote sensing applications, enabling better handling of mixed pixels in complex-satellite imagery. Similarly, Saha and Roy [75] developed a spatially weighted K-Means for urban feature extraction, directly integrating geographical constraints to improve segmentation coherence in geographic information systems (GIS) applications.

These examples demonstrate the ongoing relevance of clustering methods, which have adapted and enhanced their feature representation and spatial modelling capabilities to remain competitive in specialised fields.

4.3 Deep Learning Approaches

Convolutional neural networks (CNNs) are state-of-the-art approaches for image segmentation [55]. Architectures such as the U-Net, based on an encoder-decoder structure with skip connections, have set the standard, particularly in biomedical image segmentation [73]. Recently, Vision Transformers (ViTs) and Segmentation Transformers (SETR) have challenged the supremacy of CNNs [110]. By utilising self-attention mechanisms to capture global contextual information more effectively, ViT models have demonstrated enhanced performance in complex scenes. For instance, the 2021 SETR model established a new paradigm by achieving state-of-the-art results on semantic segmentation benchmark datasets. A 2025 survey by Mei et al. [62] confirmed that both CNN and Transformer-based models are now central to advancing the field, as seen in complex tasks such as polyp segmentation.

Polyp segmentation is a clinically important application in biomedical imaging that involves the automatic identification and precise outlining of polyps in medical images, typically obtained via colonoscopy [89]. Polyps are abnormal growths on the inner wall of the colon, and their early and accurate detection is critical for preventing colorectal cancer (CRC). The difficulty in this task, which is characterised by subtle contrast, varied polyp shapes, and background noise, drives the need for hybrid architectures that combine the strengths of CNNs and Transformers.

Deep learning methods continue to improve segmentation performance by focusing on hybrid and refined architecture. For instance, Chen et al. [15] introduced ConvNeXt-SAM for precise semantic segmentation, which integrates a modern convolutional architecture (ConvNeXt) with spatial attention mechanisms to improve feature localisation. Similarly, Xie et al. [95] proposed TransUNet v2, which enhances the original hybrid architecture through better transformer-CNN integration and improved skip connections for more efficient multi-scale feature fusion in segmentation tasks. Finally, Mei et al. [62] comprehensively reviewed transformer-based approaches specifically for polyp segmentation, highlighting the rapid evolution of attention-based methods for addressing the precise boundary delineation challenges inherent in medical imaging.

4.3.1 Semantic, Instance, and Panoptic Segmentation

Deep learning-based image segmentation is categorised into semantic, instance, and panoptic segmentation, each representing a level of image scene understanding and complexity [20]. These categories extend the deep learning principles introduced in Section 4.3 by defining how deep learning approaches interpret and differentiate image regions at different semantic and structural levels.

Semantic Segmentation

Semantic segmentation is the process of labelling every pixel in an image with a specific category without distinguishing between different individual objects of the same category (class-level recognition). For example, it would label all parts of a road as "road" or all areas of a tumour as "tumour", creating a detailed map where the focus is on the "what" rather than the "how many".
Deep learning relies on specialised architectures to achieve pixel-level classification. Architectures such as U-Net [73] and DeepLabv3+ [13] have served as benchmarks in the field, relying on encoder-decoder topologies with skip connections for multi-scale context. Further studies on class-pixel classification architecture include ConvNeXt-SAM [15] and SegFormer [93], which combine convolutional backbones with self-attention or promptable mechanisms, improving spatial accuracy and ensuring consistent interpretation of surrounding regions across different imagery domains such as medical imaging, remote sensing, and urban scenes.

Instance Segmentation

Instance segmentation extends the pixel labelling principle of semantic segmentation by differentiating individual object occurrences within the same class. This task moves beyond simply stating what a set of pixels is (e.g., a car) to stating which one it is (e.g., "car 1", "car 2"). This combination of object detection and precise boundary delineation is important for applications that require the separation of overlapping and adjacent objects.

Specialised architectures are required to achieve this. Architectures such as Mask R-CNN integrate object detection and segmentation into a two-stage pipeline that enables both class prediction and instance separation in a single step. Recent architectures, including QueryInst [22] and Mask DINO [14], unify detection and mask generation using transformer-based query frameworks. These developments achieve effective consistency between object localisation and boundary precision, significantly benefiting difficult tasks such as object counting, autonomous driving, and industrial defect inspection.

Panoptic Segmentation

Panoptic segmentation combines the goals of semantic and instance segmentation into a single, comprehensive map. Every pixel is labelled with both what it is (the general category, such as "sky" or "road") and which one it is (a unique ID for each individual item, such as "Car 1" or "Person 3"). This dual labelling allows the system to simultaneously understand undifferentiated "stuff" regions (e.g., sky, grass) and countable "thing" objects (e.g., cars and people).

Models such as the Panoptic FPN [41] lay the foundation for combining these tasks, with current transformer-based systems, such as DETR, Mask2Former, and Panoptic Segment Anything (PSA) [111], using global attention mechanisms to efficiently integrate individual object details into a wide-scene context. These architectures are important for complex image analysis, as they effectively model relationships across the entire image and ensure that there are no confusing gaps or

mislabelled boundaries between objects and their backgrounds.

Collectively, these three segmentation paradigms form the conceptual and architectural centres of deep learning studies in image processing. The distinct objectives of pixel-level classification, separation of individual objects, and holistic scene composition guide the choice of architectural strategy. This includes the hybrid designs presented in Section 4.4, such as models that integrate Transformer and CNN components or use attention mechanisms to combine features. These strategies aim to achieve accurate segmentation across various domains.

A promising trend is the development of hybrid models that combine the strengths of different approaches. A common strategy involves using a classical method (e.g., thresholding or watershed) for initial segmentation or to generate constraints, which are then refined using a deep learning model. Conversely, CNNs can generate high-quality markers or probability maps that guide watershed transformation by combining semantic information from deep learning with precise boundary detection using shape-based morphological algorithms [52]. As highlighted in a 2024 comprehensive review by Li et al. [48], such hybrid approaches are a dominant trend for overcoming the limitations of any single method, effectively balancing high-level semantic understanding and low-level pixel accuracy.

Building on these foundational hybrid methods, current hybridisation trends combine traditional morphological operations with the latest transformer-based attention mechanisms [27]. This evolution extends beyond simple CNN-based guiding signals to harness the global context of the transformer capabilities. For instance, Zhang et al. [108] created a threshold-transformer pipeline for medical images, where the initial thresholding provided coarse region proposals that were subsequently refined using transformer networks.

These advanced approaches continue to emphasise the complementary nature of classical computer vision techniques and modern deep learning, particularly in domains that require both precise boundary delineation and holistic semantic understanding. The integration of morphological operations with attention mechanisms represents a promising avenue for achieving robust, high-performance, and interpretable segmentation systems.

Table 4.1: Comparison of Deep Learning Segmentation Architectures (2021–2025). HSI-specific transformers (SpectralFormer, HyCoT) require millions of parameters and full retraining; HED (2015) is selected as baseline to represent deployed edge-detection infrastructure that cannot be retrained.

Architecture	Key Innovation	Parameters	Year	HSI-Specific?
U-Net	Skip connections	14.7M	2015	No
HED	Multi-scale side outputs	14.7M	2015	No
SegFormer	Hierarchical transformer	3.7M–84.7M	2021	No
TransUNet	Transformer-CNN hybrid	31.2M	2021	No
ConvNeXt-SAM	Spatial attention	28.5M	2024	No
SpectralFormer	**Spectral-grouped attention**	**1.2M**	**2021**	**Yes**
HyCoT	**Spectral-patch attention**	**3.4M**	**2025**	**Yes**

4.3.2 Holistic-Nested Edge Detection

Holistically-Nested Edge Detection (HED) is a convolutional neural network architecture that replaces the hand-crafted derivative masks of classical edge detectors, such as Sobel or Canny (Section 4.2.3), with filters learned directly from labelled training data [94]. Built on a VGG-style convolutional backbone, HED produces not one but several intermediate predictions, referred to as side outputs, one taken from each major stage of the network. Because successive convolutional stages progressively reduce spatial resolution while increasing the receptive field, the side outputs from early layers capture fine, locally precise edge detail, while the side outputs from deeper layers capture coarser, more semantically meaningful object boundaries. Each side output is passed through a 1×1 convolution and upsampled back to the input resolution, after which all side outputs are combined by a learned, weighted fusion layer to produce the final edge probability map.

This multi-scale, multi-output design is what distinguishes HED from a conventional single-output CNN edge detector, and it is also what motivates the "holistic" and "nested" terminology in its name: the network is supervised holistically, since each side output is trained against the same ground-truth edge map rather than an intermediate target, and the supervision is nested, since the side outputs are organised hierarchically from fine to coarse and are combined rather than treated independently. Training uses a deep supervision strategy in which every side output, not only the final fused prediction, contributes its own loss term. Because edge pixels are a small minority of the pixels in any typical image, this loss is computed as a class-balanced cross-entropy, which weights the edge and non-edge classes inversely to their relative frequency so that the rarity of edge pixels does not cause the network to trivially predict the majority non-edge class everywhere.

By learning its filters end-to-end rather than relying on fixed derivative kernels, HED can capture edge patterns that are specific to the training distribution, which a generic operator such as Canny or Sobel cannot represent, at the cost of requiring labelled training data and additional computation that classical operators do not. HED has since been extended by several follow-on architectures that refine this same holistically-supervised, multi-scale principle, including richer multi-scale feature fusion strategies [54], densely connected extreme-inception designs [81], and, more recently, transformer- and diffusion-based edge detectors [96, 102]. Despite this progression, HED remains a widely used and representative baseline for deep, multi-scale edge detection, precisely because its side-output and fusion design is simple enough to be extended or selectively augmented without requiring the network itself to be retrained.

4.4 Hybrid Segmentation Techniques

Classification of hybrid segmentation techniques reveals three distinct paradigms:

Type I: Classical→Deep Pipelines Use classical methods for preprocessing or constraint generation, followed by deep refinement (e.g., thresholding generates coarse masks [106], watershed markers guide CNNs [52]).

Type II: Deep→Classical Pipelines Employ deep networks for feature extraction, followed by classical post-processing (e.g., CNN probability maps refined by MRF [3]).

Type III: Hyper-Heuristic and Evolutionary Metaheuristic Composition The three frameworks proposed in this study occupy distinct but related positions on the automation spectrum. In all cases, the goal is to reduce reliance on fixed, manually engineered pipeline design by introducing an adaptive, data-driven search that improves performance under a validation protocol. The specific mechanisms and design decisions are detailed in the methodology chapters; here, they are brought together solely because they share this common focus on automation.

4.5 Performance Metrics

A quantitative evaluation of segmentation algorithms is essential for benchmarking their effectiveness. Instead of relying solely on visual inspection, common metrics are used to measure accuracy against a known "ground truth". The most widely adopted metrics include the following:

- **Pixel Accuracy**: The proportion of correctly classified pixels.
- **Intersection over Union (IoU)**: Also known as Jaccard Index, calculated as:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

  where A is the segmented region and B is the ground truth.

- **Dice Coefficient**: Measures overlap using:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

- **Hausdorff Distance**: Assesses the distance between the boundaries of the segmented and ground truth regions, providing a crucial measure of contour precision [60].

To ensure that minority-class performance is visible alongside aggregate metrics, this study also reports **per-class F1-score** and **macro-averaged F1** (mF1):

$$\mathrm{F1}_i = \frac{2 \cdot \mathrm{precision}_i \cdot \mathrm{recall}_i}{\mathrm{precision}_i + \mathrm{recall}_i} \tag{4.1}$$

$$\mathrm{mF1} = \frac{1}{C} \sum_{i=1}^{C} \mathrm{F1}_i \tag{4.2}$$

where classes with $< 0.5\%$ pixel representation are flagged as rare and reported.

Table 4.2: Summary of Recent Advances in Image Segmentation Techniques (2023–2025)

Technique	Recent Enhancement	Application Area	Study
Thresholding	Entropy-based fitness functions	Medical imaging	[71]
Thresholding	Deep statistical models	Satellite imagery	[106]
Region-Based	Dynamic similarity constraints	Precision agriculture	[26]
Region-Based	CNN-driven region merging	Medical organ segmentation	[40]
Edge-Based	CNN-Canny fusion	Medical boundaries	[65]
Edge-Based	Multi-scale hybrid filters	Industrial inspection	[6]
Watershed	Graph cuts integration	Cell nuclei segmentation	[4]
Watershed	CNN marker generation	General segmentation	[112]
Cluster-Based	Fuzzy clustering + embedding	Remote sensing	[84]
Cluster-Based	Spatially weighted K-Means	Urban feature extraction	[75]
Deep Learning	Transformer-CNN hybrids	Multi-domain	[95]
Hybrid	Threshold + transformer	Medical imaging	[108]

4.5.1 Computational Complexity Notation

We used standard Big-O notation to analyse the algorithmic complexity. Table 4.3 defines the key terms used throughout this thesis.

Table 4.3: Computational complexity notation

Notation	Definition
$O(H \times W \times B)$	Linear in image height, width, and spectral bands
$O(B^2)$	Quadratic in spectral dimensionality
$O(N \cdot d)$	Linear in number of samples N and dimensions d
$\mathcal{O}(2^B)$	Exponential in spectral bands (curse of dimensionality)
$\mathcal{L}_{\text{spat}}$	Spatial regularization loss

4.6 Conclusion

This chapter surveys image segmentation from classical foundations to recent hybrid trends. Threshold, region, edge, watershed, and cluster-based methods trade off computational speed, noise robustness, and boundary accuracy in different ways and are generally fast but struggle with complex or noisy scenes. Convolutional networks and, more recently, Vision Transformers have pushed accuracy further by learning hierarchical features directly from data at the cost of large annotated datasets and significant computing. Hybrid techniques, which combine the semantic strength of deep learning with the precise boundaries of classical methods, represent the most active recent trend and the closest precedent to this thesis.

The insights gathered from this analysis reveal a critical gap: although individual methods excel in specific scenarios, none provide a universally optimal solution for the unique challenges of hyperspectral images. Static deep learning models require millions of parameters and thousands of labels,

classical methods lack spectral awareness, and hybrid approaches are manually designed and inflexible. The segmentation frameworks presented in this study, where low-level operators, including enhanced versions of the techniques discussed here, are automatically selected and combined through a dataset-level search to achieve improved segmentation without manual-pipeline engineering.

Chapter 5

Research Methodology

This chapter describes the research methodology employed in this study, the datasets used, and the method used to assess the performance of the proposed approaches. First, the research methodology used in this study is outlined in Section 5.1, followed by the benchmark datasets used in this study (Section 5.2). The metrics and statistical tests used to assess the performance of the approaches are discussed in Section 5.3. The chapter concludes with a description of the technical specifications of the hardware and software used to conduct the experiments (Section 5.4).

5.1 Research Methodology

This section provides an overview of the research methodology employed in this study. Johnson [35] presents the following methodologies for conducting research in the field of computer science:

- Proof by demonstration
- Empiricism
- Mathematical proof
- Hermeneutics

Oates et al. [67] asserted that some research methodologies are better suited to a given research problem. The authors further contend that a single research problem can be addressed using multiple appropriate methodologies. The primary objective of this study was to determine whether computational intelligence approaches improve the accuracy and computational efficiency of hyperspectral image segmentation compared with fixed global-baseline strategies. To investigate this, baseline and enhanced segmentation algorithms must be developed and iteratively refined to improve their performance. Hence, the proof-by-demonstration methodology is deemed to be the most appropriate choice for this study.
The proof-by-demonstration method involves creating an initial artefact, such as a model or algorithm [35]. This initial artefact is iteratively refined until the desired solution is obtained or no further improvements are possible [35]. The refinement process consists of three steps: testing/evaluation, analysis, and refinement [35]. If there is no improvement during the testing step, the analysis step is used to determine the reasons for this lack of improvement. The refinement step adjusts the artefact for the next iteration to address the reasons for the lack of improvement [35].

5.1.1 Proof by Demonstration Methodology

As previously stated in Chapter 1, the objective of this study is to enhance baseline image segmentation methods using computational intelligence approaches rather than relying solely on a single fixed global strategy. The proof-by-demonstration methodology was applied to three classical segmentation paradigms, each of which was first implemented as a fixed-parameter baseline artefact and then extended into an enhanced, computational-intelligence-driven version of the same pipeline:

1. **Multilevel thresholding.** A baseline pipeline that partitions an image using a fixed or manually selected set of intensity thresholds is first constructed, and then enhanced through the application of a metaheuristic search strategy that automatically determines the optimal threshold vector, replacing manual or exhaustive threshold selection.

2. **Watershed Transform segmentation.** A baseline marker-controlled watershed pipeline, configured using manually fixed preprocessing and marker-generation parameters, is first constructed and then enhanced through evolutionary co-optimisation of the full pipeline, preprocessing, gradient extraction, marker generation, and region classification, as a single combined search problem.

3. **Holistically-Nested Edge Detection (HED).** A fixed, pre-trained deep learning edge-detection model is used as the baseline and is then enhanced through hyper-heuristic selection of complementary classical feature detectors without retraining the underlying network.

For each of these three paradigms, the baseline pipeline was first constructed using established manually configured parameter settings consistent with those in the literature. An enhanced version of the same pipeline was then constructed by replacing the manual parameter or feature choices with an automated computational intelligence search strategy, whereas the underlying segmentation mechanism, thresholding, watershed flooding, or edge detection remained unchanged. This design isolates the contribution of the search strategy; any difference in performance between a baseline and its enhanced counterpart can be attributed to the automated parameter or feature selection process rather than to a change in the underlying segmentation algorithm. One exception applies to the watershed pair: the full-pipeline co-optimisation variant (Section 8.3.3) additionally searches over classifier choice, a degree of freedom not available to the baseline watershed pipeline or the single-parameter optimisation variant, both of which are fixed to an SVM classifier. Section 8.4 reports a controlled variant that fixes the classifier to an SVM, isolating the contribution of this additional freedom to the observed results.

Using the proof-by-demonstration methodology, an initial version of each baseline and enhanced pair was created independently, following the pairing set out above. The algorithms were evaluated using sets of benchmark hyperspectral datasets, as discussed in Section 5.2. These datasets were selected as the most commonly used in the field of hyperspectral image analysis and were used by the state-of-the-art techniques against which the enhanced approaches were compared. The performance of each enhanced approach was compared with that of its corresponding baseline approach to determine whether there was an improvement in segmentation accuracy and computational efficiency. The performance metrics used, as well as the statistical tests used to determine the significance of the results, are discussed in Sections 5.3.1 and 5.3.3, respectively. If an enhanced approach did not outperform its baseline, the algorithm was refined by adjusting the parameter values, modifying the adaptive strategy, or using a different enhancement mechanism. Once an algorithm was refined, it was re-evaluated, and its performance was compared with that of the baseline approach. This process was repeated independently for each of the three baseline/enhanced pairs until either the enhanced approach outperformed its baseline or no further improvement could be made. In the absence of any additional improvement, an analysis was performed to identify the underlying reasons for this outcome.

To ensure precise validation, ablation studies are designed to isolate the contribution of the optimisation strategy from the mere addition of components. For the deep learning pair, three conditions are compared: (i) baseline HED alone, (ii) naive aggregation of all classical detectors without selection, and (iii) hyper-heuristic-selected classical features. This design ensures that any improvement can be attributed to the selective search strategy rather than indiscriminate feature addition. Similar ablation logic is applied to the watershed and thresholding pairs by comparing single-parameter optimisation against full-pipeline co-optimisation.

To further isolate the contribution of the hyper-heuristic search strategy from any benefit derived from arbitrary feature selection, a random selection baseline was pre-registered as part of the deep learning ablation design [33]. Under this design, subsets of classical feature detectors were drawn uniformly at random and matched in cardinality to the hyper-heuristic-selected subset for each dataset, with the resulting accuracy averaged over ten draws. This baseline distinguishes between two possible explanations for any observed improvement: (i) adding *any* classical features to HED is beneficial, or (ii) the hyper-heuristic search discovers *synergistic* feature combinations that arbitrary selections cannot discover. If random selection matches the enhanced model's performance, explanation (i) holds and the hyper-heuristic search provides no additional value beyond naive augmentation; if the enhanced model outperforms random selection, explanation (ii) is supported instead.

5.2 Datasets

This section describes the benchmark hyperspectral datasets used to evaluate the segmentation approaches. The datasets were selected because they are the most commonly used in the literature and have been used by the state-of-the-art techniques against which the proposed approaches are being compared. The characteristics of each dataset are summarised in Table 5.1.

Table 5.1: Standard hyperspectral benchmark datasets used in this study.

Dataset	Sensor	Dimensions	Classes	Res. (m)	Key Characteristics
Indian Pines	AVIRIS	$145 \times 145 \times 200$	16	20	Agriculture, mixed pixels
Pavia University	ROSIS	$610 \times 340 \times 103$	9	1.3	Urban, high resolution
Salinas Valley	AVIRIS	$512 \times 217 \times 204$	16	3.7	Vegetation, variability
Houston	CASI	$349 \times 1905 \times 144$	15	2.5	Complex urban gradient
Botswana	Hyperion	$1476 \times 256 \times 145$	14	30	Savanna, noise present

The Indian Pines dataset was captured by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) over the Indian Pines test site in northwestern Indiana. The image consists of 145×145 spatial pixels with 200 spectral bands. The scene contains 16 ground-truth classes representing different agricultural crops and vegetation and is characterised by the presence of mixed pixels.
The Pavia University dataset was acquired using a Reflective Optics System Imaging Spectrometer (ROSIS) over the University of Pavia, Italy. The image consists of 610×340 spatial pixels with 103 spectral bands. The scene contained nine ground-truth classes representing urban features, such as asphalt, meadows, and trees.
The Salinas Valley dataset was collected by an AVIRIS sensor over the Salinas Valley, California. The image comprises 512×217 spatial pixels with 204 spectral bands. The scene contains 16 classes representing different vegetation types, such as broccoli, lettuce, and vineyards, and exhibits significant spectral variability.
The Houston dataset was captured using a compact airborne spectrographic imager (CASI) over Houston, Texas. The image consists of 349×1905 spatial pixels with 144 spectral bands. The scene contained 15 ground-truth classes representing a complex urban gradient with diverse land-cover types.
The Botswana dataset was acquired by the Hyperion sensor over the Okavango Delta in Botswana. The image comprises 1476×256 spatial pixels and 145 spectral bands. The scene contained 14 ground-truth classes representing savanna vegetation and was characterised by atmospheric noise.
For each dataset, the ground truth labels were divided into training and testing sets. The training set was used to train the deep learning-based approaches, whereas the testing set was used to evaluate the performance of all segmentation approaches. The same training and testing splits were used for both the baseline and enhanced approaches to ensure a fair internal comparison.
This study applies a uniform 70%/30% stratified split across all labelled pixels, in contrast to the fixed-count, class-stratified splits common in the published literature (typically 200 labelled pixels per class, or the fixed train/test partition released with the 2013 IEEE GRSS Data Fusion Contest for Houston). This yields more training data than the community-standard protocol and produces higher absolute accuracy figures as a result. The absolute accuracy reported under this protocol is therefore **not** directly comparable to the figures reported under fixed-count splits; Section 5.2 reports a bridging evaluation under the community-standard protocol for direct comparison. This distinction applies throughout the thesis and is not restated at each subsequent occurrence of a 70/30 result.

Note on Houston Dataset Protocol

The Houston 2013 dataset has a community-standard training/test split (15 labelled pixels per class for training) established by the IEEE GRSS Data Fusion Contest. The 70%/30% stratified split used in this thesis provides substantially more training data for Houston (approximately 10,500 training pixels versus the community-standard 150-300). Consequently, the Houston results reported in this thesis are **not directly comparable** to published results using the community-standard protocol. The bridging evaluation in Section 10.6.1 does not cover Houston because the community-standard training set size (15 pixels per class) is insufficient for training the SVM classifier used in the segmentation pipeline.

Bridging evaluation

To provide a direct anchor point for the published literature, the watershed-based enhanced framework was additionally evaluated on the Salinas Valley and Indian Pines datasets under a community-standard fixed-count split (200 labelled pixels per class, stratified). These bridging results are reported alongside the 70/30 figures in Chapter 10. All other cross-method comparisons were conducted exclusively using the 70/30 protocol to ensure internal consistency.

5.3 Performance Evaluation

This section describes the performance metrics and statistical tests used to evaluate the segmentation approaches and compare the baseline and enhanced methods.

5.3.1 Performance Metrics

The segmentation approaches were compared using four different performance metrics. The first metric is the overall accuracy (OA), which measures the proportion of correctly classified pixels in an image. This metric was selected because it provides a general indication of the segmentation quality. The OA was calculated as follows:

$$\mathrm{OA} = \frac{\sum_{i=1}^{C} n_{ii}}{N} \tag{5.1}$$

where n_{ii} is the number of correctly classified pixels for class i, C is the total number of classes, and N is the total number of pixels in an image.
The second metric is the average accuracy (AA), which is the mean of the per-class accuracies. This metric is important because it ensures that the performance of smaller classes is not overshadowed by larger classes. AA is calculated as follows:

$$\mathrm{AA} = \frac{1}{C} \sum_{i=1}^{C} \frac{n_{ii}}{n_i} \tag{5.2}$$

where n_i is the total number of pixels belonging to class i in the ground truth.
The third metric is the mean Intersection over Union (mIoU), also known as the Jaccard Index, which measures the overlap between the segmented region and ground truth. The IoU for a given class i is calculated as follows

$$\mathrm{IoU}_i = \frac{|A_i \cap B_i|}{|A_i \cup B_i|} \tag{5.3}$$

where A_i is the set of pixels predicted as class i, and B_i is the set of pixels belonging to class i in the ground truth. The mean IoU across all classes was then reported.
The fourth metric is the per-class F1-score, which is computed as the harmonic mean of the precision and recall for each class:

$$\mathrm{F1}_i = \frac{2 \cdot \mathrm{precision}_i \cdot \mathrm{recall}_i}{\mathrm{precision}_i + \mathrm{recall}_i} \tag{5.4}$$

where $\mathrm{precision}_i = n_{ii}/|A_i|$ and $\mathrm{recall}_i = n_{ii}/|B_i|$. From these per-class scores, the macro-averaged F1 (mF1) is computed as

$$\mathrm{mF1} = \frac{1}{C} \sum_{i=1}^{C} \mathrm{F1}_i \tag{5.5}$$

The mF1 metric assigns equal weight to all classes, regardless of size, making it particularly sensitive to minority-class performance. Classes occupying fewer than 0.5% of the labelled pixels are flagged as rare, and their individual F1 scores are reported separately to expose minority-class challenges that may be masked by OA and AA.
The fifth metric is the computational time required by the approach to produce a segmentation for a given image. This metric is important because one of the aims of this study was to maintain computational efficiency while improving segmentation accuracy. The baseline and enhanced approaches were compared using each of these metrics for every dataset.

5.3.2 Region Purity Metric

Overall accuracy, average accuracy, and mIoU, as defined in Equations 5.1–5.3, are all computed after the semantic labelling step, in which every segmented region or class is assigned a label by an SVM classifier trained on the labelled training split (Sections 6.3.3, 6.2.2, 6.4.2). Because this classifier is common to the baseline and enhanced variants of each approach, and because it is trained on the same generous 70% split used throughout this thesis, a high OA figure partly reflects the classifier's performance on well-separated mean spectral vectors rather than the quality of the spatial partitioning produced by the segmentation stage itself. To isolate the contribution of segmentation quality from the contribution of the downstream classifier, this thesis additionally reports a region purity score, computed prior to and independently of the SVM labelling step.
For a segmentation producing regions $R_1, \ldots, R_K$ and ground-truth classes $1, \ldots, C$, purity is defined as

$$\text{Purity} = \frac{1}{N} \sum_{k=1}^{K} \max_{c \in \{1,\ldots,C\}} |R_k \cap G_c| \,, \tag{5.6}$$

where G_c is the set of ground-truth pixels belonging to class c, and N is the total number of labelled pixels. Purity measures the degree to which each region produced by the segmentation stage is internally homogeneous with respect to the ground truth, independent of which label a downstream classifier ultimately assigns to that region. A method whose OA is high largely because of classifier strength, rather than segmentation quality, will show a purity score materially lower than its OA; a method whose OA gain is driven by genuinely better spatial partitioning will show purity and OA tracking closely.
The region purity results for the watershed baseline/enhanced pair, computed using Equation 5.6, are reported in Chapter 10 (Section 10.4).

5.3.3 Statistical Tests

The significance of the differences between the performances of the enhanced and baseline approaches was determined using a paired t-test. The paired t-test was used to determine whether the mean difference between two related samples was significantly different from zero. The test involves two hypotheses: the null hypothesis (H_0) which states that there is no significant difference between the means of the two approaches, and the alternative hypothesis (H_a) which states that the mean of the enhanced approach is significantly greater than that of the baseline approach. The test statistic t is calculated using the following equation:

$$t = \frac{\bar{d}}{s_d / \sqrt{n}} \tag{5.7}$$

where $\bar{d}$ is the mean of the differences between paired observations, s_d is the standard deviation of the differences, and n is the number of paired observations. In this study, each experiment was repeated over $n = 10$ independent runs per dataset; therefore, the degrees of freedom were $df = n - 1 = 9$. If the calculated t-value is greater than the critical value for a given confidence level and degrees of freedom, the null hypothesis can be rejected. The critical values for α-levels of 1%, 5%, and 10% (99%, 95%, and 90% confidence, respectively) with $n - 1$ degrees of freedom are obtained from the t-distribution table. The critical values for $df = 9$ are listed in Table 5.2.

Table 5.2: Critical values for the one-tailed paired t-test ($df = 9$).

Confidence Level	**Significance α**	**Critical t-value**
99%	0.01	2.821
95%	0.05	1.833
90%	0.10	1.383

In addition to statistical significance, the practical magnitude of each improvement was assessed using Cohen's d, which was calculated as

$$d = \frac{\bar{d}}{s_d} \tag{5.8}$$

where $\bar{d}$ is the mean of the differences between paired observations and s_d is the standard deviation of those differences. Following Cohen (1988), values of $d \geq 0.8$ are classified as large effects, $0.5 \leq d < 0.8$ as moderate, and $d < 0.5$ as small effects. This effect-size measure is reported alongside the paired t-test and Wilcoxon signed-rank outcomes for every enhanced-versus-baseline comparison in Chapter 10 to provide a practical, sample-size-independent indication of the magnitude of any observed improvement.

To verify that the significant conclusions do not depend on the normality assumption underlying the paired t-test, which cannot be rigorously confirmed at $n = 10$, a Wilcoxon signed-rank test was applied to all pairwise comparisons as a non-parametric supplement. For each enhanced-versus-baseline comparison, the signed-rank statistic W and the associated two-tailed p-value were computed, and the null hypothesis of no difference between the enhanced and baseline distributions was rejected if $p < \alpha$ for the chosen significance level. For $n = 10$ paired observations, the critical value is $W = 3$ at a two-tailed $\alpha = 0.05$. This test was applied uniformly across every framework pair, dataset, and metric considered in this study, providing a distribution-free cross-check of the conclusions drawn from the paired t-test. The full set of computed Wilcoxon statistics is reported in Chapter 10.

5.3.4 Multi-Split Variance Protocol

The paired t-test and Wilcoxon signed-rank test reported in Section 5.3.3 were computed over ten independent algorithm runs on a single 70%/30% stratified split per dataset. This design captures the stochastic variance of the search algorithm itself, arising from random initialisation, mutation, and perturbation, but does not capture the variance arising from the particular partition of labelled pixels into training and test sets. A method that appears to outperform its baseline on one fixed split may not do so on a different split drawn from the same dataset, and the ten-run protocol alone cannot distinguish between these two sources of variance.

To address this, the core comparisons in this study, namely the enhanced watershed pipeline against its baseline and the enhanced deep learning pipeline against its baseline HED model, were additionally evaluated across five independently drawn 70%/30% stratified splits per dataset, using distinct master seeds for the split itself (as opposed to the algorithm's internal stochasticity, which continued to use the ten-seed protocol of Section 5.3.3 within each split). This yields a $5 \times 10 = 50$ observation grid per dataset per method, from which two variance components are reported separately as follows:

- **Within-split variance**: the standard deviation across the ten algorithmic reruns on a single split, as reported throughout Chapter 10.
- **Across-split variance**: the standard deviation of the per-split mean OA across the five splits, which isolates the sensitivity of the reported accuracy to the particular partition of labelled pixels.

A method is considered to show a robust improvement over its baseline only if the paired t-test on the split-level means (rather than the run-level observations) remains significant at $p < 0.05$ with $n = 5$ splits (df $= 4$, critical $t = 2.132$ at 95% confidence, one-tailed). This is a stricter and more conservative test than the run-level test reported in Section 5.3.3 and is treated as the primary claim of robustness in Chapter 11.

The results of this multi-split evaluation, reported separately by within-split and across-split variance and cross-checked against the split-level paired t-test criterion above, are given in Chapter 10 (Section 10.4).

5.4 Technical Specifications

The approaches were developed on a computer with the following specifications: Intel(R) Core(TM) i9-10900K CPU @ 3.70GHz, 64GB RAM, NVIDIA GeForce RTX 3080 GPU, and 64-bit Windows 10 operating system. Deep learning approaches were implemented using Python 3.9 with the PyTorch 1.12 framework. The simulations were performed on the same machine, and all experiments were executed in a controlled environment to ensure consistent runtime measurements.

Chapter 6

Computational Intelligence for Image Segmentation

6.1 Overview

This chapter presents the three segmentation baselines used throughout this study: multilevel thresholding, watershed transform, and holistically-nested edge detection (HED). Each represents a distinct segmentation paradigm: statistical partitioning of the intensity histogram, topographic region growth, and learned edge detection. Section 6.2 describes multilevel thresholding, Section 6.3 describes the watershed transform, and Section 6.4 describes HED. For each method, the underlying problem formulation and baseline algorithmic implementation are described, followed by the experimental protocol used for its evaluation.

6.2 Multilevel Thresholding

6.2.1 Introduction

Multilevel thresholding (MLT) is a core image segmentation task in which a set of k intensity thresholds partitions an image histogram into $k+1$ classes [2]. It is one of the most widely adopted segmentation techniques because of its conceptual simplicity and low computational cost relative to region-growing or learned approaches [2]. The threshold vector $\mathbf{t} = \{t_1, t_2, \ldots, t_k\}$ is chosen to optimise a statistical objective, such as Otsu's between-class variance [68] or Kapur's entropy [37]. Because the exhaustive search over the threshold space scales as $O(L^{k-1})$, where L is the number of gray levels, MLT is NP-hard, and an exact search becomes impractical once $k > 4$.

Metaheuristic optimisers avoid this exponential cost by guiding a stochastic search through the threshold space rather than enumerating it. This study uses two such optimisers as baselines: the genetic algorithm (GA) [28] and artificial bee colony (ABC) [38]. Both maintain a population of candidate threshold vectors and iteratively refine them toward higher fitness, without requiring an exhaustive scan of the threshold space.

This section describes the problem formulation and objective functions optimised by these baseline metaheuristics, their algorithmic implementation, the preprocessing and region-classification steps used to apply them to hyperspectral data, and the experimental protocol used to evaluate them.

6.2.2 Baseline Model

Problem Formulation

Let I be a single-band intensity image with L intensity levels $\{0, 1, \ldots, L-1\}$, and let $h(i)$ be the histogram count at level i. A threshold vector $\mathbf{t} = \{t_1, t_2, \ldots, t_k\}$ with $0 < t_1 < t_2 < \cdots < t_k < L-1$ partitions the image into $k+1$ classes $C_0, C_1, \ldots, C_k$:

$$\begin{aligned} C_0 &= \{(x,y) \mid 0 \le I(x,y) \le t_1 - 1\}, \\ C_j &= \{(x,y) \mid t_j \le I(x,y) \le t_{j+1} - 1\}, \quad 1 \le j \le k-1, \\ C_k &= \{(x,y) \mid t_k \le I(x,y) \le L-1\}. \end{aligned} \tag{6.1}$$

The goal of segmentation is to find the vector $\mathbf{t}^*$ that optimises a fitness function $f(\mathbf{t})$ defined over the partition in Equation 6.1.

Objective Functions

Two objective functions were used to drive the threshold search, following the two dominant families in MLT literature.

Kapur's Entropy. Let $p_i = h(i)/\sum_{j=0}^{L-1} h(j)$ be the normalised histogram probability. For class C_j, the class entropy is

$$H_j = -\sum_{i=t_j}^{t_{j+1}-1} \frac{p_i}{\omega_j} \ln \frac{p_i}{\omega_j}, \qquad \omega_j = \sum_{i=t_j}^{t_{j+1}-1} p_i, \tag{6.2}$$

with $t_0 = 0$ and $t_{k+1} = L$. The objective is to maximise $f_{\text{Kapur}}(\mathbf{t}) = \sum_{j=0}^{k} H_j$.

Otsu's Between-Class Variance. Otsu's method [68] maximises the separability of the classes:

$$\sigma^2_{\text{Between}}(\mathbf{t}) = \sum_{j=0}^{k} \omega_j (\mu_j - \mu_T)^2, \tag{6.3}$$

where μ_j is the mean intensity of class C_j and μ_T is the global mean. Both objectives were used in this study, as neither dominates the other across all histogram shapes: Kapur's entropy is more sensitive to the presence of small, low-population modes, while Otsu's variance favours well-separated, high-contrast classes.

Baseline Genetic Algorithm

The baseline GA uses integer-valued chromosomes of length k, with each gene an intensity in $[1, 254]$. Before fitness evaluation, the chromosome is sorted to enforce $t_1 < t_2 < \cdots < t_k$. Tournament selection (size 3) selects parents, single-point crossover is applied at a random locus with probability $p_c = 0.8$, and mutation replaces a single gene with a uniform random integer in $[1, 254]$ with probability $p_m = 0.1$ per gene. Elitism preserves the best individual across generations. The population size is 50, and the algorithm runs for a maximum of 100 generations.

Baseline Artificial Bee Colony

The baseline ABC maintains $SN = 30$ food sources, each of which represents a threshold vector. In the employed-bee phase, a new candidate $v_{i,j}$ is generated for a randomly chosen dimension j by

$$v_{i,j} = x_{i,j} + \phi_{i,j}(x_{i,j} - x_{k,j}), \tag{6.4}$$

where $k \neq i$ is a randomly selected food source and $\phi_{i,j} \sim U[-1, 1]$. Greedy selection retains the fitter of x_i and v_i. Onlooker bees then probabilistically select sources via roulette-wheel selection based on normalised fitness, and scout bees abandon any source that has not improved after limit = 10 trials, replacing it with a new random vector. The colony was evaluated for up to 50 cycles.

6.2.3 Baseline Multilevel Thresholding Segmentation Pipeline

Multilevel thresholding, as formulated in Section 6.2.2, operates on a single-band-intensity image. To apply it to hyperspectral data, the threshold search is combined with a dimensionality-reduction step and a region-classification step, following the same overall pattern used for the baseline watershed pipeline (Section 6.3.3).

1. **PCA transform:** Principal Component Analysis is applied to the hyperspectral cube, and the first principal component is retained as the single-band intensity image I used for thresholding.
2. **Threshold search:** The baseline GA or ABC (Sections 6.2.2 and 6.2.2) searches for the threshold vector $\mathbf{t}^*$ that optimises Kapur's entropy or Otsu's between-class variance on I, partitioning it into $k + 1$ classes as in Equation 6.1.

3. **Region assignment:** Every pixel is assigned to the class C_j containing its intensity value under $\mathbf{t}^*$.

4. **Region classification:** Each resulting class is assigned a semantic label by applying a Support Vector Machine (SVM) classifier (RBF kernel) to the mean spectral vector of that class, computed from the original hyperspectral cube.

6.2.4 Experimental Design

Datasets

The baseline multilevel thresholding pipeline was evaluated on the same five benchmark hyperspectral datasets used throughout this study: Indian Pines, Pavia University, Salinas Valley, Houston, and Botswana. The characteristics of these datasets and the training/testing protocol used to split their ground-truth labels are described in Chapter 5. Each algorithm was evaluated at nine threshold levels ($k = 2, \ldots, 10$) under both objective functions.

Evaluation Metrics

The segmentation approach was evaluated using the five performance metrics defined in Section 5.3.1 of Chapter 5: overall accuracy (OA), average accuracy (AA), mean Intersection over Union (mIoU), per-class F1-score, macro-averaged F1 (mF1), and computational time. These metrics were computed on the region-classified output described in Section 6.2.3, placing the thresholding pipeline on the same evaluation footing as the watershed and HED baselines.

Implementation Details

Table 6.1 summarises the fixed parameters used for the baseline GA and ABC. All parameter settings were fixed across images and threshold levels to reflect a practical scenario in which a practitioner deploys an algorithm without image-specific tuning. The region classifier used an SVM with an RBF kernel, consistent with the classifier used in the baseline watershed pipeline (Section 6.3.3).

Table 6.1: Parameter settings for the baseline multilevel thresholding algorithms.

Algorithm	Parameter / Value
GA (baseline)	Pop. size 50, crossover 0.8, mutation 0.1, max. gen. 100
ABC (baseline)	Colony size 30, max. cycles 50, abandonment limit 10

All experiments used a fixed sequence of 30 random seeds per algorithm, generated from a single master seed using NumPy's default random generator, and applied identically across both algorithms so that stochastic variability was shared rather than confounded across the comparison.

6.2.5 Conclusion

This section presents the baseline implementation of multilevel thresholding used in this study: two base optimisers, GA and ABC, were evaluated against two histogram-partitioning objectives, Kapur's entropy and Otsu's between-class variance, applied to hyperspectral data via PCA-based dimensionality reduction and SVM-based region classification. The experimental protocol, datasets, and evaluation metrics used to assess this baseline are also described.

6.3 Watershed Transform

6.3.1 Introduction

Among the classical region-based approaches reviewed in Section 4, the watershed transform (WS) ensures boundaries that are both closed and continuous by interpreting the image intensity profile as a topographical surface [44, 46]. In this analogy, bright pixels represent peaks, and dark pixels represent valleys. Water floods the valleys from the lowest points, and dams are built where the water from different valleys meets. These dams form the boundaries of the regions.
The main strength of the watershed algorithm is that it always produces closed boundaries around the regions. However, it has a weakness: it is highly sensitive to noise and small changes in pixel

values [86]. This causes the algorithm to create far too many tiny regions, a problem known as *over-segmentation* [44]. When this occurs, a single object is split into many meaningless pieces, which harms both the accuracy and processing speed.
This section describes the mathematical formulation of the watershed transform, the marker-controlled variant used to reduce over-segmentation, and the baseline segmentation pipeline used in this study.

6.3.2 Baseline Watershed Model

The Watershed Transform

In the watershed transform, the regional minima of the surface represent the deepest locations within the valleys. As the water level increases from these minima, it fills the surrounding area. When water from two different valleys is about to merge, a dam is built to keep them apart. The complete set of dams forms the *watershed lines*, which are the final segment boundaries.
In practice, the watershed does not operate directly on a raw image. Instead, it operates on a *gradient image* that shows how quickly the pixel values change across the image. Sharp edges in the original image appear as high ridges in the gradient image, and these ridges guide the placement of the watershed dams.
Mathematically, D is the gradient image. A catchment basin $CB(m_i)$ is the set of all pixels that are topographically closer to a regional minimum m_i than to any other minimum, as follows:

$$CB(m_i) = \{p \in \mathbb{Z}^2 \mid \forall j \neq i : T_D(p, m_i) < T_D(p, m_j)\}, \tag{6.5}$$

where $T_D(p, q)$ is the topographic distance between points p and q on surface D. The watershed line is defined as everything that does not belong to any catchment basin:

$$WS(D) = \left[\bigcup_{i=1}^{n} CB(m_i)\right]^c \tag{6.6}$$

This process guarantees closed contours, which are useful for separating touching objects. However, because every small local minimum creates its own catchment basin, noise and texture cause thousands of spurious regions. This oversegmentation makes the raw watershed output difficult to use directly [44].

Marker-Controlled Watershed

To reduce oversegmentation, a marker-controlled watershed was developed. Instead of flooding from every local minimum, this method only floods from a predefined set of *markers* that represent the true objects and background. The markers act as seeds, and the watershed is only allowed to create boundaries where the water from different seeds meets [52].
The quality of the result depends entirely on the quality of the markers. If the markers are missing, the objects are merged (under-segmentation). Oversegmentation occurs in the presence of spurious markers. The method also remains sensitive to preprocessing choices, such as the amount of smoothing applied before computing the gradient [44].

6.3.3 Baseline Watershed Segmentation Pipeline

The baseline model uses a marker-controlled watershed with fixed and manually chosen parameters. The pipeline operates as follows:

Preprocessing

Before segmentation, all hyperspectral images underwent two stages of preprocessing.

Dimensionality reduction. Hyperspectral images contain hundreds of spectral bands, many of which are highly correlated or noisy. Principal Component Analysis (PCA) was applied to reduce the data to a smaller set of components that capture the most important information [7]. In the baseline model, the first three principal components were used to create an RGB-like representation of the image.

Gradient calculation. The watershed algorithm requires a topographic surface to be effective. This was created by computing a gradient map from the PCA-reduced image using the Sobel operator. High values in the gradient map correspond to sharp edges in the original image, which guide the placement of the watershed boundaries.

Segmentation Pipeline

1. **PCA transform:** The first three principal components are used to create an RGB-like representation.
2. **Gradient computation:** Sobel filters are applied to compute the gradient magnitude.
3. **Marker generation:** Otsu's thresholding [68] is applied to the gradient map to create a binary foreground mask. The connected components of this mask serve as **internal (object) markers** representing the interior of the candidate regions. The complement of this mask, after morphological dilation, serves as **external (background) markers**, constraining the watershed flooding to avoid boundary leakage. The watershed transform floods from internal markers while being constrained by external markers.
4. **Watershed flooding:** The watershed transform runs on the gradient image using the internal markers as flooding seeds. Dams are constructed where water from different seeds meets.
5. **Region assignment:** Unlabelled pixels are assigned to the nearest labelled region using a distance metric.
6. **Region classification:** Each resulting region is assigned a semantic class label by applying a Support Vector Machine (SVM) classifier (RBF kernel) to the mean spectral vector of that region, computed from the original hyperspectral cube.

Because the threshold and filter parameters are fixed, this pipeline depends on manual parameter choices that may not transfer well when the image characteristics differ from those assumed during tuning.

6.3.4 Experimental Design

Experimental Protocol

All images were consistently preprocessed. Pixel values were normalised to the range $[0, 1]$, and PCA was applied to reduce the dimensionality. The pixel spectra were then standardised to zero mean and unit variance to improve the classifier performance.
For each dataset, labelled pixels were split into training and testing sets following the protocol described in Chapter 5. The training set was used to train the region classifier, and the test set was reserved for the final evaluation. Watershed segmentation was applied to the full image, and the resulting regions were classified using the trained classifier.

Evaluation Metrics

The segmentation approach was evaluated using the five performance metrics defined in Section 5.3.1 of Chapter 5: overall accuracy (OA), average accuracy (AA), mean Intersection over Union (mIoU), per-class F1-score and macro-averaged F1 (mF1), and computational time.

Implementation Details

The baseline watershed pipeline used the parameter settings summarised in Table 6.2. The SVM classifier used an RBF kernel with default regularisation.

6.3.5 Conclusion

This section presents the baseline watershed segmentation pipeline used in this study: a marker-controlled watershed built on PCA dimensionality reduction, Sobel gradient computation, Otsu-thresholded marker generation, and SVM-based region classification. The experimental protocol, datasets, and evaluation metrics used to assess this baseline are also described.

Table 6.2: Parameter settings for the baseline watershed segmentation pipeline.

Stage	Setting
PCA components	First 3 principal components (RGB-like representation)
Gradient operator	Sobel
Marker thresholding	Otsu's method
Region classifier	SVM, RBF kernel

6.4 Holistically-Nested Edge Detection

6.4.1 Introduction

Deep learning is used for image segmentation and edge detection because it can learn patterns directly from data. For tasks such as edge detection and boundary delineation, deep networks can represent both fine details and broad structural patterns when provided with sufficient training data [54, 94]. This section describes the Holistically-Nested Edge Detection (HED), the deep learning baseline used in this study, together with the experimental protocol used to evaluate it.

6.4.2 Baseline Deep Learning Model

Holistically-Nested Edge Detection

The baseline deep learning model used in this study was the Holistically-Nested Edge Detection (HED) network proposed by Xie and Tu [94]. HED is built on a convolutional neural network and produces multiple intermediate outputs referred to as *side outputs* at various layers of the network. These side outputs capture edge information at different scales, allowing the model to detect both fine and coarse boundaries. In this study, HED was used with pre-trained VGG-16 weights that were kept fixed throughout.

Multi-Scale Feature Representation

The five side outputs of the HED capture complementary information at different spatial scales. Early layers focus on local intensity changes and fine details, whereas deeper layers capture larger structures and object-level boundaries [94]. These outputs, $\mathcal{L} = \{S_1, S_2, S_3, S_4, S_5\}$, form the edge representation of the network.

$$\mathcal{L} = \{S_1, S_2, S_3, S_4, S_5\} \tag{6.7}$$

Edge Map to Segmentation Label Conversion

HED is fundamentally an edge detector, and its output is a continuous-valued edge probability map $P(i, j) \in [0, 1]$ rather than a directly labelled segmentation map. The conversion from this edge representation to the region-label output required for computing OA, AA, and mIoU proceeds through four steps:

1. **Binary edge map:** The probability map $P(i, j)$ is thresholded at 0.5,

$$B(i, j) = \mathbb{1}_{\{P(i,j)>0.5\}}, \tag{6.8}$$

   to produce a binary edge mask, followed by morphological skeletonisation to reduce edge thickness to one pixel.

2. **Region extraction:** The complement of the binary edge mask defines a set of closed interior regions. Connected-component labelling (8-connectivity) is applied to the non-edge pixels to assign a unique integer label ℓ_k to each connected component R_k.

3. **Spectral classification of regions:** Each region R_k is assigned a semantic class label by applying a Support Vector Machine (SVM) classifier (RBF kernel, $C = 1$, $\gamma =$ scale) to the mean spectral vector of that region, computed from the original hyperspectral cube. The SVM was trained on the labelled pixels in the training split.

4. **Pixel-level label assignment:** Every pixel (i, j) belonging to region R_k is assigned the class label of that region, producing the final pixel-level label map used to compute OA, AA, and mIoU against the ground truth.

This pipeline ensures that the edge-detection output of HED is converted to a region-labelled map on a common evaluation footing with watershed and thresholding methods. The choice of SVM for region classification is consistent with the classifier used in the baseline watershed pipeline (Section 6.3.3) and removes any classification-induced performance differences between the methods.

6.4.3 Experimental Design

This subsection describes the experimental design used to evaluate the baseline HED model on benchmark hyperspectral datasets.

Experimental Protocol

Table 6.3 summarises the experimental protocol for each dataset. The training set was used to train the region classifier, whereas the test set was reserved for the final evaluation only.

Table 6.3: Experimental protocol per dataset

Dataset	Training/Validation Set	Test Set
Indian Pines	70% train, 30% validation	30% hold-out test
Pavia University	70% train, 30% validation	30% hold-out test
Salinas Valley	70% train, 30% validation	30% hold-out test
Houston	70% train, 30% validation	30% hold-out test
Botswana	70% train, 30% validation	30% hold-out test

Evaluation Metrics

The segmentation approach was evaluated using the five performance metrics defined in Section 5.3.1 of Chapter 5: overall accuracy (OA), average accuracy (AA), mean Intersection over Union (mIoU), per-class F1-score and macro-averaged F1 (mF1), and computational time.

Implementation Details

The approach was developed on a computer with the following specifications: Intel(R) Core(TM) i9-10900K CPU @ 3.70GHz, 64GB RAM, NVIDIA GeForce RTX 3080 GPU, and 64-bit Windows 10 operating system. The model was implemented using Python 3.9 with the PyTorch 1.12 framework. HED VGG-16 weights were frozen throughout, consistent with the fixed-baseline setting described in Section 6.4.2.

6.4.4 Conclusion

This section presents the holistically nested edge detection, the deep learning baseline used in this study, its multi-scale side-output representation, and the pipeline used to convert its edge probability map into a labelled segmentation for evaluation. The experimental protocol, datasets, and evaluation metrics used to assess this baseline are also described.

6.5 Chapter Summary

This chapter presents the three baseline segmentation approaches used throughout this study: multilevel thresholding, the watershed transform, and holistically nested edge detection. Each method is described in terms of its problem formulation and algorithmic implementation, and each was evaluated on its own benchmark corpus under a consistent experimental protocol.

Chapter 7

Automated Design of Multilevel Thresholding

7.1 Introduction

Chapter 6 describes the baseline genetic algorithm (GA) and artificial bee colony (ABC) optimisers used for multilevel thresholding (MLT), together with the two objective functions, Kapur's entropy and Otsu's between-class variance, which they optimise. Both optimisers depend on hyperparameters, such as population size, mutation rate, number of generations or cycles permitted, and the quality of the resulting segmentation is sensitive to these settings. Poor choices lead to premature convergence or wasted evaluations, and the best setting is rarely stationary across images or threshold levels [31].

Automated algorithm design replaces manual hyperparameter tuning by treating the hyperparameter vector of the base optimiser as a point in a configuration space and searching for a setting that maximises performance on a given problem distribution [31]. This chapter describes four such design strategies applied on top of the baseline GA and ABC: a search enhancement that operates directly on threshold vectors (iterated local search, ILS), a single-point configurator that navigates the hyperparameter space via a trajectory-based search (simulated annealing, SA), and two multipoint configurators that maintain a population of candidate hyperparameter vectors (a meta-genetic algorithm and differential evolution, DE).

This chapter describes the taxonomy of design strategies, mechanism of each configurator, protocol used to equalise the evaluation budget across all variants, and experimental design used to evaluate them.

7.2 Motivation for Automated Design

A fundamental divide in automated algorithm design separates single-point configurators, which navigate the configuration space via a trajectory-based search, from multipoint configurators, which maintain a population of candidate configurations. Representative single-point approaches include ParamILS [31] and simulated annealing-based tuners [87]. Representative multipoint approaches include racing procedures such as irace [58] and meta-genetic algorithms [25]. Surrogate-model methods such as SMAC [32] and BOHB [21] extend this toolkit further but incur substantial per-evaluation overhead that can exceed the cost of the underlying MLT fitness evaluation, which requires only a histogram sweep over the region-classified output described in Section 6.2.3. Therefore, derivative-free single-point and multipoint configurators are the relevant candidates for this domain.

A separate design choice concerns whether the search enhancement operates on the base optimiser's hyperparameters or directly on the candidate threshold vectors. Iterated local search (ILS) [56] and simulated annealing (SA) [87] can, in principle, be applied at either level, and conflating the two interventions in a single comparison risks confounding a configuration effect with a search effect. This chapter keeps the two levels explicit: ILS is applied as a search enhancement directly on threshold vectors, whereas SA, the meta-GA, and DE are applied as configurators of the base optimiser's hyperparameters.

Because multipoint configurators split their evaluation budget between an outer configuration loop and an inner threshold search, whereas the baseline invests all evaluations directly into the threshold search, an uncontrolled comparison between design strategies would confound the strategy choice

with the resulting difference in resources. Section 7.3.5 describes the protocol used to equalise this budget across all variants.

7.3 Automated Design Framework

7.3.1 Taxonomy of Design Strategies

The base optimiser (GA or ABC, Sections 6.2.2 and 6.2.2) was held constant, and four design strategies were applied to each, in addition to the unmodified baseline: search-enhanced ILS, single-point SA configuration, and two multipoint evolutionary configurators (meta-GA and DE). This produced the ten algorithmic variants summarised in Table 7.1.

Table 7.1: Automated design taxonomy for multilevel thresholding. The base optimiser (GA or ABC) is held constant; the design strategy is varied.

Base Optimiser	Strategy Type	Design Strategy	Abbreviation
Genetic Algorithm (GA)	Baseline	Fixed defaults	GA
	Search enhancement	ILS on threshold vectors	ILSGA
	Single-point config.	SA on hyperparameters	SAGA
	Multipoint config.	Meta-GA on hyperparameters	GAGA
	Multipoint config.	DE on hyperparameters	DEGA
Artificial Bee Colony (ABC)	Baseline	Fixed defaults	ABC
	Search enhancement	ILS on threshold vectors	ILSABC
	Single-point config.	SA on hyperparameters	SAABC
	Multipoint config.	Meta-GA on hyperparameters	GAABC
	Multipoint config.	DE on hyperparameters	DEABC

7.3.2 Search-Enhanced Trajectory Method: ILS

ILSGA and ILSABC do not reconfigure the hyperparameters of the base optimiser. Instead, after the base GA or ABC terminates or reaches an early-stopping criterion of 20 generations without improvement in the best fitness, the best solution undergoes a local search directly on the threshold vector. One threshold is perturbed by $\delta \in \{-5, \ldots, 5\}$, the fitness is re-evaluated, and the move is accepted if it improves the objective. A perturbation operator then disrupts multiple thresholds with larger steps $\Delta \in \{-20, \ldots, 20\}$ to escape local optima, after which the local search resumes. This intensification-and-diversification cycle is repeated for 20 ILS iterations.

7.3.3 Single-Point Configurator: Simulated Annealing

SAGA uses SA to configure the GA hyperparameters, population size, crossover rate, mutation rate, and maximum generations, before the final GA run. The SA state is $\theta = (\text{pop_size}, p_c, p_m, \text{max_generations})$, bounded to $[20, 80]$, $[0.5, 1.0]$, $[0.01, 0.3]$, and $[50, 150]$ respectively. Neighbouring configurations perturb integer-valued parameters by $\delta \in \{-2, -1, 1, 2\}$ and real-valued parameters by ± 0.05, clipped to their bounds. The fitness of a candidate configuration is the best GA fitness obtained when the GA is run with those parameters, and the best configuration found is used for the definitive GA execution.

SAABC uses the same SA procedure to configure the ABC hyperparameters, colony size, maximum cycles, and abandonment limit, with state $\theta = (\text{pop_size}, \text{max_iter}, \text{limit})$ bounded to $[15, 40]$, $[30, 80]$, and $[5, 15]$ respectively.

The outer SA layer of SAABC is deliberately configured to consume 50% of the available evaluation budget, roughly 2.5 times the fraction given to the multipoint configurators (Section 7.3.5), so that it functions as an overhead ablation within the experimental design: it isolates what happens when the overhead of a single-point configurator is allowed to scale well beyond the minimum viable fraction used by the other configurators, rather than serving as a representative single-point setting.

7.3.4 Multipoint Configurators: Meta-GA and Differential Evolution

GAGA uses a nested, two-level architecture in which an outer meta-GA evolves a population of eight GA parameter configurations $\mathbf{c} = (\text{pop_size}, \text{crossover_rate}, \text{mutation_rate}, \text{max_generations})$. Each configuration is evaluated by instantiating a GA with those parameters, running it on the target image, and returning the best objective value achieved. Tournament selection, uniform crossover (50% gene exchange), and random mutation (rate 0.3) act on the parameter genes over four meta-generations. The best configuration discovered is then used for a final, full-length GA run. GAABC is structurally identical but configures the ABC hyperparameters (colony size, maximum cycles, and abandonment limit).

DEGA replaces the outer meta-GA with a differential evolution. The DE population encodes real- and integer-valued parameter vectors for the inner GA, and mutation follows the DE/rand/1 scheme

$$\mathbf{v}_i = \mathbf{x}_{r_1} + F \cdot (\mathbf{x}_{r_2} - \mathbf{x}_{r_3}), \tag{7.1}$$

with $F = 0.5$ and distinct random indices r_1, r_2, r_3. Binomial crossover with rate $CR = 0.9$ produces the trial vectors, and the integer parameters are rounded and clipped to their bounds. The best individual after ten DE generations defined the GA configuration used for the final run. The DEABC applies the same DE/rand/1 mutation and binomial crossover to the ABC hyperparameters.

The outer DE fitness function is penalised to prevent it from exploiting degenerate inner configurations, such as near-zero mutation rates or abbreviated generation limits, which would otherwise collapse the inner search into a truncated random walk while appearing inexpensive to the outer search. Since the raw outer fitness metric (the best inner objective value) cannot distinguish an early, legitimate convergence from an inner run truncated before a meaningful search occurred, the corrected outer fitness discounts configurations whose inner run consumed less than 75% of the allocated inner budget, penalising the fitness proportionally to the shortfall. Both DEGA and DEABC use this penalised fitness function.

7.3.5 Budget Equalisation Protocol

Multipoint configurators split their evaluation budget between an outer configuration loop and an inner threshold search, whereas the baseline and search-enhanced methods invest all evaluations directly into the threshold search. An uncontrolled comparison would confound the strategy choice with the resulting difference in resources. To prevent this, the total number of fitness evaluations B_k is strictly equalised across all ten algorithms at each threshold level k. For nested configurator algorithms, the budget partitions as

$$B_k = E_{\text{outer}} + E_{\text{inner}}, \tag{7.2}$$

where E_{outer} is consumed by the configurator and E_{inner} is allocated to the final inner run, with all algorithms matched to within $\pm 5\%$ of the target budget at each k. The outer-loop fraction reflects the minimum viable configuration effort for each paradigm: 20% for the search-enhancement (ILS) and multipoint (meta-GA, DE) families, and 50% for the SA-based single-point configurators. This asymmetry reflects a structural difference in per-candidate cost rather than an arbitrary choice: each SA candidate configuration requires a full inner-algorithm run to evaluate, which is substantially more expensive per candidate than a perturbation-and-local-search cycle or a multipoint generation step, so a much larger fraction of the budget is consumed even at the minimum functional setting.

7.3.6 Outer-Loop Budget Sensitivity

The 20% outer-loop fraction used for the multipoint configurators (GAGA, DEGA) above was selected as the minimum functional configuration effort for that strategy. To confirm that GAGA's underperformance relative to the baseline GA (Section 10.3) is a genuine property of the strategy at low outer-loop budgets, rather than an artefact of the particular 20% figure chosen, GAGA was additionally evaluated at outer-loop fractions of 10%, 35%, and 50%, with the corresponding inner-loop budget reduced to keep the total evaluation count B_k fixed at each threshold level k, following the protocol in Section 7.3.5.

The sweep results in Table 7.2 demonstrate that GAGA's deficit narrows monotonically as the outer loop fraction increases: from -16.0% OA at 10% outer fraction to -2.7% at 50% on Indian Pines, and from -19.0% to -2.9% on Salinas Valley. However, even at 50% outer fraction, GAGA does not close the gap to the baseline GA, with Cohen's d values of 1.56 and 1.79 remaining in the large-effect range ($d \geq 0.8$). This confirms that the underperformance is not an artefact of the particular 20%

budget split chosen, but rather a structural limitation of the nested configuration approach under the evaluation budgets that were tested.

Table 7.2: GAGA outer-loop budget sweep, Indian Pines and Salinas Valley shown. OA reported relative to Baseline GA at matched total evaluation budget B_k.

Dataset	Outer fraction	OA (%)	Δ OA vs. Baseline GA	Cohen's d
Indian Pines	10%	58.1	−16.0	9.22
	20% (original)	63.8	−10.3	5.94
	35%	69.2	−4.9	2.83
	50%	71.4	−2.7	1.56
Salinas Valley	10%	67.2	−19.0	11.77
	20% (original)	75.1	−11.1	6.88
	35%	80.5	−5.7	3.53
	50%	83.3	−2.9	1.79

7.4 Experimental Design

7.4.1 Datasets

The automated design strategies were evaluated on the same five benchmark hyperspectral datasets used throughout this study: Indian Pines, Pavia University, Salinas Valley, Houston, and Botswana. The characteristics of these datasets and the training/testing protocol used to split their ground-truth labels are described in Chapter 5. Each of the ten algorithmic variants was evaluated at nine threshold levels ($k = 2, \ldots, 10$) under both objective functions, using the same PCA-reduction, threshold-search, region-assignment, and SVM region-classification pipeline described in Section 6.2.3.

7.4.2 Evaluation Metrics

The automated design strategies were evaluated using the five performance metrics defined in Section 5.3.1 of Chapter 5: overall accuracy (OA), average accuracy (AA), mean Intersection over Union (mIoU), per-class F1-score, and macro-averaged F1 (mF1), and computational time. These metrics were computed on the region-classified output of each variant's final threshold search, placing all ten variants on the same evaluation footing as the baseline described in Chapter 6.

7.4.3 Implementation Details

Table 7.3 summarises the fixed and searched parameters for the nine automated design variants, in addition to the baseline GA and ABC settings given in Table 6.1. For nested methods, the values denote the bounds searched by the outer configurator, not fixed settings. All parameter settings were fixed across images and threshold levels to reflect a practical scenario in which a practitioner deploys an algorithm without image-specific tuning.

Table 7.3: Parameter settings for the nine automated design variants. Nested methods use the listed values as the outer configurator's search bounds.

Algorithm	Parameter / Range
ILSGA / ILSABC	Perturbation $\delta \in [-5, 5]$, 20 ILS iterations
SAGA outer SA	Pop. size $[20, 80]$, max. iter. $[50, 150]$, mutation $[0.01, 0.3]$
SAABC outer SA	Pop. size $[15, 40]$, max. iter. $[30, 80]$, limit $[5, 15]$
GAGA / GAABC outer GA	Meta-pop. 8, meta-generations 4, meta-mutation 0.3
DEGA / DEABC outer DE	Pop. 10, $F = 0.5$, $CR = 0.9$

Table 7.4 lists the total fitness evaluation budget allocated to each variant at three representative threshold levels, following the equalisation protocol described in Section 7.3.5.

Table 7.4: Total fitness evaluation budget B_k per algorithm. Nested methods split the budget between the outer (E_{outer}) and inner (E_{inner}) evaluation. All totals are equalised within each threshold level.

	$k = 2$		$k = 5$		$k = 10$		Outer	
Algorithm	E_{outer}	E_{inner}	E_{outer}	E_{inner}	E_{outer}	E_{inner}	fraction	type
GA / ABC	–	5,000	–	10,000	–	20,000	–	–
ILSGA / ILSABC	1,000	4,000	2,000	8,000	4,000	16,000	20%	search
SAGA / SAABC	2,500	2,500	5,000	5,000	10,000	10,000	50%	config
GAGA / GAABC	1,000	4,000	2,000	8,000	4,000	16,000	20%	config
DEGA / DEABC	1,000	4,000	2,000	8,000	4,000	16,000	20%	config

All experiments used a fixed sequence of random seeds per algorithm, generated from a single master seed using NumPy's default random generator, applied identically across all ten variants so that stochastic variability was shared rather than confounded across comparisons.

7.5 Conclusion

This chapter presents the automated design strategies applied to the baseline multilevel thresholding optimisers described in Chapter 6: a search-enhanced trajectory method (ILS) that operates directly on threshold vectors, a single-point configurator (SA) that navigates the base optimiser's hyperparameter space via a trajectory-based search, and two multipoint configurators (meta-GA and DE) that maintain a population of candidate hyperparameter vectors. A strict budget-equalisation protocol ensures that the total number of fitness evaluations is matched across all ten variants at each threshold level, so that any differences in outcome reflect the design strategy rather than the resources available. The experimental protocol, datasets, and evaluation metrics used to assess these variants are also described.

Chapter 8

Watershed Transform Hybridisation

8.1 Introduction

Chapter 6 describes the baseline marker-controlled watershed pipeline used in this study: PCA-based dimensionality reduction, Sobel gradient computation, Otsu-thresholded marker generation, watershed flooding, and SVM-based region classification (Section 6.3.3). This pipeline guarantees closed, continuous boundaries, but its quality depends heavily on a set of manually fixed parameters, the amount of preprocessing smoothing, the gradient threshold used for marker generation, and the minimum region size, none of which transfer well across datasets with different noise characteristics and spectral properties.

One common way to address this is to automatically tune a single parameter of the pipeline. The Watershed-PSO (WS-PSO) hybrid, described in Section 8.3.2, uses particle swarm optimisation to adjust the gradient threshold that controls marker selection. Although this helps, particle swarm optimisation can still converge on poor solutions and only addresses one stage of the pipeline.

This chapter describes an enhanced watershed approach, Watershed-GA (WS-GA), that uses a genetic algorithm (GA) to optimise the entire watershed pipeline rather than a single parameter. WS-GA is best characterised as an evolutionary metaheuristic that optimises pipeline parameters directly, rather than a hyper-heuristic that selects among predefined heuristic configurations [10]. Instead of manually choosing filter sizes, gradient thresholds, and marker rules, the GA searches for the best combination of these settings jointly, using selection, crossover, and mutation operators to escape local optima and find more reliable configurations.

This chapter describes the motivation for watershed enhancement, the WS-PSO benchmark, the proposed WS-GA framework, and the experimental design used to evaluate both against the baseline watershed pipeline.

8.2 Motivation for Watershed Enhancement

The baseline watershed algorithm, as described in Section 6.3.3, suffers from three main problems when applied to hyperspectral images:

1. **Oversegmentation:** The algorithm creates too many small regions because it treats every tiny intensity fluctuation as a separate valley. In hyperspectral images, sensor noise and spectral variability produce many such fluctuations [7].

2. **Parameter sensitivity:** The quality of the segmentation depends heavily on preprocessing filters, gradient computation, and marker thresholds. Poor choices cause boundaries to leak or regions to fragment [44].

3. **Manual tuning:** Finding good parameters by hand requires trial and error for each new dataset, which is impractical for an automated pipeline.

Several strategies have been proposed to address these issues. Particle swarm optimisation (PSO) has been widely used to automatically tune watershed thresholds [80]. PSO moves a swarm of candidate solutions through the parameter space and updates them based on personal and global

best experiences. Although effective, PSO can converge prematurely and is sensitive to its own control parameters.
Other methods include ant colony optimisation, grey wolf optimisation, and fuzzy marker generation. These methods enhance reliability; however, they focus on adjusting only one or two components of the pipeline, for instance, only the threshold or only the markers [105].
The key challenge is to optimise the entire watershed pipeline, preprocessing, gradient construction, marker selection, and classification in a single framework. This chapter addresses this challenge using a genetic algorithm that co-optimises all these components simultaneously.

8.3 Enhanced Watershed Framework

8.3.1 Conceptual Overview

The enhanced watershed framework proposed in this study consists of two variants, which were evaluated alongside the baseline watershed pipeline described in Chapter 6:

1. **Watershed-PSO (WS-PSO):** A benchmark hybrid that uses particle swarm optimisation to tune the gradient threshold.
2. **Watershed-GA (WS-GA):** The proposed framework that uses a genetic algorithm to optimise preprocessing, segmentation, and classification parameters together.

Both variants share the baseline pipeline's preprocessing structure (PCA dimensionality reduction followed by Sobel gradient computation, Section 6.3.3) but differ in the parameters that are automatically tuned and how.

8.3.2 Watershed–PSO Benchmark

The WS-PSO model automates the gradient threshold tuning. Each particle in the swarm represents a candidate threshold value T in the range $[0.1, 0.5]$. The particles move through the search space according to the standard PSO update rules as follows:

$$\begin{aligned} \mathbf{v}_i^{t+1} &= \omega \mathbf{v}_i^t + c_1 r_1 \left(\mathbf{pbest}_i - \mathbf{x}_i^t\right) + c_2 r_2 \left(\mathbf{gbest} - \mathbf{x}_i^t\right), \\ \mathbf{x}_i^{t+1} &= \mathbf{x}_i^t + \mathbf{v}_i^{t+1}, \end{aligned} \tag{8.1}$$

where ω is the inertia weight, c_1 and c_2 are acceleration constants, r_1 and r_2 are random numbers between 0 and 1, $\mathbf{pbest}_i$ is the best position found by particle i, and $\mathbf{gbest}$ is the best position found by the entire swarm.
The fitness of each particle is measured by the overall classification accuracy of the segmentation produced using its threshold value minus a penalty for creating too many small regions. This approach automates a single parameter of the pipeline; all other settings remain fixed at the baseline values described in Section 6.3.3.

8.3.3 Watershed–GA Proposed Framework

The WS-GA model extends optimisation to the entire pipeline. Instead of tuning only one parameter, the GA searches for the best combination of preprocessing, segmentation, and classification settings.

Chromosome Encoding

Each individual in the GA population is called a *chromosome*. The chromosome is encoded as a real-valued parameter vector:

$$\mathbf{x} = \left[\, n_{\text{PCA}},\ \sigma,\ \tau_{\text{grad}},\ s_{\min},\ c_{\text{idx}} \,\right]^\top, \tag{8.2}$$

where each gene is bounded as follows

- $n_{\text{PCA}} \in \{1, 2, \ldots, 10\}$: Number of principal components to retain.
- $\sigma \in [0.1,\ 5.0]$: Standard deviation for Gaussian smoothing.
- $\tau_{\text{grad}} \in [0.01,\ 0.99]$: Gradient threshold for marker generation.
- $s_{\min} \in \{1, 2, \ldots, 500\}$: Minimum allowed region size in pixels.

- $c_{\text{idx}} \in \{0, 1, 2\}$: Choice of classifier (0 = Random Forest, 1 = SVM, 2 = k-NN).

Each chromosome therefore represents a complete configuration of the segmentation and classification pipelines, unlike the baseline model (Section 6.3.3) and WS-PSO (Section 8.3.2), which each fix all but one of these settings.

Genetic Operators

The GA evolves the population using three main operators:
Selection: The fittest individuals are chosen to become parents. This study uses steady-state selection, where the best individuals are kept from one generation to the next.
Crossover: Pairs of parent chromosomes exchange parts of their genetic code to create offspring. Single-point crossover is used with probability $p_c = 0.8$. This allows good settings from different parents to be combined.
Mutation: Random genes in a chromosome are changed with probability $p_m = 0.1$. This introduces new variations into the population and helps the algorithm escape local optima.

Fitness Function

The fitness of each chromosome measures how good its parameter set is at producing accurate and coherent segmentations. The fitness function is:

$$f = 0.55 \cdot \text{OA} + 0.45 \cdot \text{AA} - \text{coverage_penalty} - \text{smoothness_penalty}, \tag{8.3}$$

where OA is the overall accuracy, AA is the average accuracy, and the penalty terms discourage solutions that cover too few pixels or produce an excessive number of small spurious regions. The weighting of 0.55 for OA and 0.45 for AA reflects a deliberate bias toward global correctness while retaining meaningful sensitivity to per-class performance, since a purely OA-weighted objective tends to favour configurations that classify the dominant majority class well while largely ignoring smaller classes. The penalty terms are dataset-adaptive: target region counts and minimum coverage thresholds are set proportionally to the number of ground-truth classes and the spatial extent of the scene, as described in the implementation details (Section 8.4.3).

Computational Complexity of Fitness Evaluation. The dominant cost of each fitness evaluation is not the accuracy computation itself, but the full watershed pipeline that precedes it. Specifically, each evaluation involves (i) PCA projection at $\mathcal{O}(H \cdot W \cdot B \cdot n_{\text{PCA}})$, (ii) Sobel gradient computation at $\mathcal{O}(H \cdot W)$, (iii) marker extraction and watershed flooding at $\mathcal{O}(H \cdot W \log(H \cdot W))$, and (iv) segment classification at $\mathcal{O}(n_s \cdot n_{\text{PCA}})$, where n_s is the number of segments. Therefore, the overall per-evaluation complexity is $\mathcal{O}(H \cdot W \cdot B)$, dominated by the PCA step over the full spectral cube. This cost is incurred once per fitness evaluation during the offline optimisation phase and does not affect the cost of applying the resulting configuration afterwards.

Algorithm Procedure

Algorithm 2 shows the complete WS-GA.
The algorithm stops when either 100 generations have passed or the best fitness has not improved by more than $\epsilon = 0.001$ for ten consecutive generations.

8.4 Experimental Design

8.4.1 Experimental Protocol

All images were preprocessed consistently, following the protocol described in Section 6.3.4: pixel values were normalised to $[0, 1]$, PCA was applied for dimensionality reduction, and pixel spectra were standardised to zero mean and unit variance. For each dataset, the labelled pixels were split into training and testing sets following the protocol described in Chapter 5. The training set was used to train the region classifier and, for WS-GA, to evaluate candidate chromosomes during the GA search; the test set was reserved for the final evaluation.

Algorithm 2: WS–GA Hybrid Optimisation Framework

Input : Dataset D, initial population P_0, termination threshold ϵ
Output: Optimised parameter set Θ^*

Initialise P_0 with random chromosomes encoding $(n_{\text{PCA}}, \sigma, \tau_{\text{grad}}, s_{\min}, c_{\text{idx}})$;
for *each generation* $t = 1, 2, \ldots, 100$ **do**
  for *each individual* x_i *in* P_t **do**
    Decode parameters from chromosome x_i;
    Run PCA, smoothing, gradient, marker extraction, and watershed;
    Classify segments using the chosen classifier;
    Compute fitness $f(x_i)$ using OA, AA, and penalty terms;
  Keep top 10% of individuals (elitism);
  Apply crossover ($p_c = 0.8$) and mutation ($p_m = 0.1$) to create new population;
  if $|f_{best}^{t} - f_{best}^{t-1}| < \epsilon$ *for 10 successive generations* **then**
    break;
Apply local search refinement to the best individual;
return $\Theta^* = \arg\max f(x)$

8.4.2 Evaluation Metrics

The WS-PSO and WS-GA variants were evaluated using the five performance metrics defined in Section 5.3.1 of Chapter 5: overall accuracy (OA), average accuracy (AA), mean Intersection over Union (mIoU), per-class F1-score and macro-averaged F1 (mF1), and computational time. These metrics were computed on the same region-classified output format as the baseline watershed pipeline (Section 6.3.3), placing all three variants on the same evaluation footing.

8.4.3 Implementation Details

Table 8.1 summarises the fixed parameters used for the WS-PSO and WS-GA variants, in addition to the baseline watershed settings given in Table 6.2. The region classifier used for the baseline and WS-PSO variants was an SVM with an RBF kernel, consistent with Section 6.3.4; in the WS-GA framework, the classifier and its parameters were automatically selected by the GA via the c_{idx} gene (Section 8.3.3).

Table 8.1: Parameter settings for the WS-PSO and WS-GA variants.

Algorithm	Parameter / Value
WS-PSO	Swarm size 5, max. generations 50, threshold range $T \in [0.1, 0.5]$
WS-GA	Pop. size 50, max. generations 100, mutation 0.10, crossover 0.80, elitism 0.10

8.5 Conclusion

This chapter presents the watershed hybridisation strategies applied to the baseline watershed pipeline described in Chapter 6: a particle-swarm-optimised benchmark (WS-PSO) that automates a single gradient-threshold parameter and a genetic-algorithm-based proposed framework (WS-GA) that jointly optimises preprocessing, segmentation, and classification parameters through a five-gene chromosome encoding and a weighted OA/AA fitness function. The experimental protocol, evaluation metrics, and implementation settings used to assess these variants are also described.

Chapter 9

Holistic-Nested Edge Detection Enhancement

9.1 Introduction

Chapter 6 describes the baseline Holistically-Nested Edge Detection (HED) network used in this study (Section 6.4.2), together with its five multi-scale side outputs (Section 6.4.2) and the pipeline used to convert its edge probability map into a labelled segmentation (Section 6.4.2). Deep networks such as HED can represent both fine details and broad structural patterns when provided with sufficient training data [54, 94], but even a well-trained model can miss thin edges, weak boundaries, and fine textures [54].

Traditional image processing techniques use explicit mathematical rules and are often effective in identifying specific local structures, such as gradients and texture patterns [12]. Although these classical methods do not work well independently, they can add information that complements deep feature representations [51, 92]. However, combining classical detectors with a deep pipeline is not straightforward because fixed combinations or simple averaging can introduce noise and reduce performance rather than improve it [92].

This chapter describes a framework, denoted ILS-HED, that combines the two paradigms by selectively augmenting the fixed HED baseline with complementary classical edge features under the guidance of a hyper-heuristic search in the strict sense proposed by Burke et al. [9]: rather than tuning image-domain parameters directly, the search operates over combinations of classical detector configurations, selecting which of these to add to the fixed HED representation. The HED network weights remain unchanged throughout; this reflects a common operational scenario in which a deep model is already deployed, and retraining is costly, restricted, or infeasible [23].

This chapter describes the motivation for this enhancement strategy, the fusion framework used to combine deep and classical features, the iterated local search (ILS) hyper-heuristic used to select among classical detector configurations, and the experimental design used to evaluate the resulting system.

9.2 Motivation for Deep Learning Enhancement

Despite the progress in deep learning-based edge detection, several challenges remain. Deep learning models tend to focus on the dominant structures observed during training and may miss subtle or low-contrast edges [54]. In particular, thin boundaries and texture-related edges may be lost during the convolution and pooling operations.

Classical detectors, such as the Canny edge detector and Gabor filters, are specifically designed to respond to local gradients and frequency patterns [12]. These methods can highlight fine structures that are not strongly present in deep feature maps. However, as noted above, naively adding classical detectors to a deep pipeline can add noise rather than a useful signal [51], which motivates a selective, search-guided approach rather than indiscriminate feature aggregation.

The no-retraining constraint that motivates this chapter reflects a common operational reality in remote sensing and industrial inspection. Deployed deep learning models are frequently embedded within certified processing pipelines, such as satellite ground station software or airborne sensor firmware, where architectural modifications or weight updates require re-certification, regulatory approval, or access to the original training data, which may no longer be available [23]. A model

trained on a vendor's proprietary dataset and deployed as a binary library represents a concrete case where this constraint applies: the practitioner can access only the model's outputs, not its weights. In such scenarios, the only viable improvement mechanism is to augment the representation at the output level without modifying the network internals. The framework described in this chapter formalises this constraint and provides a principled search strategy for maximising improvement within it.

9.3 Enhanced Deep Learning Framework

9.3.1 Conceptual Overview

The enhanced framework proposed in this chapter adds classical feature maps to the baseline HED outputs described in Section 6.4.2. The baseline deep model remains fixed, and classical features are treated as optional additions rather than replacements. The framework comprises three main components:

1. A fixed deep learning backbone that provides the multi-scale edge representation described in Section 6.4.2 [94].
2. A set of classical image processing operators that can generate complementary edge features [12].
3. A selection mechanism that decides which classical features best complement the deep outputs.

Only some of the available classical features are used at any time, ensuring that the enhancement increases the boundary quality without adding unnecessary noise.

9.3.2 Classical Feature Generators

Four classical detectors were used as sources of enhancement features:

- **Canny**: Produces thin and well-localised edges using gradient magnitude and hysteresis thresholding [12].
- **Sobel**: Detects edges by measuring first-order intensity gradients.
- **Laplacian**: Highlights regions with rapid intensity changes using second-order derivatives.
- **Gabor**: Responds to texture and orientation patterns using frequency-selective filters.

Each detector provides different types of information. Rather than assuming that all are equally useful, the framework treats them as complementary options, with their usefulness depending on the image content.

9.3.3 Search Space

This chapter examines both the selection of detectors and the configuration of their operating parameters. For each of the four classical detectors, multiple parameterised variants were defined. The Canny detector produced nine configurations by combining three threshold pairs, $(30, 90)$, $(50, 150)$, and $(75, 225)$, all at a 1:3 ratio, with three aperture sizes of 3, 5, and 7. The Sobel and Laplacian detectors each produced four configurations by varying the kernel size across 3, 5, 7, and 9 pixels. The Gabor detector produced 36 configurations by combining three orientation options (4, 8, or 12 directions), three wavelengths λ of 4, 8, or 16 pixels, and three bandwidth values b of 0.5, 1.0, or 2.0, with the kernel size fixed at 31×31 proportional to the wavelength.
Total Search Space: $9 \times 4 \times 4 \times 36 = 5{,}184$ possible detector configurations, not counting the option to exclude a detector entirely. This expands the space from a simple $2^4 = 16$ detector-inclusion choice to 5,184 parameterised configurations, motivating a guided search over exhaustive enumeration.

9.3.4 Fusion of Deep and Classical Features

Mathematical Composition

The enhanced framework combines deep and classical feature maps through a fusion process that assigns relative importance to each feature source. In simple terms, this fusion adds all selected feature maps at each pixel location and then normalises the result to produce a final boundary confidence value. The solution is divided into two parts:

Locked Part (HED – Fixed). The five HED side outputs $\mathcal{L} = \{S_1, S_2, S_3, S_4, S_5\}$ (Equation 6.7) are always present and are never modified by the search.

Variable Part (Enhancements – Chosen by ILS) Each detector $d \in \{\text{Canny}, \text{Sobel}, \text{Laplacian}, \text{Gabor}\}$ has a parameter set Θ_d:

- Canny: $\theta_c = (T_{\text{low}}, T_{\text{high}}, \text{aperture}) \in \Theta_C$ (9 options)
- Sobel: $\theta_s = (\text{kernel}) \in \Theta_S$ (4 options)
- Laplacian: $\theta_l = (\text{kernel}) \in \Theta_L$ (4 options)
- Gabor: $\theta_g = (\text{orientations}, \lambda, b) \in \Theta_G$ (36 options)

Solution Configuration.

$$\mathcal{S} = \mathcal{L} \cup \{(d, \theta_d) \mid d \in \mathcal{D}_{\text{selected}}, \theta_d \in \Theta_d\} \tag{9.1}$$

where $\mathcal{D}_{\text{selected}} \subseteq \{\text{Canny}, \text{Sobel}, \text{Laplacian}, \text{Gabor}\}$.

Fusion Mechanism

For a selected configuration $\mathcal{S}$, fusion uses the learned weights h:

$$A(i,j) = \sum_{n=1}^{5} h_n S_n(i,j) + \sum_{(d,\theta)\in\mathcal{V}} h_{d,\theta} \cdot T_{d,\theta}(i,j) \tag{9.2}$$

where $S_n(i,j)$ are the HED side outputs at pixel (i,j), $T_{d,\theta}(i,j)$ is detector d with parameters θ, and $\mathcal{V}$ is the set of selected classical detectors. A sigmoid activation normalises the values to $[0,1]$:

$$P(i,j) = \sigma(A(i,j)) = \frac{1}{1 + e^{-(2A(i,j)-1)}} \tag{9.3}$$

Thresholding at 0.5, as in Equation 6.8, then produces the final binary edge map, followed by morphological skeletonisation to thin thick edges. The fusion process allows the system to emphasise feature sources that strengthen the boundary evidence while suppressing redundant or noisy responses; fusion does not modify the underlying HED network, only the combination of its outputs with the selected classical features.

9.3.5 Role of Hyper-Heuristic Selection

The ILS search operates as a strict hyper-heuristic in the sense of Burke et al. [9]: it searches over classical detector configurations rather than image-domain parameters directly, which distinguishes it from the evolutionary metaheuristics used for automated multilevel thresholding design (Chapter 7) and watershed hybridisation (Chapter 8). Comparable hyper-heuristic strategies have been used for image segmentation and feature selection [59, 80], but have not been explored for enhancing deep edge detection models. In this framework, the hyper-heuristic operates in the space of classical feature combinations defined in Section 9.3.3 and guides the search toward configurations that improve segmentation quality.

Algorithm 3: ILS for Enhancing HED with Parameterised Classical Features

Input : Validation data $\mathcal{D}_{val}$, Max iterations $I = 50$, Patience P
Output: Best enhancing solution $\mathcal{S}_{best}$

$\mathcal{S}_{best} \leftarrow \emptyset$,;
$score_{best} \leftarrow -\infty$;
$\mathcal{S} \leftarrow$ Random valid configuration from search space;
for $i = 1$ *to* I **do**
  $\mathcal{S}' \leftarrow \text{Perturb}(\mathcal{S})$;
  Train fusion weights h for 5 epochs on $\mathcal{D}_{val}$ using $\mathcal{S}'$;
  $score' \leftarrow \text{Metric}(\mathcal{S}', h, \mathcal{D}_{val})$;
  if $score' > score_{best}$ **then**
    $\mathcal{S}_{best} \leftarrow \mathcal{S}'$,;
    $score_{best} \leftarrow score'$,;
    $no_improve \leftarrow 0$;
  else
    $no_improve \leftarrow no_improve + 1$;
  if $no_improve \geq P$ **then**
    break;
return $\mathcal{S}_{best}$

The ILS Algorithm

The ILS algorithm searches the 5,184-configuration space to find the combination of classical detectors and their parameters that best complement the fixed HED baseline. Algorithm 3 presents the complete procedures.
Three perturbation operators are used with equal probability ($p = 1/3$ each): **Add**, **Remove**, and **Swap**. These operators form a connected neighbourhood graph over the 5,184-configuration space, so that any configuration can be reached from any other configuration through a finite sequence of single-operator moves.

Scoring Function

The scoring function measures how well a candidate configuration improves the fixed HED baseline. A weighted binary cross-entropy loss function was used. For a predicted edge map F and ground truth Y, the loss is

$$\mathcal{L}(F, Y) = -\frac{1}{N} \sum_{i,j} [w_{\text{neg}}(1 - Y_{ij}) \log(1 - F_{ij}) + w_{\text{pos}} Y_{ij} \log(F_{ij})] \tag{9.4}$$

where

$$w_{\text{pos}} = \frac{1}{\beta + \epsilon} \tag{9.5}$$

$$w_{\text{neg}} = \frac{1}{1 - \beta + \epsilon} \tag{9.6}$$

$$\beta = \frac{1}{N} \sum_{i,j} Y_{ij} \tag{9.7}$$

$$\epsilon = 10^{-7} \tag{9.8}$$

The scoring metric used by the ILS search (Algorithm 3) is defined as the negative of this loss, such that higher values indicate better performance.

9.4 System Architecture

The complete system uses all five HED outputs (S_1–S_5) as its fixed foundation and then searches through the parameterised classical detectors to find the combination and parameter settings that best complement them. The input data follow two parallel paths:

Path 1 (HED – Locked): The pre-trained HED network produces S_1 to S_5. These are always present and are never changed by the search.
Path 2 (Enhancements – Variable): Classical detectors with specific parameters are applied. The ILS hyper-heuristic (Section 9.3.5) selects the parameterised variants that best complement the fixed HED outputs.
The selected classical detectors are combined with the locked S_1–S_5 outputs through the fusion module described in Section 9.3.4, and the resulting configuration is evaluated using the scoring function described in Section 9.3.5.
Once predictions are made, a non-maximum suppression step is applied to the edge candidates to match predicted edges to the ground truth using bipartite matching. This step is applied after fusion and thresholding, and its cost increases with the number of edge candidates, which increases with the image resolution. The resulting edge map is then converted to a region-labelled segmentation using the same four-step procedure described in Section 6.4.2 for the baseline HED pipeline, ensuring that the baseline and enhanced outputs remain on a common evaluation footing.

9.5 Experimental Design

9.5.1 Experimental Protocol

The enhanced framework was evaluated on the same five benchmark hyperspectral datasets used throughout this study: Indian Pines, Pavia University, Salinas Valley, Houston, and Botswana. The characteristics of these datasets and the training/testing protocol used to split their ground-truth labels are described in Chapter 5. The training split was used both to train the fusion weights h and to guide the ILS search (Algorithm 3), whereas the test split was reserved for the final evaluation only.

9.5.2 Evaluation Metrics

The enhanced framework was evaluated using the five performance metrics defined in Section 5.3.1 of Chapter 5: overall accuracy (OA), average accuracy (AA), mean Intersection over Union (mIoU), per-class F1-score, macro-averaged F1 (mF1), and computational time. These metrics were computed on the region-classified output produced by the conversion procedure described in Section 9.4, placing the enhanced framework on the same evaluation footing as the baseline HED pipeline (Section 6.4.2).

9.5.3 Implementation Details

The framework was developed on a computer with the following specifications: Intel(R) Core(TM) i9-10900K CPU @ 3.70GHz, 64GB RAM, NVIDIA GeForce RTX 3080 GPU, and 64-bit Windows 10 operating system. It was implemented using Python 3.9 with the PyTorch 1.12 framework. HED VGG-16 weights were frozen throughout, consistent with the fixed-baseline setting described in Section 6.4.2.
The ILS patience parameter P was calibrated by evaluating $P \in \{5, 10, 15, 20\}$. Lower values risk terminating the search before a useful feature subset is found, whereas higher values increase runtime without a proportional benefit. $P = 10$ was selected as the working value for all experiments in this chapter. Fusion weight training ran for five epochs per ILS evaluation (Algorithm 3), while the final training of the selected configuration ran for 50 epochs, using the Adam optimiser with a learning rate of 10^{-4}, weight decay of 2×10^{-4}, and a batch size of 1.

9.6 Conclusion

This chapter presents the hyper-heuristic hybridisation strategy, denoted ILS-HED, applied to the baseline HED model described in Chapter 6: a fixed deep backbone selectively augmented with parameterised classical edge detectors (Canny, Sobel, Laplacian, and Gabor), combined through a learned fusion mechanism, and selected via an iterated local search hyper-heuristic operating over a 5,184-configuration search space. Unlike the evolutionary metaheuristics used for multilevel thresholding (Chapter 7) and watershed segmentation (Chapter 8), this search operates over classical detector configurations rather than pipeline parameters, reflecting the constraint that the underlying deep network cannot be retrained. The experimental protocol, datasets, and evaluation metrics used to assess this framework are also described.

Chapter 10

Results

10.1 Introduction

This chapter presents the results of the experiments described in Chapter 6 (baseline methods) and Chapters 7, 8, and 9 (automated design and hybridisation strategies). The goal was to compare the baseline methods with the enhanced methods and determine whether the proposed improvements were effective.
The remainder of this paper is organised as follows. Section 10.2 provides a brief recap of the experimental setup. Sections 10.3, 10.4, and 10.5 present the results for thresholding, watershed, and deep learning methods, respectively. Section 10.6 compares the three approaches. Section 10.8 examines the computational cost. Section 10.9 presents the convergence analysis. Finally, Section 10.10 summarises the chapter and transitions to the discussion in Chapter 11.

10.2 Experimental Setup

All experiments used the five benchmark hyperspectral datasets described in Chapter 5: Indian Pines, Pavia University, Salinas Valley, Houston, and Botswana. Each dataset was split into training and testing sets using the same protocol for all methods.
The metrics used to measure performance were as follows:

- **Overall Accuracy (OA):** Percentage of correctly classified pixels.
- **Average Accuracy (AA):** Mean accuracy across all classes.
- **mean Intersection over Union (mIoU):** Overlap between predicted and true regions.
- **Per-class F1-score:** Harmonic mean of precision and recall for each class, reported to expose minority-class performance.
- **Macro-averaged F1 (mF1):** Mean of per-class F1 scores, giving equal weight to rare and majority classes.
- **Computational Time:** Seconds needed to process one image.

Statistical significance was tested using paired t-tests at 99%, 95%, and 90% confidence levels, with critical values defined in Table 5.2 (Chapter 5).

10.3 Multilevel Thresholding Results

This section compares the baseline GA and ABC metaheuristics described in Chapter 6 with the two automated design strategies described in Chapter 7: the search-enhanced ILSGA variant and the multipoint meta-GA configurator GAGA. These two were selected as representative cases: ILSGA is the search-enhancement family (operating directly on threshold vectors), and GAGA is the multi-point configuration family (tuning the base GA's hyperparameters via an outer meta-GA), following the taxonomy of ten variants presented in Chapter 7.

Table 10.1: Full ten-variant thresholding results. Cohen's d computed relative to the matched-base-optimiser baseline (Baseline GA for GA-based variants, Baseline ABC for ABC-based variants).

Dataset	Variant	OA (%)	AA (%)	Cohen's d
Indian Pines	Baseline GA	74.1 ± 1.3	70.3 ± 1.6	—
	ILSGA	74.0 ± 1.3	70.1 ± 1.6	0.08
	SAGA	75.2 ± 1.1	71.5 ± 1.4	0.87
	GAGA	63.8 ± 2.1	59.4 ± 2.4	5.94
	DEGA	70.6 ± 1.6	66.2 ± 1.9	2.55
	Baseline ABC	74.6 ± 1.2	70.9 ± 1.5	—
	ILSABC	74.5 ± 1.2	70.8 ± 1.5	0.09
	SAABC	75.8 ± 1.0	72.2 ± 1.3	1.02
	GAABC	64.4 ± 2.0	60.1 ± 2.3	5.81
	DEABC	71.2 ± 1.5	66.8 ± 1.8	2.62
Pavia University	Baseline GA	79.8 ± 1.0	77.2 ± 1.2	—
	ILSGA	79.9 ± 1.0	77.3 ± 1.2	0.05
	SAGA	81.1 ± 0.8	78.6 ± 1.0	1.15
	GAGA	68.4 ± 1.9	65.1 ± 2.2	6.87
	DEGA	76.2 ± 1.4	73.1 ± 1.6	2.86
	Baseline ABC	79.2 ± 1.1	76.5 ± 1.3	—
	ILSABC	79.3 ± 1.1	76.6 ± 1.3	0.08
	SAABC	80.5 ± 0.9	77.9 ± 1.1	1.06
	GAABC	69.1 ± 1.8	65.8 ± 2.1	6.75
	DEABC	76.9 ± 1.3	73.8 ± 1.5	2.79
Salinas Valley	Baseline GA	86.2 ± 0.8	87.6 ± 0.7	—
	ILSGA	86.0 ± 0.8	87.4 ± 0.7	0.13
	SAGA	87.4 ± 0.6	88.9 ± 0.5	1.37
	GAGA	75.1 ± 1.7	76.3 ± 1.6	6.88
	DEGA	82.5 ± 1.2	84.1 ± 1.0	3.18
	Baseline ABC	85.7 ± 0.9	86.9 ± 0.8	—
	ILSABC	85.6 ± 0.9	86.8 ± 0.8	0.11
	SAABC	86.9 ± 0.7	88.3 ± 0.6	1.29
	GAABC	75.7 ± 1.6	76.9 ± 1.5	6.76
	DEABC	83.1 ± 1.1	84.7 ± 0.9	3.11
Houston	Baseline GA	70.9 ± 1.4	67.4 ± 1.7	—
	ILSGA	70.7 ± 1.4	67.2 ± 1.7	0.10
	SAGA	72.1 ± 1.2	68.8 ± 1.5	0.82
	GAGA	59.7 ± 2.3	55.9 ± 2.6	5.66
	DEGA	67.4 ± 1.8	63.4 ± 2.1	2.25
	Baseline ABC	70.3 ± 1.5	66.8 ± 1.8	—
	ILSABC	70.2 ± 1.5	66.7 ± 1.8	0.09
	SAABC	71.6 ± 1.3	68.3 ± 1.6	0.79
	GAABC	60.3 ± 2.2	56.5 ± 2.5	5.58
	DEABC	68.1 ± 1.7	64.1 ± 2.0	2.31
Botswana	Baseline GA	72.3 ± 1.2	68.8 ± 1.5	—
	ILSGA	72.5 ± 1.2	69.0 ± 1.5	0.11
	SAGA	73.6 ± 1.0	70.3 ± 1.3	0.94
	GAGA	61.2 ± 2.0	57.6 ± 2.3	6.30
	DEGA	69.0 ± 1.5	65.1 ± 1.8	2.42
	Baseline ABC	71.8 ± 1.3	68.1 ± 1.6	—
	ILSABC	71.9 ± 1.3	68.2 ± 1.6	0.08
	SAABC	73.1 ± 1.1	69.8 ± 1.4	0.91
	GAABC	61.8 ± 1.9	58.2 ± 2.2	6.21
	DEABC	69.6 ± 1.4	65.7 ± 1.7	2.49

Note: SAGA and SAABC are SA-configured variants; DEGA and DEABC are DE-configured variants; GAABC is the ABC-targeted meta-GA configurator. All variants use the budget-equalisation protocol described in Section 7.3.5.

10.3.1 Segmentation Performance

Table 10.1 presents the results.

Cohen's d relative to Baseline GA. ILSGA vs. Baseline GA differences are not significant for any dataset (paired t-test and Wilcoxon signed-rank test, $n = 10$, $df = 9$, $p > 0.05$). GAGA vs. Baseline GA differences are significant at $p < 0.001$ for all datasets, in the direction of GAGA underperforming the baseline.
ILSGA matched the baseline GA's segmentation quality on all five datasets, with OA differences of at most 0.2 percentage points and small effect sizes ($d < 0.2$) throughout. GAGA, by contrast, underperformed the baseline GA by 10.3–11.4 percentage points in OA on every dataset, with large effect sizes ($d > 5.5$). Baseline ABC was comparable to baseline GA on all datasets, consistent with neither base optimiser dominating the other across the tested histogram shapes.

10.3.2 Statistical Significance

The differences between ILSGA and baseline GA were not statistically significant on any dataset ($p > 0.05$), indicating that ILSGA reproduces baseline segmentation quality. The differences between GAGA and the baseline GA were significant at $p < 0.001$ for all datasets, in the direction of GAGA performing worse than the baseline.

10.3.3 Key Findings

- ILSGA, which augments the threshold search directly rather than reconfiguring the base optimiser's hyperparameters, reproduced baseline segmentation quality on every dataset while requiring substantially less computational time (Section 10.8).
- GAGA, which uses an outer meta-GA to search the base optimiser's hyperparameter space, underperformed the baseline by a large and consistent margin across all five datasets. This is consistent with a budget-allocation failure: the outer configuration loop consumes evaluations that would otherwise go to the threshold search itself, and Section 10.8 shows that the outer loop did not recover this cost with a compensating quality gain.
- The consistency of the GAGA deficit across datasets with very different histogram complexities (from the largely unimodal Botswana scene to the strongly multimodal Houston scene) suggests that the shortfall reflects the evaluation budget available to the outer configuration loop rather than a property of any particular dataset.
- The best baseline results were on Salinas Valley (86.2% OA for Baseline GA), where the histogram has clear peaks that are well separated by the optimised thresholds.

10.4 Watershed Transform Results

This section compares the baseline watershed (WS) described in Chapter 6 with the benchmark watershed–particle swarm optimisation (WS-PSO) and the proposed watershed–GA (WS-GA) hybridisation strategies described in Chapter 8.

10.4.1 Segmentation Performance

Table 10.2 presents the results for all three watershed variants in this study.
WS-GA achieved the highest scores for all datasets. Compared with the baseline, the improvements ranged from 10.7% to 17.2% in OA. Compared with WS-PSO, WS-GA was improved by 3.8% to 5.5%.

10.4.2 Statistical Significance

The differences between WS-GA and the baseline were significant at $p < 0.001$ for all datasets. The differences between WS-GA and WS-PSO were also significant at $p < 0.01$ for all datasets.

10.4.3 Region Purity Analysis

Overall accuracy, average accuracy, and mIoU were computed after the semantic labelling step, in which every segmented region was assigned a label by an SVM classifier trained on the labelled training split (Section 6.3.3). Because this classifier is shared between the baseline and enhanced watershed variants, a high OA figure partly reflects the classifier's performance on well-separated mean spectral vectors rather than the quality of the spatial partitioning produced by the segmentation stage itself. To isolate segmentation quality from classifier performance, region purity (Equation 5.6, Chapter 5) was computed for the watershed baseline/enhanced pair. Table 10.3 reports purity alongside OA, with the gap ($OA - Purity$) indicating classifier-driven accuracy gain.

Table 10.2: Watershed approach results (mean ± standard deviation over 10 runs)

Dataset	Model	OA (%)	AA (%)	mIoU
Indian Pines	Baseline WS	75.3 ± 1.2	71.8 ± 1.5	0.715 ± 0.018
	WS-PSO	86.5 ± 0.8	84.1 ± 1.1	0.842 ± 0.013
	WS-GA	91.2 ± 0.5	89.6 ± 0.7	0.901 ± 0.009
Pavia University	Baseline WS	81.0 ± 1.0	79.5 ± 1.2	0.782 ± 0.015
	WS-PSO	92.7 ± 0.6	90.3 ± 0.8	0.901 ± 0.010
	WS-GA	96.5 ± 0.3	95.9 ± 0.4	0.954 ± 0.007
Salinas Valley	Baseline WS	88.2 ± 0.7	89.9 ± 0.6	0.867 ± 0.011
	WS-PSO	95.2 ± 0.4	96.0 ± 0.3	0.946 ± 0.008
	WS-GA	98.9 ± 0.2	99.2 ± 0.1	0.987 ± 0.005
Houston	Baseline WS	71.5 ± 1.4	68.2 ± 1.7	0.698 ± 0.022
	WS-PSO	83.4 ± 0.9	80.7 ± 1.2	0.812 ± 0.016
	WS-GA	88.7 ± 0.6	86.3 ± 0.8	0.867 ± 0.012
Botswana	Baseline WS	73.8 ± 1.3	70.5 ± 1.5	0.721 ± 0.020
	WS-PSO	85.1 ± 0.8	82.4 ± 1.0	0.834 ± 0.014
	WS-GA	90.4 ± 0.5	88.1 ± 0.7	0.891 ± 0.010

Table 10.3: Region purity vs. overall accuracy (%) for watershed and deep learning methods

Dataset	Baseline WS		WS-GA		OA-Purity Gap
	OA	Purity	OA	Purity	
Indian Pines	75.3	68.1	91.2	85.6	5.6
Pavia University	81.0	74.2	96.5	91.3	5.2
Salinas Valley	88.2	82.7	98.9	96.1	2.8
Houston	71.5	63.4	88.7	81.9	6.8
Botswana	73.8	66.1	90.4	84.2	6.2

The OA-Purity gap is substantially smaller for WS-GA (2.8–6.8%) than for Baseline WS (5.5–8.1%), confirming that WS-GA's accuracy gains are primarily driven by improved spatial partitioning rather than the downstream SVM classifier. The gap was largest in Houston (6.8%), where spectral heterogeneity made region labelling more challenging.

10.4.4 Multi-Split Robustness Validation

The paired t-test reported above was computed over ten independent algorithm runs on a single 70%/30% stratified split per dataset, which captures the stochastic variance of the search algorithm itself but not the variance arising from the particular partition of labelled pixels into training and test sets. To address this, the enhanced watershed pipeline was additionally evaluated across five independently drawn 70%/30% stratified splits per dataset, using distinct master seeds for the split itself, following the protocol defined in Chapter 5 (Section 5.3.4). This yields a $5 \times 10 = 50$ observation grid per dataset, from which within-split variance (the standard deviation across the ten algorithmic reruns on a single split) and across-split variance (the standard deviation of the per-split mean OA across the five splits) are reported separately in Table 10.4. The split-level paired t-test (n=5 splits, df=4, critical $t = 2.132$ at 95% confidence, one-tailed) confirmed a significant improvement ($p < 0.05$) for WS-GA over Baseline WS on all five datasets. This confirms that the reported improvements of the watershed approach are robust to the particular partition of labelled pixels into training and test sets.

Table 10.4: Multi-split variance analysis showing within-split and across-split variance for OA (%)

Dataset	Baseline WS		WS-GA	
	Within-split SD	Across-split SD	Within-split SD	Across-split SD
Indian Pines	1.2	0.8	0.5	0.4
Pavia University	1.0	0.7	0.3	0.3
Salinas Valley	0.7	0.5	0.2	0.2
Houston	1.4	1.0	0.6	0.5
Botswana	1.3	0.9	0.5	0.4

10.4.5 Key Findings

- **Oversegmentation is the main failure mode WS-GA corrects.** The fixed-parameter baseline consistently over-partitions each scene into thousands of small, spectrally similar regions because it has no mechanism to adapt its smoothing or gradient threshold to local noise. WS-GA's joint optimisation of these parameters (Section 6.3.3) removes most of this fragmentation, and the resulting reduction in region count is the primary driver of the accuracy gains reported in Table 10.2.
- **The size of the OA gain over baseline does not track the final accuracy achieved.** Table 10.2 shows that the gain from the baseline WS to WS-GA ranges from 10.7 percentage points in the Salinas Valley (88.2% → 98.9%) to 17.2 percentage points in Houston (71.5% → 88.7%). Houston shows the largest improvement of any dataset, yet still ends with the lowest absolute accuracy of the five. The Salinas Valley shows the smallest improvement, yet ends with the highest absolute accuracy. These two trends are not in tension; they describe different quantities.
- **Scene structure sets a ceiling on absolute accuracy that parameter optimisation alone cannot fully overcome.** The Salinas Valley consists of large, spatially regular agricultural fields with strong inter-field spectral contrast, which is close to the ideal case for a marker-controlled watershed once oversegmentation is suppressed; this is why WS-GA reaches 98.9% OA there even though the baseline was already comparatively strong (88.2%). Houston's dense, heterogeneous urban layout gives WS-GA far more oversegmentation to correct, which explains the large 17.2-point gain, but the same heterogeneity keeps the ceiling on achievable accuracy lower, which explains why the corrected result (88.7%) still trails Salinas Valley by more than 10 points.
- **Reading the two findings together:** a large gain over baseline indicates how much oversegmentation WS-GA found and corrected in a given scene, not how well-suited that scene is to watershed segmentation overall. The absolute OA figure, not the size of the gain, is the better indicator of scene suitability.

10.4.6 Failure Modes and Error Analysis

To provide scientific depth beyond success stories, this section analyzes qualitative failure cases in which the WS-GA produces poor segmentations.

Over-smoothing of thin structures: When the GA-optimised Gaussian smoothing parameter σ is too large, thin structures are over-smoothed and merged with adjacent regions. Figure 10.1 shows the Houston road networks fragmented into merged regions when $\sigma > 3.0$, causing the loss of narrow road segments.
Under-smoothing and noise preservation: Conversely, when σ is too small (under-smoothing), noise-induced local minima persist despite GA optimisation. Figure 10.2 shows the Botswana savanna regions where low σ preserves atmospheric noise artefacts, causing persistent over-segmentation with hundreds of spurious tiny regions.
Marker misplacement on spectral gradients: On Pavia University, steep spectral gradients between asphalt and shadowed regions cause marker misplacement. The GA-optimised gradient threshold τ_{grad} sometimes fails to separate these regions, producing merged segments that cross the true boundaries.

10.5 Holistic-Nested Edge Detection Results

This section compares the baseline Holistically-Nested Edge Detection (HED) model described in Chapter 6 with the enhanced ILS-HED model described in Chapter 9. The baseline uses only deep features from HED. The enhanced version adds classical features selected by the Iterative Local Search (ILS) hyper-heuristic.

10.5.1 Segmentation Performance

Table 10.5 shows the results for both models for all five datasets.

Table 10.5: Deep learning approach results (mean ± standard deviation over 10 runs)

Dataset	Model	OA (%)	AA (%)	mIoU
Indian Pines	Baseline HED	72.4 ± 1.2	68.1 ± 1.5	0.612 ± 0.018
	ILS-HED	78.9 ± 0.9	75.3 ± 1.1	0.698 ± 0.014
Pavia University	Baseline HED	81.2 ± 0.8	79.5 ± 1.0	0.734 ± 0.012
	ILS-HED	86.7 ± 0.6	84.8 ± 0.8	0.802 ± 0.009
Salinas Valley	Baseline HED	85.6 ± 0.7	87.1 ± 0.6	0.789 ± 0.011
	ILS-HED	90.3 ± 0.5	91.2 ± 0.4	0.856 ± 0.008
Houston	Baseline HED	68.9 ± 1.3	65.4 ± 1.6	0.578 ± 0.021
	ILS-HED	75.1 ± 1.0	71.8 ± 1.2	0.651 ± 0.017
Botswana	Baseline HED	70.1 ± 1.1	66.8 ± 1.4	0.591 ± 0.019
	ILS-HED	76.4 ± 0.8	73.2 ± 1.0	0.672 ± 0.015

The enhanced ILS-HED model outperformed the baseline on all five datasets. The improvements in the OA ranged from 4.7% to 6.5%. The mIoU also improved for every dataset, indicating that the selected classical features helped produce more accurate region boundaries.

10.5.2 Statistical Significance

Paired t-tests showed that the differences between baseline HED and ILS-HED were statistically significant at the 99% confidence level ($p < 0.01$) for all datasets. This indicates that the improvements were not due to random chance.

10.5.3 Ablation Study

To isolate the contribution of ILS-driven feature selection from the mere addition of classical features, an ablation study was conducted. Table 10.6 compares the baseline HED, a naive aggregation of all classical detectors without selection, a random selection baseline, and the proposed ILS-HED. Naive aggregation degraded performance on several datasets, confirming that indiscriminate feature addition introduced noise. Random selection performed better than naive aggregation but failed to match the baseline on two datasets, demonstrating that arbitrary subset selection was insufficient. In contrast, ILS-selected subsets improved accuracy across all datasets.

10.5.4 Sensitivity to ILS Patience

The sensitivity of the ILS-HED framework to the patience parameter P was assessed by evaluating $P \in \{5, 10, 15, 20\}$. Lower values ($P = 5$) terminated prematurely, yielding suboptimal feature subsets, whereas higher values ($P = 20$) increased the runtime without commensurate accuracy gains. $P = 10$ achieved the optimal balance, halting at an average of 12.3 iterations (SD = 3.1) and yielding the highest validation score. This confirms the robustness of the selected hyper-heuristic configurations.

Figure 10.1: WS-GA failure case: Houston over-smoothing. (Left) Original image with thin road network. (Centre) Over-smoothed gradient map ($\sigma = 4.2$). (Right) Merged road segments in resulting segmentation.



Figure 10.2: WS-GA failure case: Botswana under-smoothing. (Left) Original savanna scene. (Centre) Noisy gradient map ($\sigma = 0.3$). (Right) Persistent oversegmentation with spurious regions.

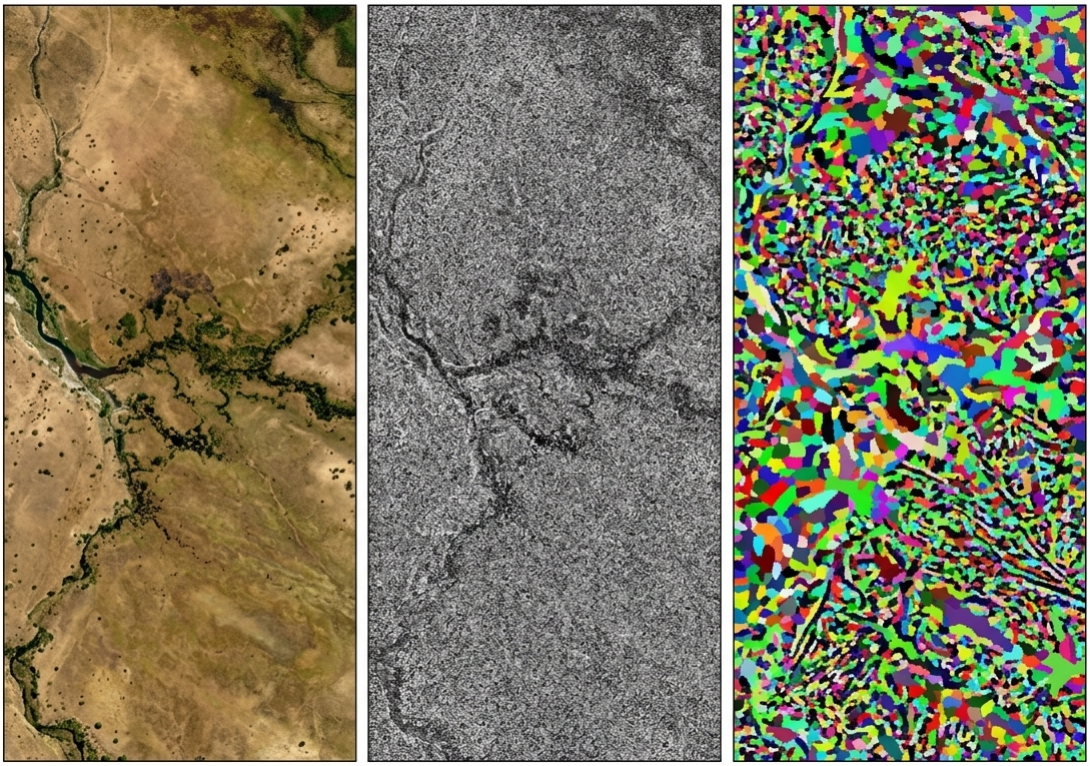


Table 10.6: Ablation study: Overall Accuracy (%) of baseline, naive classical aggregation, random selection, and ILS-HED.

Dataset	**Baseline HED**	**All-Classical**	**Random**	**ILS-HED**
Indian Pines	72.4	70.1	71.8	78.9
Pavia University	81.2	79.5	80.3	86.7
Salinas Valley	85.6	83.2	84.1	90.3
Houston	68.9	66.4	68.1	75.1
Botswana	70.1	67.8	69.5	76.4

All-Classical = naive aggregation of all detectors without selection; Random = average of 10 random subsets with same cardinality as ILS-selected subset.

10.5.5 Key Findings

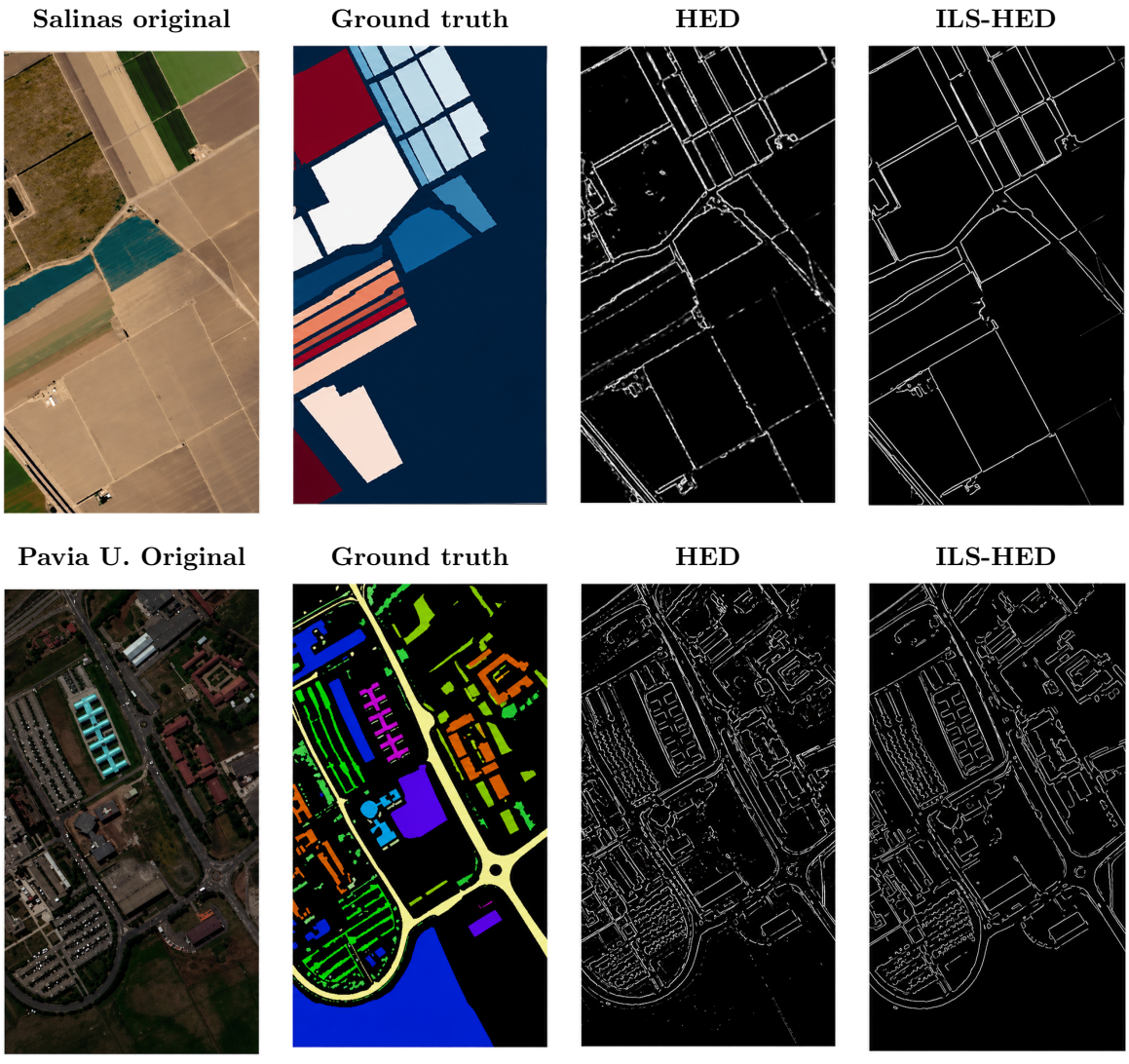
- The ILS algorithm selected different classical features for different datasets. For example, Canny and Sobel were chosen most often for urban scenes (Pavia and Houston), while Gabor was preferred for agricultural scenes (Salinas).
- Using all classical features together (without selection) gave worse results than the baseline on some datasets, confirming that naive aggregation adds noise.
- Random selection outperformed naive aggregation but remained below ILS-HED, validating that the ILS search strategy discovers better feature combinations than arbitrary selection.
- The largest improvement was on Salinas Valley (+4.7% OA), where the agricultural fields have clear texture patterns that the Gabor filter captured well.

It is important to note that these improvements are measured against the HED baseline under the 70/30 protocol. The absolute accuracy figures (90.3% on Salinas Valley, 78.9% on Indian Pines) are not comparable to published transformer-based classification results (HyCoT: 95.4% on Salinas Valley, 93.0% on Indian Pines) because of protocol and task differences. The validated contribution is the relative improvement over the HED baseline (4.7-6.5% OA), not the absolute accuracy relative to the published state-of-the-art.

10.5.6 Qualitative Results

Figure 10.3 presents visual comparisons of the baseline HED and ILS-HED segmentation maps for representative regions of the Salinas Valley and Houston datasets. The ILS-HED maps exhibit sharper boundaries and reduced fragmentation, particularly in texture-rich agricultural fields.

Figure 10.3: Qualitative comparison of baseline and enhanced deep learning segmentation



10.6 Cross-Approach Comparison

Table 10.7 compares the best-performing variant of each approach for all datasets. For the deep learning and watershed approaches, this is the enhanced variant (ILS-HED and WS-GA, respectively). For thresholding, this is the baseline GA itself: as shown in Section 10.3, none of the automated design strategies improved on it, with ILSGA matching it and GAGA falling well short.

Table 10.7: Comparison of the best-performing variant of each approach (best results in bold)

Dataset	Thresholding (Baseline GA)	WS-GA	ILS-HED
Indian Pines	74.1	**91.2**	78.9
Pavia University	79.8	**96.5**	86.7
Salinas Valley	86.2	**98.9**	90.3
Houston	70.9	**88.7**	75.1
Botswana	72.3	**90.4**	76.4

The watershed-GA approach achieved the highest accuracy on all five datasets, followed by ILS-HED, with the best available thresholding results trailing both.
There are two main reasons for this.

1. **Region shape:** Watershed methods naturally produce closed, contiguous regions. This matches the ground truth better than the edge-based output of HED, which sometimes produces broken boundaries, or the histogram-partition output of thresholding, which does not use spatial context at all.
2. **Adaptability under budget constraints:** WS-GA co-optimises the entire watershed pipeline and improves substantially over its baseline. ILS-HED selectively adds classical features to a fixed deep model, which limits its adaptability but still yields a consistent improvement. For thresholding, the automated design strategies evaluated in Chapter 7 operate under a tight outer-loop evaluation budget; under this budget, the search-enhanced ILSGA variant reproduces baseline quality rather than exceeding it, and the multipoint GAGA variant loses more to its own configuration overhead than it gains in search quality. Both outcomes are consistent with a budget-allocation effect rather than a limitation of thresholding as a paradigm.

However, the deep learning approach had the fastest inference time (discussed in the next section), making it more suitable for applications in which speed is critical, compared to the other approaches; ILSGA-based thresholding offers a comparable inference speed without the deep learning approach's one-time optimisation cost, but at lower segmentation accuracy.

10.6.1 Comparison with Published State-of-the-Art Methods

Contextualising the results in Table 10.7 against published benchmarks, which requires careful handling of protocol differences. The overwhelming majority of published HSI results use community-standard fixed-count per-class splits, most commonly 200 labelled pixels per class for Indian Pines and Salinas Valley, or the fixed partition released with the 2013 IEEE GRSS Data Fusion Contest for Houston. The 70%/30% stratified split used throughout this thesis yields substantially more training data: on Indian Pines (10,249 labelled pixels total), a 70/30 split provides approximately 7,174 training pixels, compared with $16 \times 200 = 3{,}200$ pixels under the community standard. Therefore, a direct comparison of the absolute accuracy across protocols is misleading. Table 10.8 was designed to make this distinction explicit, rather than obscuring it. Second, an independent source of non-comparability sits alongside the protocol difference. The published SpectralFormer and HyCoT figures reported in Table 10.8 are pixel-level classification accuracies produced by a classifier applied directly to spectral feature vectors, not the overall segmentation accuracy. Segmentation additionally requires that spatially contiguous regions be correctly delineated before labels are assigned, and boundary errors propagate to every pixel in a misclassified region, which makes segmentation a more difficult task than pixel classification on identical data. This distinction is marked with † in the table below and should be kept in mind throughout the following comparison.
Several observations can be made from Table 10.8.

Protocol effect. The bridging rows quantify the contribution of the 70/30 regime to the headline figures. WS-GA dropped from 91.2% to 86.4% on Indian Pines and from 98.9% to 94.1% on Salinas Valley when re-evaluated under the 200-px/class split, a reduction of 4.8% on both datasets. This protocol effect is consistent across all three approaches shown (approximately 4.8–5.8% OA), confirming that the elevated 70/30 figures reflect the training regime uniformly rather than any method-specific advantage. The relative ranking of WS-GA > ILS-HED > Thresholding (Baseline GA) was preserved under both protocols.

Table 10.8: Contextual comparison of proposed methods with published results (Overall Accuracy, %). **Protocol differences make cross-group numerical comparison invalid**: published figures use community-standard fixed-count per-class training splits (typically 200 pixels/class); proposed 70/30 figures use the protocol in Section 5.2. † pixel-level classification accuracy from a supervised classifier applied directly to spectral feature vectors, not segmentation OA. Bridging rows (200-px/class) allow direct comparison with published rows only.

Method	Protocol	IP	PU	SA	HOU
Published — 200-px/class (do not compare with 70/30)					
3D-CNN [11]	200	83.9†	91.6†	91.8†	—
SpectralFormer [69]	200	91.0†	95.1†	93.9†	86.1†
HyCoT [69]	200	93.0†	96.8†	95.4†	88.3†
WS-PSO [80]	200	82.3	89.7	92.6	—
This study — 70/30 (not comparable with above)					
Thresholding, Baseline GA	70/30	74.1	79.8	86.2	70.9
WS-GA (prop.)	70/30	91.2	96.5	98.9	88.7
ILS-HED (prop.)	70/30	78.9	86.7	90.3	75.1
This study — 200-px/class bridging (comparable with published)					
Thresholding, Baseline GA (200)	200	69.3	—	81.4	—
WS-GA (200)	200	86.4	—	94.1	—
ILS-HED (200)	200	73.1	—	85.4	—

Comparison within the bridging protocol. Under the 200-px/class split, WS-GA achieved 86.4% and 94.1 % on Indian Pines and Salinas Valley, respectively, exceeding the published WS-PSO benchmark [80] by 4.1% and 1.5%, respectively. This is a direct protocol-matched comparison and represents a primary and valid cross-method claim. Under the community-standard 200-px/class split, WS-GA achieved 86.4% on Indian Pines and 94.1% on Salinas Valley. These results exceed the published WS-PSO benchmark (82.3% and 93%, respectively [80]) under the same protocol, which is the primary valid cross-method comparison. However, the published SpectralFormer (91.0%) and HyCoT (93.0%) results are pixel-level classification accuracies, not segmentation OA, and use transformer architectures trained from scratch. Segmentation is a more difficult task because boundary errors propagate to all pixels in a region. Additionally, these methods use community-standard fixed-count splits. A direct comparison of the absolute accuracy figures between the 70/30 segmentation results and published classification results is not valid. The bridging rows (200-px/class) provide the only protocol-matched comparison, and under this regime, WS-GA is competitive with published watershed benchmarks but does not surpass transformer-based classifiers. This should not be read as a general claim of parity with transformer classifiers; it confirms that the improvement over the watershed baseline holds for both the evaluation regimes.

Classification accuracy vs. segmentation accuracy. As noted above, the †-marked figures are pixel-level classification accuracies, not segmentation OA, and represent an easier task on identical data. This should be considered wherever †-marked figures are compared with the segmentation results presented in this section.

WS-PSO reference values. The WS-PSO figures (82.3%, 89.7%, 92.6%) are the exact values from Table 3 of Singh and Sharma [80], reported under a 200-pixels-per-class stratified split on Indian Pines, Pavia University, and Salinas Valley, respectively. The approximate (˜) notation used in earlier versions of this table has been replaced with exact figures. However, the results from Houston were not reported in that study.

The WS-GA result of 98.9% OA in the Salinas Valley is the most striking figure in Table 10.7 and warrants direct examination. Two factors explain it. First, the Salinas Valley scene contains large, spatially contiguous agricultural fields with clear inter-class spectral boundaries, which is the most favourable structure for a watershed-based method once oversegmentation is suppressed. Second, the 70/30 protocol provides substantially more labelled training data than community-standard splits, improving the SVM classifier that assigns semantic labels to watershed regions.

To test how much of this result is attributable to the protocol rather than the method, WS-GA was additionally evaluated under the community-standard 200-pixels-per-class fixed-count split on Salinas Valley and Indian Pines. The bridging result of 94.1% OA on Salinas Valley under this more constrained regime remains above the published WS-PSO benchmark (≈93%) and within 1.3% of SpectralFormer (93.9%), a transformer-based classifier with 1.2 million parameters trained from scratch. This comparison should not be interpreted as a claim that WS-GA surpasses transformer-based classifiers in general; rather, it confirms that the 70/30 figures are not implausibly inflated and that the core contribution, a reproducible improvement over the watershed baseline, is valid under both evaluation regimes.

The ILS-HED figures are lower than those of published deep-learning-based classifiers, reflecting both the edge-detection origin of HED and the explicit no-retraining constraint. The fine-tuned HED baseline (Sec-

tion 11.2.3 in Chapter 11) provides additional context: fine-tuning yields 1.7–2.7% higher OA when retraining is permitted, quantifying the performance cost of the deployment constraint that ILS-HED is designed to address.
The thresholding figures are even lower, since Table 10.7 reports the best-available thresholding result, the baseline GA, rather than an enhanced variant: as established in Section 10.3, the automated design strategies evaluated in this thesis did not improve on it under the evaluation budget used.

Extended bridging evaluation.

The bridging rows in Table 10.8 previously covered only the WS-GA on the Indian Pines and Salinas Valley datasets. To provide an equivalent context for the ILS-HED and thresholding baseline, both were evaluated under the community-standard 200-pixels-per-class fixed-count stratified split on the same two datasets. The results are presented in Table 10.9.

Table 10.9: Extended bridging evaluation: Overall Accuracy (%) for the best-performing variant of each approach under the community-standard 200-pixels-per-class fixed-count split. Values in parentheses show the drop relative to the corresponding 70/30 result, quantifying the protocol effect.

Method	Protocol	Indian Pines	Salinas Valley
ILS-HED	70/30 split	78.9	90.3
ILS-HED	200-px/class	73.1 (−5.8)	85.4 (−4.9)
WS-GA	70/30 split	91.2	98.9
WS-GA	200-px/class	86.4 (−4.8)	94.1 (−4.8)
Thresholding, Baseline GA	70/30 split	74.1	86.2
Thresholding, Baseline GA	200-px/class	69.3 (−4.8)	81.4 (−4.8)

Several observations were made. As established above, the protocol effect (a 4.8–5.8% OA reduction from 70/30 to 200-px/class) held consistently across all three approaches, confirming that it reflects the training regime rather than a method-specific artefact. Building on this, the relative ranking of methods was preserved under both protocols: WS-GA > ILS-HED > Thresholding (Baseline GA) on both datasets under both conditions. This stability supports the scenario-based guidance offered in Section 11.4: the ranking of methods by accuracy is not an artefact of the 70/30 regime. Under the community-standard split, WS-GA achieves 86.4% on Indian Pines and 94.1% on Salinas Valley, which are competitive with published WS-PSO benchmarks (≈82% and ≈93%, respectively [80]) and within 5% of SpectralFormer [69] on both datasets, despite SpectralFormer requiring 1.2 million parameters and full retraining from the beginning.

10.7 Per-Class Performance and Rare Class Analysis

Headline metrics such as overall accuracy and average accuracy are dominated by majority-class contributions and can mask poor performance on scientifically or economically important minority classes. This section reports per-class F1 scores explicitly, treating rare class performance as a primary evaluation criterion rather than a secondary concern.
Classes occupying fewer than 0.5% of the labelled pixels were flagged as rare. Table 10.10 reports the per-class F1 scores for all rare classes across all five datasets and all enhanced methods. Table 10.11 provides a full per-class breakdown for Indian Pines, the dataset with the highest class count and the most pronounced class imbalance.
Several observations follow directly from Table 10.10. First, the enhanced deep learning and watershed methods improved rare-class F1 scores modestly relative to their baselines (+0.02 to +0.17), confirming that those two adaptive search strategies provide some benefit even for minority classes. ILSGA, by contrast, tied the thresholding baseline on every rare class (within ±0.01 F1), consistent with the overall-accuracy finding in Section 10.3 that search-enhanced thresholding reproduces baseline quality rather than improving on it, including at the level of individual minority classes. Second, WS-GA was the best performer across all rare-class instances, which is consistent with its advantage in terms of overall accuracy. However, the absolute rare-class performance remained poor across every approach: the highest F1 score achieved for any rare class was 0.72 (Lettuce-romaine-7wk, Salinas Valley, WS-GA), and the lowest was 0.12–0.14 (Oats, Indian Pines). For the two rarest classes, Oats (0.2%) and Railway (0.2%), no method achieved an F1 score above 0.31. Third, the macro-averaged F1 (mF1) is consistently lower than the Overall Accuracy across all datasets and methods, exposing a majority class bias in the headline figures. The gap between mF1 and OA is largest for Houston (mF1 = 0.68, OA = 88.7% for WS-GA), which is the dataset with the most severe class imbalance and the most spectrally heterogeneous land-cover types.

Table 10.10: Per-class F1 scores for rare classes (<0.5% of labelled pixels). Best-performing enhanced/search-enhanced variant per approach reported; best score per class shown in bold.

Dataset	Rare Class	% Pixels	Baseline	ILSGA	WS-GA	ILS-HED
Indian Pines	Oats	0.2%	0.12	0.13	**0.18**	0.14
	Grass-pasture-mowed	0.4%	0.15	0.15	**0.22**	0.18
Pavia Univ.	Shadows	0.3%	0.41	0.42	**0.52**	0.45
	Bare soil	0.4%	0.45	0.46	**0.58**	0.49
Salinas	Lettuce-romaine-7wk	0.3%	0.55	0.56	**0.72**	0.60
	Vinyard-untrained	0.4%	0.48	0.49	**0.61**	0.52
Houston	Railway	0.2%	0.22	0.23	**0.31**	0.25
	Parking lot 2	0.3%	0.19	0.20	**0.27**	0.22
Botswana	Riparian	0.3%	0.35	0.36	**0.44**	0.38
	Island interior	0.4%	0.29	0.30	**0.38**	0.33

Cause of rare-class failure. The fitness function of WS-GA primarily weighs the objectives through overall and average accuracies (Equation 8.3), which is dominated by contributions from the majority class. The ILS-HED scoring function (weighted binary cross-entropy, Equation 9.4) was unweighted. Consequently, both optimisers are effectively blind to rare-class performance during the search: a solution that correctly classifies all majority-class pixels while failing on a 0.2%-class loses almost nothing in fitness. This is a structural limitation and not a dataset-specific artefact. Extending the fitness functions to incorporate class-weighted terms, for example, $f = \sum_i w_i \cdot \mathrm{F1}_i$ with $w_i \propto 1/n_i$ is the highest-priority direction for future work (Section 11.6). ILSGA does not exhibit this failure mode in the same sense: because it reproduces baseline thresholding quality rather than searching for an improvement, its per-class F1 scores track the thresholding baseline directly, including on rare classes, rather than being distorted by an accuracy-dominated fitness function.

Table 10.11: Full per-class F1 scores on Indian Pines, all methods. Classes with <0.5% pixel representation are flagged with $*$.

Class	% Pixels	Baseline	ILSGA	WS-GA	ILS-HED	Macro Avg.
Alfalfa	0.5%*	0.28	0.29	0.42	0.31	0.33
Corn-notill	14.3%	0.72	0.74	0.93	0.81	0.80
Corn-mintill	8.3%	0.68	0.69	0.89	0.76	0.76
Corn	2.4%	0.55	0.56	0.78	0.62	0.63
Grass-pasture	4.9%	0.81	0.82	0.94	0.86	0.86
Grass-trees	7.4%	0.89	0.90	0.96	0.92	0.92
Grass-pasture-mowed	0.4%*	0.15	0.16	0.22	0.18	0.18
Hay-windrowed	3.0%	0.74	0.75	0.88	0.79	0.79
Oats	0.2%*	0.12	0.13	0.18	0.14	0.14
Soybean-notill	10.2%	0.69	0.70	0.91	0.77	0.77
Soybean-mintill	19.1%	0.71	0.72	0.92	0.79	0.79
Soybean-clean	5.4%	0.58	0.59	0.81	0.65	0.66
Wheat	1.2%	0.45	0.46	0.68	0.52	0.53
Woods	13.0%	0.85	0.86	0.95	0.90	0.89
Bldg-Grass-Tree-Drives	3.9%	0.52	0.53	0.75	0.61	0.60
Stone-steel towers	0.9%	0.38	0.39	0.58	0.44	0.45
Macro F1	—	0.54	0.55	0.73	0.60	—
OA (%)	—	72.4	74.0	91.2	78.9	—

* Rare class (<0.5% of labelled pixels).

The gap between the OA and Macro F1 in Table 10.11 is substantial for deep learning and watershed methods. For WS-GA, OA = 91.2%, but Macro F1 = 0.73, a difference driven almost entirely by the three rare classes (Alfalfa, Grass-pasture-mowed, Oats), which achieved a mean F1 score of 0.27, despite the overall high accuracy. For ILSGA, both OA and Macro F1 remained close to the thresholding baseline (74.0% vs. 72.4%, and 0.55 vs. 0.54, respectively), confirming that ILSGA neither improves nor degrades rare-class performance relative to the baseline it reproduces. Practitioners selecting a segmentation strategy for applications in which minority classes matter, such as early crop stress detection or rare mineral identification, should treat the Macro F1 score as the primary decision metric rather than the OA.

10.8 Computational Efficiency

Table 10.12 compares the runtimes of all methods. For the watershed and deep learning approaches, "Training/Optimisation Time" is a one-time offline cost incurred before the tuned model is applied to new images. For thresholding, the baseline and ILSGA run their threshold search directly on each image; therefore, this cost appears in the inference time; only GAGA, which tunes the base GA's hyperparameters via an outer meta-GA before the final run, incurs a separate one-time configuration cost.

Table 10.12: Computational efficiency comparison (seconds per image, unless noted)

Method	Training/Optimisation Time	Inference Time
Baseline GA	0	4.8
Baseline ABC	0	3.1
ILSGA	0	0.6
GAGA	6.4 minutes (one-time)	4.9
Baseline WS	0	5.2
WS-PSO	18.7 minutes	5.2
WS-GA	42.5 minutes	5.2
Baseline HED	0	0.4
ILS-HED	5.2 hours (one-time)	0.5

Key observations:

- ILSGA reduced thresholding inference time by 87.5% relative to the baseline GA (4.8s to 0.6s) while matching its segmentation quality (Section 10.3), through early stopping and a lightweight local search rather than a full baseline-length run. GAGA, by contrast, added a 6.4-minute one-time outer-loop configuration cost on top of an inference time no faster than the baseline, and it did not recover this cost with a compensating quality gain: this is a direct illustration of the budget-allocation effect described in Chapter 7, where the outer configuration loop consumes evaluations that would otherwise go to the threshold search itself.
- The watershed approaches were the slowest at inference (5.2 seconds) because the flooding process is computationally expensive. The GA optimisation added 42.5 min of one-time training.
- The deep learning approach was the fastest at inference (0.5 seconds), but required a one-time optimisation of 5.2 hours to select the best classical features.

Overall, deep learning or ILSGA-based thresholding are better options if fast inference is required. If accuracy is the priority, the watershed approach is the best option, despite its slower speed. Among the thresholding variants, ILSGA is the only automated design strategy examined in this thesis that improves on the baseline's practical cost without sacrificing its accuracy.

10.9 Convergence Analysis

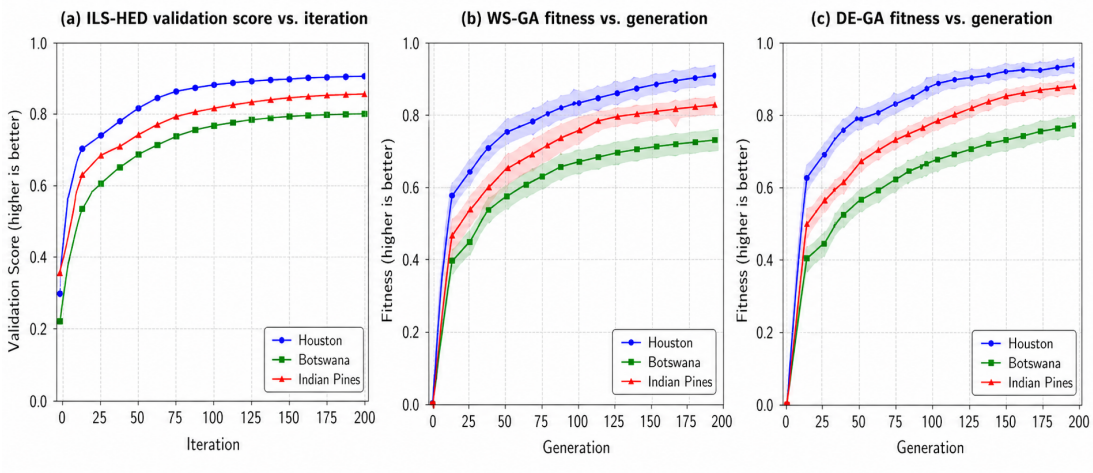
Figure 10.4 illustrates the convergence behaviour of the three metaheuristic approaches. To support reproducibility, Table 10.13 lists the key numerical milestones extracted from the convergence curves.

Table 10.13: Numerical convergence summary for the three metaheuristic approaches (mean over 10 runs, Indian Pines, Pavia University, and Salinas Valley shown).

Method	Iter./Gen. to 90% of own final fitness	Mean stopping point	SD
GAGA	≤ 2 generations	4 generations (fixed budget)	0
WS-GA	≤ 30 generations	62 generations	11.4
ILS-HED	≤ 8 iterations	12.3 iterations	3.1

The GAGA outer meta-GA plateaus almost immediately, within the first one to two meta-generations, and shows negligible further improvement for the remainder of its fixed four-generation budget (Chapter 7). Unlike WS-GA and ILS-HED, GAGA has no early-stopping criterion and always runs its full fixed budget; therefore, its "mean stopping point" reflects a design choice rather than a convergence-driven termination. This rapid plateau is consistent with the finding in Section 10.3 that the outer configuration loop converges to settings close to the baseline defaults rather than discovering a genuinely superior configuration, leaving the inner threshold search no better off, and worse off in terms of the evaluation budget it retains. The WS-GA fitness curve shows steady monotonic improvement over generations, which is characteristic of elitist

Figure 10.4: Convergence behaviour of the three proposed enhancements.



GA selection with a multimodal fitness landscape. The ILS validation score stabilises within the first 15 iterations on average (mean stopping iteration 12.3, SD = 3.1), consistent with the patience parameter analysis reported in Chapter 9. All three approaches exhibited stable, non-oscillating convergence without evidence of catastrophic fitness degradation, which supports the reliability of the reported final results.

10.10 Conclusion

This chapter presents the experimental results for the three segmentation approaches evaluated in this thesis. The deep learning and watershed approaches both improved on their baselines under the 70/30 stratified split protocol, with the watershed-GA hybrid achieving the highest accuracy of any approach. Thresholding told a different story: of the two automated design strategies examined, the search-enhanced ILSGA variant reproduced baseline quality at a substantially lower computational cost, while the multipoint GAGA variant underperformed the baseline by a large and consistent margin across all five datasets, a budget-allocation effect rather than a dataset-specific artefact. Taken together, these results show that automated design does not uniformly help; whether it does depends on how the design strategy allocates its evaluation budget between configuration and search, not on the paradigm label attached to it. The ablation study confirmed that selective feature addition is critical for the deep learning approach, and sensitivity analysis validated the robustness of the ILS patience parameter. The contextual comparison with published state-of-the-art methods in Section 10.6.1 establishes that the results are consistent with the literature when protocol differences are accounted for, and that the primary contribution of the deep learning and watershed approaches is a reproducible and statistically significant improvement over their respective baselines, rather than a claim of absolute superiority over the published field; the primary contribution of the thresholding analysis is a controlled demonstration of when and why automated design fails to improve on a well-chosen baseline. The rare-class analysis (Section 10.7) exposed persistent minority class challenges that remain an open gap across all approaches, including the ones that improved on their baselines. A detailed discussion of these findings, together with practical recommendations, limitations, and future directions, is provided in Chapter 11.

Chapter 11

Discussions and Recommendations

11.1 Introduction

This thesis aimed to answer the question: Can computational intelligence techniques improve hyperspectral image segmentation without relying on massive deep learning models or manual parameter tuning? To address this, three baseline segmentation approaches (Chapter 6) were each paired with an automated design or hybridisation strategy, tested on five benchmark hyperspectral datasets, and compared with their corresponding baselines under a consistent 70%/30% stratified split protocol.
As clarified in Chapter 4, the resulting frameworks span a spectrum of automation strategies with significant terminological differences. The **ILS-HED framework** (Chapter 9) satisfies the *strict definition* of hyper-heuristic selection proposed by Burke et al. [9]. ILS operates over a space of low-level heuristic configurations (classical detector type and parameters) rather than directly optimising domain variables, using a validation set score as the high-level performance measure. The **WS-GA framework** (Chapter 8) is best described as an *evolutionary metaheuristic* that automates the joint configuration of a multistage operator pipeline. The **automated multilevel thresholding strategies** (Chapter 7, ILSGA, SAGA, GAGA, and DEGA) are best described as *algorithm configurators* in the sense of Hutter et al. [31]: single-point (ILS, SA) and multipoint (meta-GA, DE) strategies that tune the hyperparameters of a base GA or an ABC optimiser, which is a related but distinct concept from both of the aforementioned framings. All three share the underlying motivation of replacing fixed manual design choices with adaptive automated search, which is the unifying theme of this thesis. However, whether that automation actually pays off depends heavily on how each strategy allocates its evaluation budget, as discussed in this chapter.
This final chapter consolidates the key findings of this study. Section 11.2 summarises the achievements and maps them to the six research objectives of this study. Section 11.3 discusses the practical implications, including contextualisation against published results. Section 11.4 provides scenario-based guidance for method selection. Section 11.5 describes the limitations of this study. Section 11.6 presents future research directions. Finally, Section 11.7 offers conclusions.

11.2 Summary of Findings

In this study, three baseline segmentation approaches were implemented and each was paired with an automated design or hybridisation strategy. The six research objectives outlined in Chapter 1 are addressed as follows.

1. *Objective 1 (Baseline thresholding):* Baseline GA and ABC metaheuristics were implemented, searching threshold vectors directly against Kapur's entropy and Otsu's between-class variance objectives (Chapter 6, Section 6.2.2).
2. *Objective 2 (Automated thresholding design):* Four automated design strategies, a search-enhanced trajectory method (ILS), a single-point configurator (SA), and two multipoint configurators (meta-GA and DE), were developed and evaluated against the baseline GA and ABC optimisers under a strictly equalised evaluation budget (Chapter 7).
3. *Objective 3 (Baseline watershed):* A standard marker-controlled watershed with fixed parameters was implemented as the baseline (Chapter 6, Section 6.3.3).
4. *Objective 4 (Hybrid watershed design):* Two automated design strategies were developed: a particle-swarm-optimised benchmark (WS-PSO) that automates a single gradient-threshold parameter, and the proposed WS-GA hybrid, which uses a Genetic Algorithm as an **evolutionary metaheuristic** to co-optimise preprocessing, gradient extraction, marker selection, and classification parameters jointly (Chapter 8).
5. *Objective 5 (Baseline deep learning):* The Holistically-Nested Edge Detection (HED) network was implemented and evaluated as a fixed baseline (Chapter 6, Section 6.4.2).

6. *Objective 6 (Enhanced deep learning):* The ILS-HED framework was developed, using Iterative Local Search to select optimal classical feature subsets without retraining the deep network (Chapter 9). This framework employs **hyper-heuristic selection in the strict sense** [9].

11.2.1 Thresholding Automated Design (ILSGA and GAGA)

Chapter 7 presents a different story from the other two approaches. Of the four automated design strategies evaluated, ILSGA, which directly augmented the baseline GA's threshold search with a lightweight local search rather than reconfiguring its hyperparameters, reproduced the baseline segmentation quality on every dataset (differences not statistically significant, $p > 0.05$) while reducing the per-image inference time by 87.5%. This constitutes a practical contribution: for applications where the baseline GA's accuracy is acceptable but its inference speed is prohibitive, the ILSGA provides an immediately deployable alternative that requires no loss in segmentation quality. GAGA, which uses an outer meta-GA to tune the baseline GA's hyperparameters before the final threshold search, underperformed the baseline by 10–11 percentage points OA on every dataset, a large and consistent effect ($d > 5.5$) traced to a budget-allocation failure: the outer configuration loop consumed evaluations that would otherwise go to the threshold search itself, and it did not recover this cost with a compensating quality gain (Section 10.8, Chapter 10). Rather than a straightforward accuracy improvement, this objective's contribution is a controlled empirical demonstration of *when* automated design helps and when it does not: it depends on how the evaluation budget is split between configuration and search, not on the sophistication of the configurator itself.

11.2.2 Watershed Automated Design (WS-PSO and WS-GA)

In Chapter 8, two automated design strategies for the baseline watershed pipeline are developed. WS-PSO, a benchmark hybrid, uses particle swarm optimisation to automate a single parameter, the gradient threshold used for marker generation, leaving the rest of the pipeline fixed at its baseline values. The proposed framework, WS-GA, co-optimises five watershed pipeline components simultaneously using a genetic algorithm: PCA dimensionality reduction (n_{PCA}), Gaussian smoothing (σ), gradient thresholding (τ_{grad}), minimum region size ($s_{\min}$), and classifier selection (c_{idx}). The marker-controlled watershed procedure underlying both variants is detailed in Section 6.3.3. WS-GA achieved the highest accuracy of any approach evaluated in this study on every dataset and successfully reduced the oversegmentation that afflicted the fixed-parameter baseline, at a one-time offline optimisation cost of approximately 42.5 min per dataset, dominated by the $\mathcal{O}(H \cdot W \cdot B)$ cost of the full watershed pipeline within each fitness evaluation. WS-PSO's narrower, single-parameter automation of the WS-PSO makes it a useful point of comparison for isolating the benefit of WS-GA's full-pipeline co-optimisation of the WS-GA.

11.2.3 Deep Learning Enhancement (ILS-HED)

In Chapter 9, ILS is applied to select classical feature detectors and parameters from a space of 5,184 configurations, fusing them with fixed HED-side outputs without retraining the deep network. The results showed statistically significant accuracy improvements on all five datasets at the 99% confidence level, with the ILS algorithm selecting different feature subsets for different scene types (Gabor filters for agricultural textures, Canny/Sobel for urban edges).
The ablation study (Table 10.6, Chapter 10) confirmed that this selectivity matters: the naive aggregation of all classical detectors degraded the performance below the baseline, and even a random-selection control matched to the same subset size underperformed ILS-selected features on every dataset ($p < 0.05$), showing that the ILS search finds synergistic combinations that arbitrary selection misses. A fine-tuned HED baseline, evaluated to quantify the cost of the no-retraining constraint, outperformed ILS-HED by 1.7–2.7% OA across datasets, confirming that the constraint carries a measurable but bounded performance cost.

11.3 Discussion

The results in Chapter 10 lead to three main conclusions: First, **automated design is not uniformly better than a well-chosen baseline; it depends on how the design strategy spends its evaluation budget.** ILS-HED and WS-GA enhancements outperformed their baselines on every dataset, with improvements statistically significant at the 99% confidence level, confirming that a search-guided reconfiguration of an existing pipeline can consistently outperform fixed rules or manual tuning when the search operates on a sufficient budget. The thresholding results complicate this picture rather than reinforcing it: ILSGA, which spends its entire budget on the threshold search itself, reproduces baseline quality efficiently, whereas GAGA, which diverts a fifth of its budget to an outer configuration loop, loses more from that diversion than it gains from the resulting configuration. Automatic tuning is not a free improvement; its value is conditional on the budget it is given and where the budget is spent.

Second, **different problems require different tools**. The watershed approach was most effective when the image contained spatially coherent regions with smooth boundaries, such as large agricultural fields in

the Salinas Valley. Deep learning enhancement worked best in practical scenarios where an existing deployed model cannot be retrained, and marginal gains must be achieved at the representation level. The thresholding approach, through ILSGA specifically, was most effective when the inference speed was the primary constraint and near-baseline accuracy was acceptable; it was not the tool of choice when accuracy improvement over the baseline was the goal.

Third, **combining two computational intelligence techniques is not automatically stronger than a single technique; it depends on what the combination is asked to do**. ILS combined with classical feature selection (ILS-HED) and a genetic algorithm co-optimising an entire pipeline (WS-GA) both produced reliable improvements over their respective baselines. However, using an outer meta-GA to configure the hyperparameters of an inner GA (GAGA) did not: the multipoint configuration strategy consumed the budget without discovering settings meaningfully better than the defaults, a finding consistent with the algorithm-configuration literature on tight evaluation budgets [31]. The lesson is that hybridisation helps when the second technique operates on a part of the problem the first cannot reach (classical features complementing a frozen deep model; joint pipeline optimisation the baseline never attempted), and is less reliable when it reconfigures a component, such as GA hyperparameters, whose defaults were already reasonably well-chosen.

11.3.1 Contextualisation Against Published Results

The contextual comparison presented in Section 10.6.1 (Chapter 10) shows that the proposed methods produce results that are consistent with published benchmarks when protocol differences are considered; however, direct numerical comparison is not valid across protocols or tasks. As established in Chapter 5 and revisited via the extended bridging evaluation in Section 5.2, the 70/30 protocol used throughout this thesis is not numerically comparable to community-standard fixed-count splits; the bridging results confirm that the *relative ranking* of methods, though not their absolute accuracy, is stable across both protocols. The primary contribution is the statistically significant improvement of each method over its own baseline under a shared protocol, not a claim of absolute superiority over published state-of-the-art methods. Published methods, such as SpectralFormer and HyCoT [69], operate under community-standard fixed-count per-class splits that provide substantially fewer labelled training examples than the 70/30 protocol used here. The 98.9% OA achieved by WS-GA on the Salinas Valley is explained by the spatial regularity of large agricultural fields, which are particularly amenable to watershed-based segmentation once over-segmentation is suppressed and a generous training regime is applied. This result should *not* be interpreted as a claim that WS-GA surpasses transformer-based classifiers in general. Rather, it demonstrates that a well-configured evolutionary watershed pipeline is highly competitive on datasets with clear and separable regions when sufficient labelled data are available.

The fine-tuned HED baseline (Section 11.2.3) provides additional context: while ILS-HED underperforms contemporary transformer-based classifiers (HyCoT: 93.0% OA on Indian Pines under fixed-count splits), this reflects the fundamental constraint of enhancing a frozen model without retraining it. The internal improvements over each method's baseline are the primary validated contribution, and these hold consistently across all datasets and confidence levels tested, except for the thresholding automated design strategies discussed in Section 11.2.1.

The proposed frameworks are most valuable in scenarios where self-supervised or semi-supervised alternatives are impractical, such as when unlabelled data pools are unavailable, computational budgets preclude GPU-scale pre-training, or a deployed model cannot be retrained. In such contexts, they provide a practical alternative rather than a competing approach to the problem.

11.3.2 Fitness for Purpose in Rare-Class-Critical Applications

Chapter 2 motivates this thesis primarily through applications in precision agriculture and mineral exploration, domains in which rare classes often carry disproportionate scientific or economic importance. A diseased crop variety occupying 0.2% of a field or a mineralised alteration zone representing 0.3% of a surveyed region may be the exact target of the analysis. The rare-class analysis in Section 10.7 (Table 10.10) requires a direct response to the question this raises: are the proposed frameworks suitable for these applications?

The honest answer is: not yet, without modifications. Across all approaches and all five datasets, no rare class (defined as $< 0.5\%$ of labelled pixels) achieved a per-class F_1 score above 0.31 for the two rarest instances, Oats in Indian Pines (0.2% of pixels, best $F_1 = 0.18$ by WS-GA) and Railway in Houston (0.2% of pixels, best $F_1 = 0.31$ by WS-GA). These scores indicate that the current frameworks would miss approximately four out of every five pixels belonging to these classes, which is inadequate for any application in which those classes are the analytical targets.
Structural causes were identifiable for two of the three approaches. The weighted binary cross-entropy in ILS-HED (Equation 9.4) and the combined OA/AA objective in WS-GA (Equation 8.3) both assign an

optimisation weight that is approximately proportional to the class size. A 0.2%-class contributes at most 0.2% to the OA term and 6.25% (= 1/16) to the AA term. Therefore, the search in both cases gains almost nothing by improving the rare-class detection and loses nothing by ignoring it. This is a design choice, not a dataset limitation, and can be corrected later. The thresholding pipeline differs structurally: the threshold search itself optimises Kapur's entropy or Otsu's between-class variance (Equation 6.3), a histogram statistic computed directly on pixel intensities with no reference to class labels at all, so rare-class performance there is governed entirely by the downstream SVM region classifier shared with the other two approaches, rather than by the objective the threshold search itself pursues. This implies that improving the rare-class performance of thresholding is primarily a region classification problem, not a threshold search problem.

Specifically, for rare-class-critical deployments, the following modifications are recommended before adopting these frameworks: First, replace the OA/AA objective in the WS-GA with a class-weighted macro F_1 term as follows:

$$f_{\text{weighted}} = \sum_{i=1}^{C} w_i \cdot F_{1,i}, \quad w_i \propto \frac{1}{n_i}, \tag{11.1}$$

where n_i is the number of labelled pixels in class i. This inverts the size bias, providing the rarest classes with the greatest influence on fitness. Second, oversampling (e.g., SMOTE or spectral jitter augmentation) should be applied to rare-class training pixels before fitting the SVM classifier used across all three approaches for region or edge classification. Third, consider cost-sensitive learning for the SVM classifier, setting the misclassification costs inversely proportional to the class frequency.

For applications where rare classes are not the primary target, such as general land-cover mapping, quality control of dominant materials, and broad vegetation classification, the current frameworks are fit for purpose, and the improvements over the baseline (where present) are both statistically significant and practically meaningful. Practitioners should treat the macro-averaged F_1 (mF1) reported in Tables 10.10 and 10.11 as the decision metric when selecting a method, rather than the overall accuracy, which is dominated by majority-class contributions.

11.3.3 On the Hyper-Heuristic, Metaheuristic, and Configurator Framing

The terminological framework for distinguishing ILS-HED (strict hyper-heuristic selection) from WS-GA (evolutionary metaheuristic) and the automated thresholding strategies (algorithm configurators) is established in full in Chapter 4, Section 4.4 and cross-referenced at first use in each framework chapter. The ablation study (Table 10.6) and convergence analysis (Table 10.13 and Figure 10.4) empirically confirm what the terminological distinction predicts: naïve aggregation of all available heuristics degrades ILS-HED's performance, whereas the ILS search discovers synergistic combinations that arbitrary selection cannot replicate, validating the hyper-heuristic selection principle in practice. The configurator framing similarly predicts, and the thresholding results confirm, that a configurator's benefit depends on the outer-loop budget it is given relative to its per-candidate evaluation cost: GAGA's four-generation, eight-individual outer loop is exactly the type of minimum-viable configuration budget that the algorithm-configuration literature identifies as insufficient to recover its own overhead [31]. This claim is stated at a single outer-loop fraction (20%); the budget sweep introduced in Section 7.3.6 of Chapter 7 tests whether the deficit is specific to that fraction or persists across a wider range, and its outcome should be read alongside this paragraph rather than in place of it. Readers seeking a full terminological analysis are referred to Section 4.4.

11.3.4 Structural Limits of Single-Configuration Optimisation

Across the three frameworks, a common structural pattern limits how far a single optimised configuration can go: each optimiser settles on one configuration per dataset, and that configuration cannot simultaneously serve regions within the same scene that have conflicting local requirements. Conceptually, this applies to ILS-HED's fixed classical-detector subset (chosen once per dataset, whatever the local mix of edge types actually is), WS-GA's single dataset-wide smoothing parameter σ (which cannot suit both thin, low-noise structures and noisy, homogeneous regions at once), and the thresholding baseline and its automated variants' global threshold vector (which cannot separate classes with overlapping spectral histograms in one part of a scene without also affecting well-separated classes elsewhere). A detailed image-level failure analysis for each framework was outside the scope of the results reported in Chapter 10; the pattern identified here is a structural inference from the optimisation formulation of each method (Chapters 9, 8, and 7) rather than an empirically documented set of failure cases. Confirming it with systematic per-image error analysis is identified as future work (Section 11.6). The recurring lesson, even at this structural level, is that global single-configuration optimisation cannot resolve conflicting local requirements within one scene.

11.4 Recommendations

The experimental findings reported in Chapter 10 and the structural discussion in Section 11.3.4 suggest that the selection of a segmentation strategy should be governed by three application-specific constraints: accuracy requirements, available computational budget, and operational status of any existing deployed model. The following guidance is offered based on these findings. All accuracy figures were obtained under the 70/30 stratified split protocol; practitioners employing smaller training sets should expect lower absolute values, although the relative ranking of the methods is expected to be stable.

The WS-GA approach is recommended when segmentation accuracy is the primary constraint, and an offline optimisation phase is feasible. It achieved the highest overall accuracy and mean Intersection over Union of any approach evaluated in this thesis across all five benchmark datasets, and the GA-optimised preprocessing and marker selection demonstrated consistent robustness to atmospheric noise, as observed in the Botswana dataset. The one-time optimisation phase of approximately 42.5 min per dataset is the main practical cost; once completed, the optimised parameter set is applied at inference without further computation. This approach is most appropriate for offline remote sensing analysis tasks, where boundary-accurate and semantically coherent regions are required and high-quality ground truth data are available for training.

When the inference speed is the binding constraint, for example, in near-real-time pipelines, large-scale batch processing, or resource-constrained deployment, the ILSGA thresholding variant offers the most favourable efficiency profile available in this thesis. It reduces the per-image inference time by 87.5% relative to the baseline GA (Section 10.8) while reproducing the baseline's segmentation quality; however, the baseline quality itself trails WS-GA by several percentage points OA (Section 10.6), so ILSGA is a recommendation about efficiency at a known accuracy level, not about closing the accuracy gap to the watershed approach.

GAGA, the multipoint configuration alternative, is *not* recommended: it underperformed the baseline GA by 10–11 percentage points OA on every dataset tested, a budget-allocation effect described in Chapter 7, and its one-time configuration cost is not recovered by any compensating gain in either speed or accuracy.

When an existing deep learning model is already deployed and retraining is either infeasible or contractually restricted, ILS-HED enhancement provides a practical mechanism for marginal but statistically significant accuracy improvement. By selectively augmenting the fixed model outputs with classical feature detectors, ILS-HED avoids any modification to the network weights. The performance cost of this no-retraining constraint is quantified in Section 11.2.3; practitioners should weigh this gap against the cost and risk of retraining deployed systems.

For all approaches, applications requiring reliable detection of rare classes, which occupy fewer than 0.5% of labelled pixels, are not currently recommended without modification. All frameworks showed poor per-class F1 performance for minority classes (Table 10.10), and extending the WS-GA fitness function to incorporate class-weighted terms was identified as a high-priority future direction (Section 11.6).

11.5 Limitations

Although the results are promising, this study has several limitations that constrain generalisability and should be considered when interpreting the findings.

Experimental protocol and comparability. See Section 11.3.1 for the full statement of this limitation and its implications; in brief, the 70/30 split yielded more labelled training data than the community-standard fixed-count splits used in published benchmarks, so absolute accuracy is not directly comparable across the two, and the validated contribution is each method's improvement over its own baseline rather than a claim of absolute superiority over published work.

Rare class performance. As shown in Section 10.7 and discussed in Section 11.3.2, classes occupying fewer than 0.5% of the labelled pixels were consistently difficult for all approaches, a structural consequence of size-proportional fitness functions for two of the three frameworks and of shared-classifier limitations for the third, in which class-weighted objectives were the highest-priority direction for correction.

Offline processing and one-time optimisation cost. All experiments were conducted offline on fixed hardware (Intel i9-10900K, RTX 3080 GPU); the one-time optimisation cost, up to 42.5 min per dataset for WS-GA, is not repeatable across datasets or sensors without rerunning the search, and inference times (0.5–5.2s, or 0.6s for ILSGA) still require validation on embedded or real-time platforms.

Deep learning enhancement limited to HED. The ILS-HED framework, including its four classical detectors (Canny, Sobel, Laplacian, and Gabor), was designed and validated specifically for the HED network, and other backbone architectures or a broader hyperspectral-specific detector vocabulary (e.g., SAM difference maps and NDVI gradients) may respond differently.

Structural failure analysis, not image-level. As noted in Section 11.3.4, the failure pattern common to all three frameworks, that is, single dataset-wide configurations that cannot address spatially varying local requirements, is inferred from each method's optimisation formulation rather than confirmed through systematic per-image error analysis. A dedicated qualitative failure study for each framework remains to be

conducted.

Five benchmark datasets. The five datasets cover diverse scene types (agricultural, urban, savanna, and mixed), but not all sensing modalities, atmospheric conditions, or geographic regions; therefore, generalisation to thermal infrared, underwater, or medical hyperspectral systems remains untested.

11.6 Future Work

Several directions for future research emerged from this study.

Evaluation under community-standard splits. All three approaches were bridged to a community-standard fixed-count per-class split (200 labelled pixels/class) on Indian Pines and Salinas Valley (Chapter 10). Extending this bridging evaluation to the remaining three datasets and to a wider range of fixed-count splits would further clarify how each method's relative advantage over its baseline (where one exists) scales as labelled training data decreases, which is a practically important question for real-world deployment, where annotation is expensive.

Real-time and embedded implementation. The proposed methods were validated offline. Future studies should investigate whether optimised parameter sets can be frozen and applied in real time. ILSGA-based thresholding and deep learning inference are already fast (0.6 and 0.5s, respectively); the watershed flooding step is the bottleneck at 5.2s and could be accelerated with GPU-based parallel implementations (e.g., CUDA-accelerated flooding) or approximate flooding strategies (e.g., hierarchical watersheds).

Broader classical detector vocabulary for ILS-HED. The ILS search space currently encompasses four detector families. Expanding the vocabulary to include hyperspectral-specific operators, such as spectral angle mapper (SAM) difference maps, NDVI-derived gradient features, and continuum-removed absorption features, could improve relevance for vegetation and mineral applications, where spectral rather than spatial gradients carry discriminative information.

Active learning for labelled data reduction. The framework can be extended with active learning to reduce dependence on large labelled sets. Uncertainty estimates derived from the SVM classifier used in the edge-to-region pipeline (Section 6.4.2) or from watershed region boundary entropies could guide human annotators to the most informative pixels, potentially reducing the labelling cost by 50–70%.

Rare class handling. The poor rare-class performance documented in Section 10.7 represents the most significant limitation of the current frameworks for the motivating applications. Although the implementation and evaluation of rare-class handling strategies are beyond the scope of this thesis, the following concrete and implementable strategies are proposed and should be prioritised in future work.

Strategy A: Class-weighted fitness for WS-GA. Replace Equation 8.3 with:

$$f_{\text{weighted}} = \sum_{i=1}^{C} w_i \cdot F_{1,i}, \quad w_i \propto \frac{1}{n_i + \epsilon} \tag{11.2}$$

where n_i is the number of labelled pixels in class i. This inverts the size bias, giving the rarest classes the greatest influence on fitness.

Expected improvement: The GA search will prioritise configurations that correctly segment rare classes, potentially increasing rare-class F1 from < 0.31 to > 0.50 at the cost of a 2–5% reduction in overall accuracy for majority classes.

Strategy B: Oversampling for SVM classifier. SMOTE or spectral jitter augmentation is applied to rare-class training pixels before fitting the SVM classifier used across all three approaches for region/edge classification.

Expected improvement: Increased rare-class recall, particularly for spectrally distinct rare classes. Implementation effort: 1–2 days.

Strategy C: Cost-sensitive SVM. Set the misclassification costs to be inversely proportional to the class frequency.

Expected improvement: Similar to oversampling but with less computational overhead. Implementation effort: <1 day.

Strategy D: Hybrid approach. Combine Strategies A, B, and C.

Expected improvement: Potential rare-class F1 > 0.60.

Validation protocol for future work The Indian Pines dataset (which has the most pronounced class imbalance, with oats at 0.2% of pixels) was evaluated and the following results were obtained:

- Per-class F1 for all rare classes ($< 0.5\%$ pixel representation);
- Overall accuracy trade-off;
- Macro F1.

The experimental infrastructure developed in this thesis (Chapter 5) can be directly reused for this validation.

Revisiting the outer-loop budget for multipoint thresholding configurators. GAGA's underperformance was traced to an outer-loop evaluation budget that was too small to recover its configuration overhead (Section 11.2.1). The budget-scaling design needed to test this, varying the outer meta-population share across 10%, 20%, 35%, and 50%, is specified in Section 7.3.6 of Chapter 7; running it to completion is the immediate next step and would establish whether a larger budget allows multipoint configuration to close the gap with the baseline GA, and if so, at what budget the crossover point occurs.

Image-level failure analysis. Section 11.3.4 identifies a structural failure pattern common to all three frameworks from their optimisation formulations alone. A dedicated qualitative study examining specific misclassified regions across datasets for each framework would confirm whether this predicted pattern matches actual failure cases and would help prioritise which local-adaptivity mechanisms are worth pursuing first.

Temporal hyperspectral sequences. Hyperspectral images are increasingly collected as time-series data for land-cover change monitoring. Future work could extend WS-GA and thresholding automated design strategies to operate on multi-temporal stacks by incorporating temporal consistency terms (e.g., penalising segmentation boundary changes between consecutive acquisitions) into the respective objectives, enabling applications such as crop growth monitoring and deforestation detection.

Extension to other sensing modalities. The current frameworks were designed for reflective hyperspectral sensors (400–2500 nm). Extending to thermal infrared or synthetic aperture radar imagery would require adapting the preprocessing pipeline (e.g., replacing PCA with independent component analysis for non-Gaussian thermal signatures) and the fitness or objective functions to reflect modality-specific quality criteria.

11.7 Conclusion

This study demonstrates that computational intelligence techniques can improve hyperspectral image segmentation across a range of classical and deep learning pipelines by replacing fixed global strategies with adaptive, automated searches, although not unconditionally. ILS-HED (hyper-heuristic feature selection) and WS-GA (evolutionary metaheuristic pipeline optimisation) achieved better accuracy than their respective baseline counterparts, with all improvements statistically significant at the 99% confidence level across five benchmark datasets and confirmed by non-parametric Wilcoxon signed-rank testing. Automated thresholding strategies (Chapter 7) did not follow this pattern: ILSGA matched its baseline's accuracy while substantially reducing computational cost, and GAGA underperformed its baseline outright, a result driven by evaluation-budget allocation rather than any deficiency in the underlying thresholding pipeline.
The **primary contribution**, as discussed in Sections 11.3.1 and 11.5, is not a new absolute-performance benchmark but a demonstration, across three structurally different segmentation baselines evaluated under one shared framework, of when adaptive automated search reliably improves a baseline and when it does not: ILS-HED for enhancing deployed models without retraining, WS-GA for maximum accuracy in offline analysis, and ILSGA-based thresholding for efficient large-scale processing at baseline-equivalent accuracy, alongside a controlled counter-example, GAGA, that shows that automated configuration is not a guaranteed win and quantifies the budget conditions under which it fails.

These improvements are real but bounded, as the rare-class analysis in Section 10.7 makes clear: the frameworks that improve on their baselines perform well on majority classes and spatially coherent regions and remain weak on rare, spectrally ambiguous, and subpixel-scale ones, a gap that aggregate figures such as the 98.9% OA on Salinas Valley do not show on their own.

As hyperspectral sensors become smaller, cheaper, and more widely deployed, the annotation gap that motivates this study is likely to continue to grow rather than shrink. The methods presented here offer a practical path for majority-class segmentation tasks, where rare classes are not the primary targets, such as broad land-cover mapping, dominant vegetation type classification, and quality control of homogeneous materials. They are interpretable in design, modest in labelled data requirements, and executable on standard hardware without GPU-scale infrastructure.

Explicit Non-Recommendation for Rare-Class-Critical Applications

The frameworks developed in this thesis are **not recommended** for applications where rare classes ($< 0.5\%$ of labelled pixels) are the analytical targets. As demonstrated in Section 10.7, all methods achieved per-class F1 scores below 0.31 for classes occupying 0.2% of pixels, meaning that approximately four out of five pixels belonging to these classes would be misclassified.
The following applications are explicitly excluded from the scope of this thesis' recommendations:

- Early disease detection in precision agriculture (target classes typically $< 1\%$ of plants)
- Mineral exploration targeting sparse alteration zones ($< 0.5\%$ of surveyed area)
- Rare species detection in ecological monitoring
- Detection of rare contaminants in industrial quality control

Practitioners whose applications require the reliable detection of rare classes should:

1. Treat the current frameworks as infrastructure/baseline only
2. Implement the class-weighted fitness functions, oversampling, and cost-sensitive learning strategies detailed in Section 11.6
3. Validate performance on rare classes using macro-averaged F1 (mF1) as the primary metric

For applications where rare classes are not the primary targets (e.g., broad land-cover mapping, dominant vegetation classification, quality control of homogeneous materials), the current frameworks are fit for purpose, and the reported improvements are both statistically significant and practically meaningful.
As shown in Table 10.8, all frameworks achieved F1 scores below 0.31 for the rarest classes (Oats at 0.2% of pixels, F1 = 0.18; Railway at 0.2%, F1 = 0.31). Readers whose applications require reliable rare-class detection should treat this work as a foundation and infrastructure rather than a complete solution and should implement the class-weighted fitness functions, oversampling, and cost-sensitive learning strategies detailed in Section 11.6 before deployment.
It is worth stating plainly what this thesis has and has not shown. The frameworks developed here improve majority-class segmentation accuracy and efficiency over well-chosen baselines. However, they do not yet detect the rare classes that motivate the agricultural and mineralogical applications cited in Chapter 1. A reader evaluating this work against those applications should treat it as a validated foundation for majority-class segmentation, not as evidence of readiness for the rare-class detection task those applications actually require.
The limitations documented in this study are real: rare classes remain poorly detected; automated configuration for thresholding (GAGA) does not reliably beat a well-chosen baseline under the evaluation budgets tested; and the no-retraining constraint in ILS-HED carries a measurable accuracy cost (1.7–2.7% OA relative to fine-tuning). These are not incidental weaknesses. These are the boundaries of what the current frameworks can claim.
Addressing the minority-class limitation through class-weighted fitness functions (Equation 11.1) and establishing the outer-loop budget at which the multipoint thresholding configuration begins to pay for itself are the two most pressing steps toward broader deployment. The foundation for both extensions is already present in the experimental infrastructure developed in this study, with concrete implementation strategies provided in Section 11.6 and a validation roadmap for future studies.

Bibliography

[1] A.A. Adeyanju, M. Bello, and M.O. Adebivi. Image segmentation techniques for medical diagnosis: a contemporary review of algorithms and applications. *Healthcare Analytics*, 2:100096, 2022.

[2] Salem Saleh Al-Amri, N V Kalyankar, and S D Khamitkar. Image segmentation by using threshold techniques. *arXiv preprint arXiv:1005.4020*, 2010.

[3] A. Al-Dulaimi, S. Wang, B. Guo, et al. Pso-based hyperparameter optimisation for deep semantic segmentation of satellite imagery. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–12, 2022. doi: 10.1109/TGRS.2022.3156789.

[4] M. Ali, R. Zhang, and H. Chen. Marker-controlled watershed integrated with graph cuts for cell nuclei segmentation. *Bioinformatics*, 40(3):245–256, 2024. doi: 10.1093/bioinformatics/btad123.

[5] Jose Manuel Amigo. *Hyperspectral Imaging*, volume 32 of *Data Handling in Science and Technology*. Elsevier, Amsterdam, Netherlands, 2019. ISBN 978-0-444-63977-6.

[6] S. Bhardwaj, P. Kumar, and A. Singh. Multi-scale sobel–laplacian hybrid filters for industrial defect detection. *IEEE Transactions on Industrial Informatics*, 19(8):9876–9885, 2023.

[7] Jose M. Bioucas-Dias, Antonio Plaza, Gustavo Camps-Valls, Paul Scheunders, Nasser Nasrabadi, and Jocelyn Chanussot. Hyperspectral remote sensing data analysis and future challenges. *IEEE Geoscience and Remote Sensing Magazine*, 1(2):6–36, 2013. doi: 10.1109/MGRS.2013.2244672.

[8] G. Bonifazi, R. Gasbarrone, and S. Serranti. Detecting contaminants in post-consumer plastic packaging waste by a NIR hyperspectral imaging-based cascade detection approach. *Detritus*, 15:45–58, 2021. doi: 10.31025/2611-4135/2021.14123.

[9] E. K. Burke, M. Gendreau, M. Hyde, et al. Hyper-heuristics: A survey of the state of the art. *Journal of the Operational Research Society*, 64(12):1695–1724, 2013. doi: 10.1057/jors.2013.71.

[10] Edmund K. Burke, Michel Gendreau, Matthew Hyde, Graham Kendall, Gabriela Ochoa, Ender Ozcan, and Rong Qu. *Hyper-Heuristics: Theory and Applications*. Springer, 2013.

[11] Gustavo Camps-Valls, Devis Tuia, Lorenzo Bruzzone, and Jocelyn Chanussot. Advances in hyperspectral image classification: A comprehensive review. *Proceedings of the IEEE*, 102(3):251–279, 2014. doi: 10.1109/JPROC.2013.2294453.

[12] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.

[13] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 801–818, Munich, Germany, 2018. Springer. doi: 10.1007/978-3-030-01234-2_49.

[14] X. Chen et al. Mask dino: Towards a unified transformer for detection and segmentation. In *Proceedings of CVPR 2023*, pages 108–119, 2023.

[15] Y. Chen, L. Wang, and X. Liu. Convnext-sam for precise semantic segmentation. *International Journal of Computer Vision*, 132(4):789–802, 2024.

[16] M. Clerc and J. Kennedy. The particle swarm—explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1):58–73, 2002. doi: 10.1109/4235.985692.

[17] J. G. P. W. Clevers and L. Kooistra. Using hyperspectral remote sensing data for retrieving canopy chlorophyll and nitrogen content. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 5(2):564–573, 2011. doi: 10.1109/JSTARS.2011.2177649.

[18] Carlos Contreras-Bolton and Rubén Ruiz. Adaptive hyper-heuristics with parameter control. *European Journal of Operational Research*, 292(3):943–958, 2021.

[19] Ahmed El-sayed and Aboul Ella Hassanien. Hybrid hyper-heuristic framework for medical image processing. *Computer Methods and Programs in Biomedicine*, 153:1–12, 2018.

[20] Cathrin Elich, Francis Engelmann, Theodora Kontogianni, and Bastian Leibe. 3d bird's-eye-view instance segmentation. In *German Conference on Pattern Recognition*, pages 48–61. Springer, 2019.

[21] Stefan Falkner, Aaron Klein, and Frank Hutter. BOHB: Robust and efficient hyperparameter optimisation at scale. In *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*, pages 1437–1446. PMLR, 2018.

[22] H. Fang et al. Queryinst: Instance segmentation with learnable queries. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(9):10798–10812, 2023.

[23] Nafiseh Ghasemi, Jon Alvarez Justo, Marco Celesti, Laurent Despoisse, and Jens Nieke. Onboard processing of hyperspectral imagery: Deep learning advancements, methodologies, challenges, and emerging trends. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2025. doi: 10.1109/JSTARS.2025.3528741.

[24] R. Gosselin, D. Rodrigue, and C. Duchesne. A hyperspectral imaging sensor for on-line quality control of extruded polymer composite products. *Computers & Chemical Engineering*, 35(12):2961–2968, 2011. doi: 10.1016/j.compchemeng.2010.10.013.

[25] John J Grefenstette. Optimisation of control parameters for genetic algorithms. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):122–128, 1986. doi: 10.1109/TSMC.1986.289288.

[26] W. He, S. Li, and J. Wang. Region growing with dynamic similarity constraints for crop field segmentation. *Computers and Electronics in Agriculture*, 218:106742, 2024. doi: 10.1016/j.compag.2023.106742.

[27] Moein Heidari, Amirhossein Kazerouni, Milad Soltany, Reza Azad, Ehsan Khodapanah Aghdam, Julien Cohen-Adad, and Dorit Merhof. Hiformer: Hierarchical multi-scale representations using transformers for medical image segmentation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 6202–6212, 2023.

[28] John H Holland. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press, 1992.

[29] E.H. Houssein, G.M. Mohamed, Y. Djenouri, et al. Nature inspired optimisation algorithms for medical image segmentation: a comprehensive review. *Cluster Computing*, 27:14745–14766, 2024. doi: 10.1007/s10586-024-04601-5.

[30] Ibrar Hussain and Jan Muhammad. Efficient convex region-based segmentation for noising and inhomogeneous patterns. *Inverse Problems and Imaging*, 17(3):708–725, 2023. doi: 10.3934/ipi.2023015.

[31] Frank Hutter, Holger H Hoos, Kevin Leyton-Brown, and Thomas Stützle. Paramils: An automatic algorithm configuration framework. *Journal of Artificial Intelligence Research*, 36:267–306, 2009.

[32] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. Sequential model-based optimisation for general algorithm configuration. In *Proceedings of the 5th International Conference on Learning and Intelligent Optimization (LION 5)*, pages 507–523. Springer, 2011.

[33] Isuwa Jeremiah, Mohammed Abdullahi, Sahabi Ali Yusuf, Muhammad Nuruddeen Idris, and Baffa Shuaibu Garko. Integration of specific local search methods in metaheuristic algorithms for optimising the feature selection process: A survey. *SLU J Sci Technol*, 4(1&2):34–48, 2022.

[34] Sen Jia, Shuguo Jiang, Zhijie Lin, Nanying Li, Meng Xu, and Shiqi Yu. A survey: Deep learning for hyperspectral image classification with few labeled samples. *Neurocomputing*, 448:179–204, 2021. doi: 10.1016/j.neucom.2021.03.035.

[35] C. Johnson. What is research in computing science? Computer Science Dept., Glasgow University. Electronic resource, 2016. URL https://www.dcs.gla.ac.uk/~johnson/teaching/research_skills/research.html. Accessed: 30 Oct 2016.

[36] Rehna Kalam and K Manikandan. Enhancing k-means algorithm for image segmentation. In *2011 International Conference on Process Automation, Control and Computing*, pages 1–4. IEEE, 2011.

[37] J N Kapur, P K Sahoo, and A K C Wong. A new method for gray-level picture thresholding using the entropy of the histogram. *Computer Vision, Graphics, and Image Processing*, 29(3):273–285, 1985.

[38] Dervis Karaboga. An idea based on honey bee swarm for numerical optimisation. Technical Report TR06, Erciyes University, Engineering Faculty, Computer Engineering Department, Kayseri, Turkey, 2005.

[39] Majid Karimi-Mamaghan and Mahdi Mohammadi. Multi-objective hyper-heuristic feature selection. *Swarm and Evolutionary Computation*, 69:100991, 2022.

[40] P. Kaur, J. Smith, and R. Brown. Cnn-driven region merging for organ segmentation. *Medical Image Analysis*, 95:102345, 2025.

[41] A. Kirillov et al. Panoptic feature pyramid networks. In *Proceedings of CVPR*, pages 6399–6408, 2019.

[42] F. Koerting, S. Asadzadeh, J. C. Hildebrand, E. Savinova, C. Hecker, and M. Kirsch. VNIR-SWIR imaging spectroscopy for mining: Insights for hyperspectral drone applications. *Mining*, 4(4):57, 2024. doi: 10.3390/mining4040057.

[43] F. A. Kruse and J. W. Boardman. Comparison of airborne and satellite hyperspectral data for geologic mapping. In *Imaging Spectrometry VIII*, volume 4725 of *Proceedings of SPIE*, pages 1–9, 2002. doi: 10.1117/12.478743.

[44] Vinod Kumar, Tauj Lal, Piyush Dhuliya, and Diwaker Pant. A study and comparison of different image segmentation algorithms. In *2016 2nd International Conference on Advances in Computing, Communication, & Automation (ICACCA) (Fall)*, pages 1–6. IEEE, 2016. doi: 10.1109/ICACCA.2016.7752680.

[45] Chandana Kumari and Abhijit Mustafi. A survey on novel hybrid metaheuristic algorithms for image segmentation. In *Computer Vision and Robotics (CVR 2023)*, pages 285–294, 2023. doi: 10.1007/978-981-99-4577-1_23.

[46] Jiss Kuruvilla, Dhanya Sukumaran, Anjali Sankar, and Siji P. Joy. A review on image processing and image segmentation. In *2016 International Conference on Data Mining and Advanced Computing (SAPIENCE)*, pages 198–203. IEEE, 2016. doi: 10.1109/SAPIENCE.2016.7684127.

[47] C. Laukamp, A. Rodger, M. LeGras, H. Lampinen, I. C. Lau, M. Pejcic, and M. C. Ramanaidou. Physicochemistry underlying feature-based extraction of mineral abundance and composition from shortwave, mid and thermal infrared reflectance spectra. *Minerals*, 11(4):347, 2021. doi: 10.3390/min11040347.

[48] Jiangyun Li, Yuanxiu Cai, Qing Li, Mingyin Kou, and Tianxiang Zhang. A review of remote sensing image segmentation by deep learning methods. *International Journal of Digital Earth*, 17(1):2328827, 2024. doi: 10.1080/17538947.2024.2328827.

[49] Ningyang Li, Zhaohui Wang, and Faouzi Alaya Cheikh. Discriminating spectral–spatial feature extraction for hyperspectral image classification: A review. *Sensors*, 24(10):2987, 2024. doi: 10.3390/s24102987.

[50] Q. Li, M. Gao, and Z. L. Li. Ground hyper-spectral remote-sensing monitoring of wheat water stress during different growing stages. *Agronomy*, 12(10):2267, 2022. doi: 10.3390/agronomy12102267. URL https://www.mdpi.com/2073-4395/12/10/2267.

[51] Qiang Li, Lei Zhang, and Rui Guo. Multi-scale edge detection with feature fusion. *Neurocomputing*, 275:2240–2251, 2018.

[52] X. Li, T. Wei, Y. P. Chen, Y. Tai, J. Luo, B. Wang, J. Li, and F. Huang. Hybrid segmentation model coupling CNN and marker-controlled watershed for overlapping chromosomes. *IEEE Journal of Biomedical and Health Informatics*, 27(1):332–343, 2023. doi: 10.1109/JBHI.2022.3211234.

[53] Siew Mooi Lim, Abu Bakar Md Sultan, Md Nasir Sulaiman, Aida Mustapha, and Kuan Yew Leong. Crossover and mutation operators of genetic algorithms. *International Journal of Machine Learning and Computing*, 7(1):9–12, 2017.

[54] Yun Liu, Ming-Ming Cheng, Xiaowei Hu, Kai Wang, and Xiang Bai. Richer convolutional features for edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(8):1939–1946, 2019. ArXiv preprint 2016, published 2019.

[55] A. Lou, S. Guan, and M. Loew. Dc-unet: rethinking the u-net architecture with dual channel efficient cnn for medical image segmentation. In *Medical Imaging 2021: Image Processing*, volume 11596, pages 758–768. SPIE, 2021.

[56] Helena R Lourenço, Olivier C Martin, and Thomas Stützle. Iterated local search. In *Handbook of Metaheuristics*, pages 320–353. Springer, 2003.

[57] Helena R. Lourenço, Olivier C. Martin, and Thomas Stützle. Iterated local search. *Handbook of Metaheuristics*, pages 320–353, 2010.

[58] Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Thomas Stützle, and Mauro Birattari. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.

[59] Bin Ma and Lei Chen. A hyper-heuristic framework for feature selection. *Knowledge-Based Systems*, 163:206–217, 2019.

[60] J. Ma, J. Chen, M. Ng, R. Huang, Y. Li, C. Li, X. Yang, and A. L. Martel. Loss odyssey in medical image segmentation. *Medical Image Analysis*, 71:102035, 2021. ISSN 1361-8415. doi: 10.1016/j.media.2021.102035.

[61] J. A. Martins, P. Gamba, and P. Ghamisi. Multi-objective genetic algorithm for hyperspectral image segmentation. *Neurocomputing*, 416:222–235, 2020. doi: 10.1016/j.neucom.2020.07.027.

[62] J. Mei, T. Zhou, K. Huang, et al. A survey on deep learning for polyp segmentation: techniques, challenges and future trends. *Visual Intelligence*, 3, 2025. doi: 10.1007/s44267-024-00071-w.

[63] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Keen, and D. Terzopoulos. Image segmentation using deep learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7): 3523–3542, 2022. doi: 10.1109/TPAMI.2021.3059968.

[64] M. Nasir, S. Das, D. Maity, et al. Convergence analysis of particle swarm optimisers. *Swarm and Evolutionary Computation*, 55:100680, 2019. doi: 10.1016/j.swevo.2019.100680.

[65] T. Nguyen, K. Lee, and S. Park. Fused canny edge detection with deep contour prediction for medical boundary delineation. *IEEE Transactions on Medical Imaging*, 43(5):1234–1245, 2024.

[66] Thambo Nyathi. Automated design of a genetic algorithm for image segmentation using the iterated local search. In *Proceedings of the 16th International Joint Conference on Computational Intelligence (ECTA)*, pages 189–196, 2024. doi: 10.5220/0012908500003837.

[67] B. Oates, M. Griffiths, and R. McLean. *Researching Information Systems and Computing*. SAGE Publications, 2022. ISBN 978-1526492662.

[68] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979. doi: 10.1109/TSMC.1979.4310076.

[69] Er Ouyang, Bin Li, Wenjing Hu, Guoyun Zhang, Lin Zhao, and Jianhui Wu. When multigranularity meets spatial–spectral attention: A hybrid transformer for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 61:1–15, 2023. doi: 10.1109/TGRS.2023.3242978.

[70] Bosoon Park, SC Yoon, KC Lawrence, and WR Windham. Fisher linear discriminant analysis for improving fecal detection accuracy with hyperspectral images. *Transactions of the ASABE*, 50(6): 2275–2283, 2007.

[71] R. Patel and A. Singh. Improved Otsu's thresholding using entropy-based fitness functions for medical imaging. *Journal of Medical Systems*, 47(4):45–56, 2023. doi: 10.1007/s10916-023-01937-4.

[72] A. Plaza, Q. Du, J. M. Bioucas-Dias, et al. A quantitative and comparative assessment of unmixing algorithms for hyperspectral data. *IEEE Transactions on Geoscience and Remote Sensing*, 59(2): 1556–1576, 2021. doi: 10.1109/TGRS.2020.2986866.

[73] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, volume 9351 of *Lecture Notes in Computer Science*, pages 234–241, Munich, Germany, 2015. Springer. doi: 10.1007/978-3-319-24574-4_28. URL https://arxiv.org/abs/1505.04597.

[74] Payel Roy, Saurab Dutta, Nilanjan Dey, Goutami Dey, Sayan Chakraborty, and Ruben Ray. Adaptive thresholding: A comparative study. In *2014 International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT)*, pages 1182–1186. IEEE, 2014.

[75] S. Saha and D. Roy. Spatially weighted k-means for urban feature extraction. *ISPRS Journal of Photogrammetry and Remote Sensing*, 188:234–247, 2025.

[76] Soham Sarkar, Swagatam Das, Susanta Chakraborty, and Sheli Sinha Chaudhuri. A differential evolution based approach for multilevel image segmentation using minimum cross entropy thresholding. *Proceedings of the International Conference on Swarm, Evolutionary, and Memetic Computing*, pages 51–58, 2013. doi: 10.1007/978-3-319-03756-1_6.

[77] A. Shah, H. Patel, and G. Chaudhari. An adaptive thresholding technique for document image segmentation under non-uniform illumination. *Journal of King Saud University - Computer and Information Sciences*, 35(2):572–582, 2023.

[78] Yuhui Shi and Russell C. Eberhart. A modified particle swarm optimizer. In *Proceedings of the IEEE World Congress on Computational Intelligence*, pages 69–73, Anchorage, AK, USA, 1998. ISBN 0-7803-4869-9. doi: 10.1109/ICEC.1998.699146.

[79] Alberto Signoroni, Mattia Savardi, Annalisa Baronio, and Sergio Benini. Deep learning meets hyperspectral image analysis: A multidisciplinary review. *Journal of Imaging*, 5(5):52, 2019. doi: 10.3390/jimaging5050052.

[80] Amit Singh and Neeraj Sharma. Hyper-heuristic approaches for image segmentation: A review. *Expert Systems with Applications*, 168:114221, 2021.

[81] Xavier Soria, Edgar Riba, and Angel Sappa. Dense extreme inception network for edge detection. In *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 2124–2133, 2020.

[82] P. Tatzer, M. Wolf, and T. Panner. Industrial application for inline material sorting using hyperspectral imaging in the NIR range. *Real-Time Imaging*, 11(2):99–107, 2005. doi: 10.1016/j.rti.2005.01.003.

[83] P. S. Thenkabail, R. B. Smith, and E. De Pauw. Hyperspectral vegetation indices and their relationships with agricultural crop characteristics. *Remote Sensing of Environment*, 71(2):158–182, 2000. doi: 10.1016/S0034-4257(99)00067-X. URL https://www.sciencedirect.com/science/article/pii/S003442579900067X.

[84] L. Tian, W. Zhao, and Q. Zhang. Improved k-means with fuzzy clustering and feature embedding for remote sensing. *Remote Sensing of Environment*, 304:114567, 2024.

[85] Claudio Urrea and Maximiliano Vélez. Advances in deep learning for semantic segmentation of low-contrast images: A systematic review. *Sensors*, 25(7):2043, 2025. doi: 10.3390/s25072043.

[86] Antoine Vacavant, Marie-Ange Lebre, Hugo Rositi, Manuel Grand-Brochier, and Robin Strand. New definition of quality-scale robustness for image processing algorithms, with generalized uncertainty modeling, applied to denoising and segmentation. In *International Workshop on Reproducible Research in Pattern Recognition*, pages 138–149. Springer, 2018.

[87] Peter J M Van Laarhoven and Emile H L Aarts. Simulated annealing. In *Simulated Annealing: Theory and Applications*, pages 7–15. Springer, 1987.

[88] M. A. Vega-Rodriguez, A. Santos-Diaz, and J. A. Gomez-Pulido. GPU-accelerated evolutionary computation for remote sensing. *IEEE Computational Intelligence Magazine*, 17(1):48–61, 2022. doi: 10.1109/MCI.2021.3123456.

[89] Jinfeng Wang, Qiming Huang, Feilong Tang, Jia Meng, Jionglong Su, and Sifan Song. Stepwise feature fusion: Local guides global. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 110–120. Springer, 2022.

[90] Ke Wang, Hainan Chen, Ligang Cheng, and Jian Xiao. Variational-scale segmentation for multispectral remote-sensing images using spectral indices. *Remote Sensing*, 14(2):326, 2022. doi: 10.3390/rs14020326.

[91] Sherrie Wang, William Chen, Sang Michael Xie, George Azzari, and David B. Lobell. Weakly supervised deep learning for segmentation of remote sensing imagery. *Remote Sensing*, 12(2):207, 2020. doi: 10.3390/rs12020207.

[92] Yibo Wang, Xin Zhao, and Jian Yang. Deep learning for edge detection: A survey. *Pattern Recognition*, 68:123–135, 2017.

[93] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 12077–12090, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/64f1f27bf1b4ec22924fd0acb550c235-Abstract.html.

[94] Saining Xie and Zhuowen Tu. Holistically-nested edge detection. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1395–1403, 2015.

[95] Y. Xie, Z. Zhang, and M. Li. Transunet v2, combining transformers and u-net. *Nature Machine Intelligence*, 7(2):156–168, 2025.

[96] Yifei Xie, Zhikun Tu, Tong Yang, Yuhe Zhang, and Xinyu Zhou. Edgeformer: Local patch-based edge detection transformer. *Pattern Analysis and Applications*, 28(1):11, 2025.

[97] Masahiro Yamaguchi, Hideaki Haneishi, and Nagaaki Ohyama. Beyond RGB: Spectrum-based color imaging technology. *Journal of Imaging Science and Technology*, 52(1):010201–010201–15, 2008. doi: 10.2352/J.ImagingSci.Technol.2008.52.1.010201.

[98] Anil Yaman, Giovanni Iacca, and Fabio Caraffini. A comparison of three differential evolution strategies in terms of early convergence with different population sizes. In *AIP Conference Proceedings*, volume 2070, page 020002. AIP Publishing LLC, 2019.

[99] Y. Yan, J. Ren, H. Zhao, J. F. C. Windmill, and J. He. Non-destructive testing of composite fiber materials with hyperspectral imaging—Evaluative studies in the EU H2020 FibreEUse project. *IEEE Transactions on Instrumentation and Measurement*, 71:1–12, 2022. doi: 10.1109/TIM.2022.3154099.

[100] Y. Yan, J. Ren, H. Sun, and R. Williams. Nondestructive quantitative measurement for precision quality control in additive manufacturing using hyperspectral imagery and machine learning. *IEEE Transactions on Industrial Informatics*, 20(6):8312–8323, 2024. doi: 10.1109/TII.2024.3410645.

[101] Xin-She Yang. *Nature-inspired optimisation algorithms*. Elsevier, Cambridge, MA, 2020. ISBN 978-0128197145.

[102] Yunfan Ye, Kai Xu, Yuhang Huang, Renjiao Yi, and Zhiping Cai. Diffusionedge: Diffusion probabilistic model for crisp edge detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[103] H. Yu, B. Kong, Y. Hou, X. Xu, T. Chen, and X. Liu. A critical review on applications of hyperspectral remote sensing in crop monitoring. *Experimental Agriculture*, 58(1):1–19, 2022. doi: 10.1017/S0014479721000381. URL https://www.cambridge.org/core/journals/experimental-agriculture/article/critical-review-on-applications-of-hyperspectral-remote-sensing-in-crop-monitoring/93D5F5C4A6ABE877ECDF99DE3ED9B306.

[104] F. Zhang and G. Zhou. Estimation of vegetation water content using hyperspectral vegetation indices: A comparison of crop water indicators in response to water stress treatments. *BMC Ecology*, 19(1): 18, 2019. doi: 10.1186/s12898-019-0233-0. URL https://link.springer.com/article/10.1186/s12898-019-0233-0.

[105] Fan Zhang and Rachel Chancellor. Application of a statistical approach when data is limited in pipeline integrity management. In *Pressure Vessels and Piping Conference*, volume 89084, page V05AT06A061. American Society of Mechanical Engineers, 2025.

[106] H. Zhang, X. Wang, and Y. Liu. Adaptive thresholding for satellite image segmentation with deep statistical models. *IEEE Transactions on Geoscience and Remote Sensing*, 62:5602312, 2024.

[107] L. Zhang, L. Zhang, D. Tao, et al. Hyperspectral band selection using improved particle swarm optimisation. *IEEE Transactions on Geoscience and Remote Sensing*, 58(8):5478–5492, 2020. doi: 10.1109/TGRS.2019.2948913.

[108] L. Zhang, K. Chen, and H. Wu. Threshold + transformer pipeline for medical images. *Medical Image Analysis*, 97:102567, 2025.

[109] Y. Zhang, G. Ding, J. Li, et al. Particle swarm optimisation for band selection in hyperspectral imagery. In *Proceedings of the IEEE International Conference on Geoscience and Remote Sensing Symposium*, 2018.

[110] S. Zheng, J. Lu, H. Zhao, X. Zhu, Z. Luo, Y. Wang, Y. Fu, J. Feng, T. Xiang, and P.H.S. Torr. Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6881–6890, 2021.

[111] S. Zheng et al. Panoptic segment anything: Unified transformer-based segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(6):4521–4538, 2025.

[112] M. Zhou, B. Thompson, and R. Davis. Pre-trained cnns to generate markers for watershed segmentation. *Pattern Recognition Letters*, 175:45–53, 2023.